

Programming using Java

Java Basics

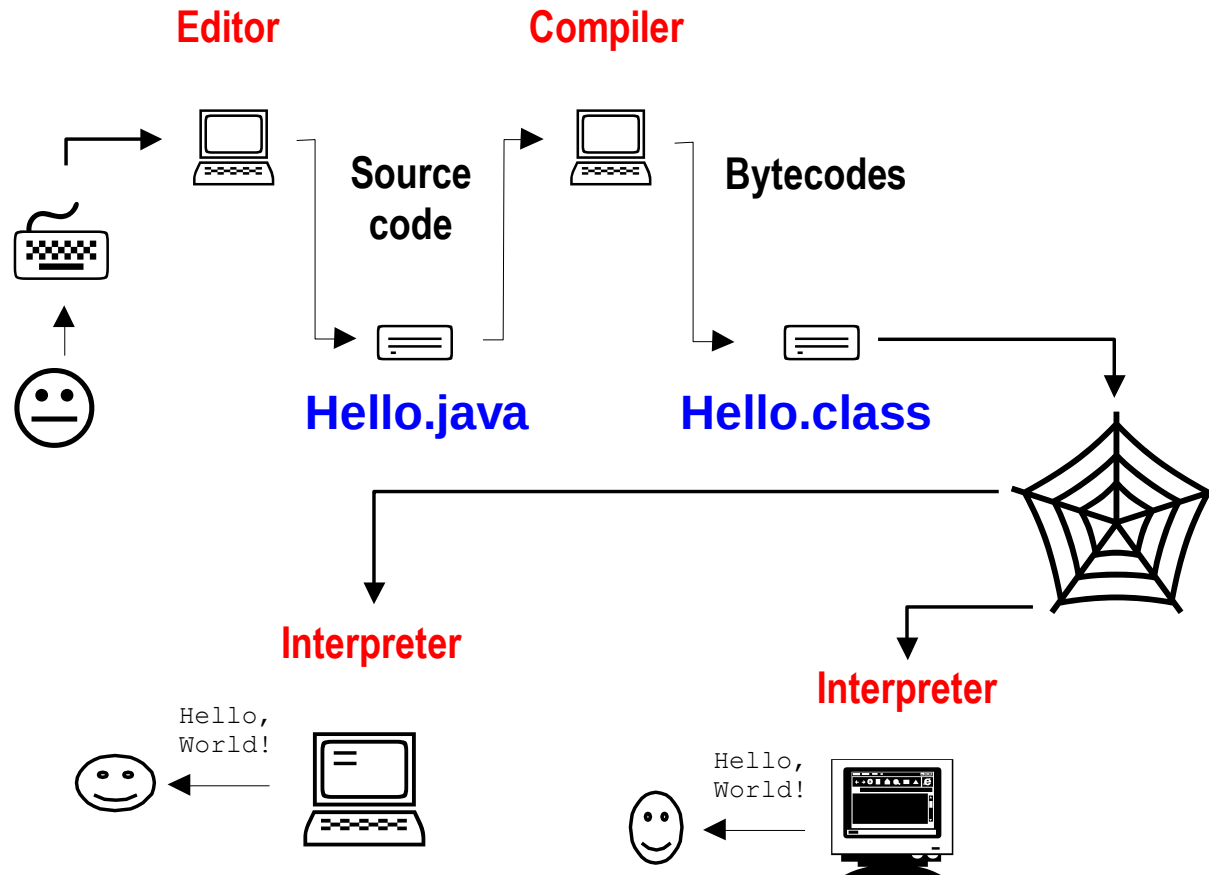
Java Technology

There is more to Java than the language.

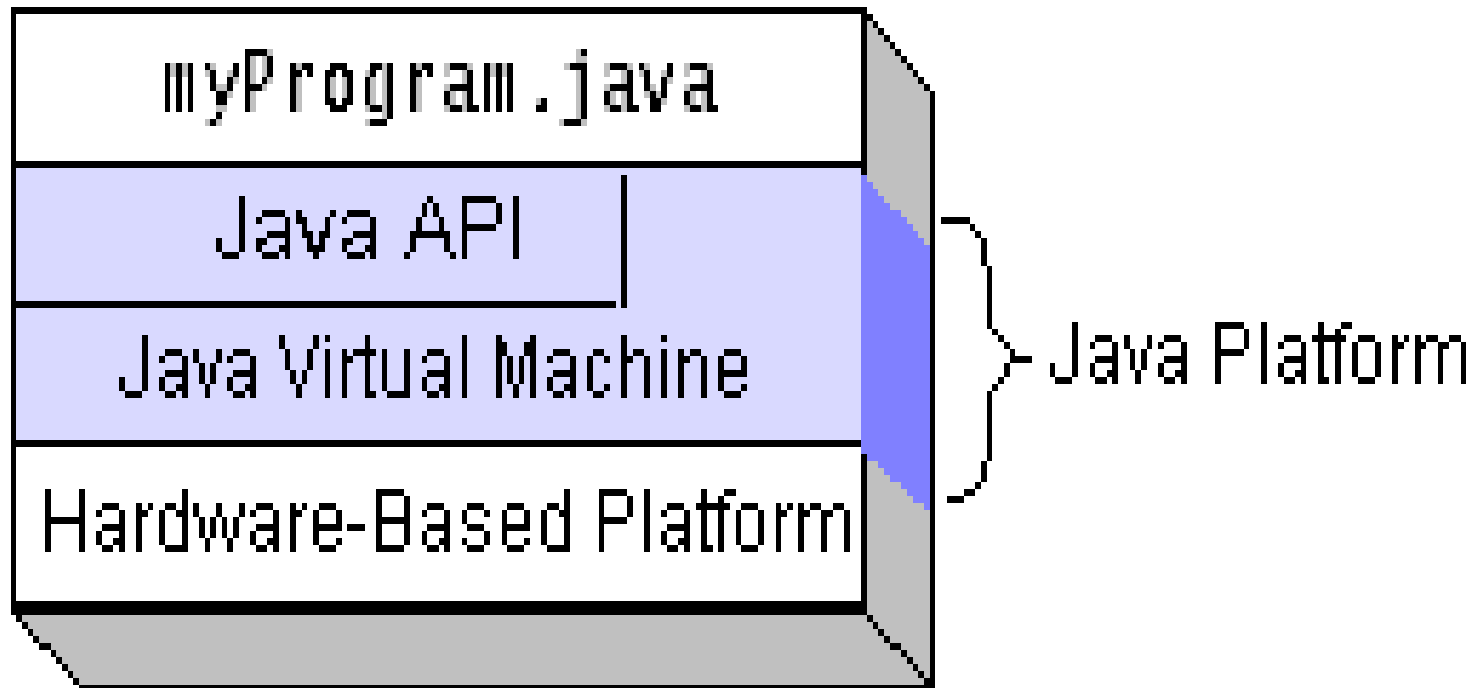
Java Technology consists of:

- 1) Java Programming Language
- 2) Java Virtual Machine (JVM)
- 3) Java Application Programming Interfaces (APIs)

Java's Compiler + Interpreter



Java Execution Platform



Start a Java Application

Java

C

“Hello.java”

Class Name

```
class Hello {
```

Main method

```
    public static void main(String[] args) {  
        System.out.println(“Hello World”);  
    }
```

“hello.c”

Main Function

```
void main() {  
    printf(“Hello  
World\n”);  
}
```

> javac Hello.java

Compile

> cc hello.c -o hello

> java Hello

Run

> hello

output

Hello World

Java Virtual Machine

- The .class files generated by the compiler are not executable binaries
 - so Java combines compilation and interpretation
- Instead, they contain “byte-codes” to be executed by the Java Virtual Machine
- This approach provides platform independence, and greater security

Lab on 19/22 Aug: Code Template to be used:

```
import java.util.*;

public class PatternPrinting {

    public static void pattern(int n)
    { //write your code here

        //Output on screen    System.out.println("    ");

    }

    // Driver code
    public static void main(String[] args)
    {
        Scanner sc= new Scanner(System.in);
        System.out.print("Enter number ");
        int n= sc.nextInt();
        pattern(n); }

}
```

Comments are almost like C

```
/* This kind of comment can span  
multiple lines  
*/
```

```
//This kind is to the end of the line
```

```
/**  
 * This kind of comment is a special  
 * 'javadoc' style comment  
 */
```


Primitive data types are like C

- Main data types are `int`, `double`, `boolean`, `char`, `byte`, `short`, `long`, `float`
- **`boolean`** has values **`true`** and **`false`**
- Declarations look like C, for example,
 - `double x, y;`
 - `int count = 0;`

Variables and Assignments

■ Variables types

- **char** 16 bits Unicode character data
- **boolean** Boolean Variable
- **byte** 8 bits signed integer
- **short** 16 bits signed integer
- **int** 32 bits signed integer
- **long** 64 bits signed integer
- **float** 32 bits signed floating point number
- **double** 64 bits signed floating point number

Variables and Assignments

Variable Declaration

```
type varName;
```

```
float x, y, x;
```

Value assignments

```
varName = value;
```

```
int num = 100;
```

```
long m = 21234234L
```

```
double type : .5 0.8 9e-2 -9.3e-5
```

```
float type : 1.5e-3f;
```

Strings and Characters

- String : sequence of character

```
String s = "Enter an integer value: " ;
```

```
Char c = 'A';
```



Unicode

- Concatenation Operator '+'

```
String s = "Lincoln said: " + "\" Four score ago\"";
```

Result :

Lincoln said: "Four score ago"

Expressions are like C

- Assignment statements mostly look like those in C
- you can use `=`, `+=`, `*=` etc.
- Arithmetic uses the familiar `+` `-` `*` `/` `%`
- Java also has `++` and `--`
- Java has boolean operators `&&` `||` `!`
- Java has comparisons `<` `<=` `==` `!=` `>=` `>`
- Java does **not have pointers** or pointer arithmetic

Arithmetic Operators

■ Operators

➤ + - * / %

➤ += -= *= /= %=

➤ ++ --

5 / 2 → 2 Why isn't it 2.5 ?

5 % 2 → 1

4 / 2 → 2

4 % 2 → 0

➤ count * num + 88 / val - 19 % count

➤ j += 6; ➔ j = j + 6;

?

Type Conversions in Expression

```
char ch;    int i;    float f;    double outcome;
```

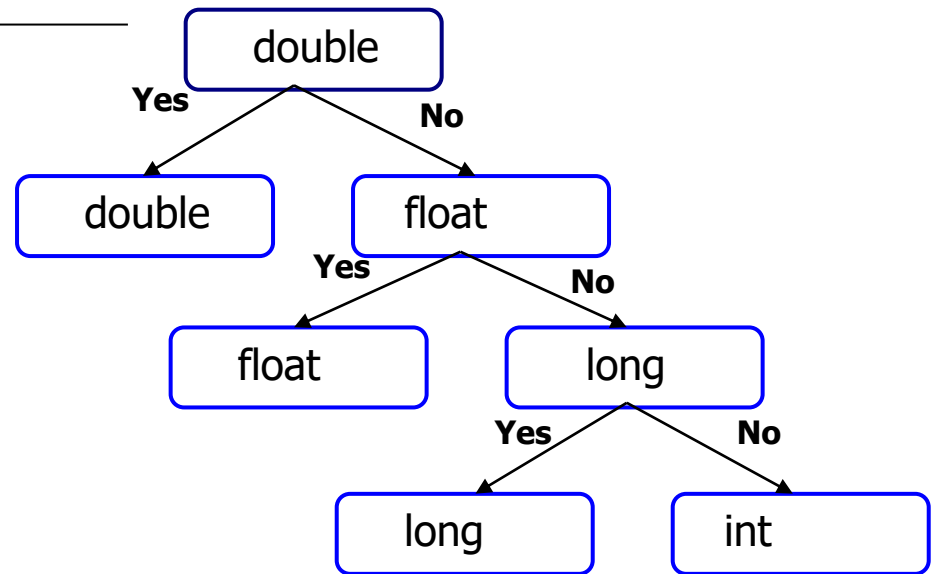
```
ch = '0';    i = 10;    f = 10.2f;
```

outcome = ch * i / f;

int

float

double



Type Conversions in Assignment

- Widening Conversion

```
byte b = 127;
```

```
int i;
```

```
i = b;
```

- Wrong Conversion

```
byte b;
```

```
int i = 127;
```

```
b = i;
```

- Conversion (But data may be lost)

```
byte b;
```

```
int i = 127;
```

```
b = (byte) i;
```

Type Casting

Increment & Decrement Operator

■ Operator

➤ ++, --

➤ Prefix operator

➤ Postfix operator

➤ Cannot be use with expressions, only with variables

➤ Cannot be applied to the real types

Example:

java

Copy code

```
int x = 5;  
int y = ++x; // x becomes 6, then y = 6
```

Result:

- x = 6
- y = 6

2. Postfix (x++ or x--)

The expression is evaluated first, then the value changes.

Example:

java

Copy code

```
int x = 5;  
int y = x++; // y = 5, then x becomes 6
```

Result:

- x = 6
- y = 5

```
n = 1;
```

```
x = ++n; // x=2, n=2
```

```
n = 1;
```

```
x = n++; // x=1, n=2
```

```
(a + b)++ // error
```

Back-Slash Codes

`\b` back space
`\n` new line
`\r` carriage return
`\f` form feed
`\t` tab
`\”` double quotation
`\’` single quotation
`\0` null
`\\` back slash
`\uxxxx` Unicode (x: hexadecimal)

```
class SpecialCharacters {  
    public static void  
    main(String args[]) {  
  
        System.out.print("\u00a0  
\u00a1 \u00a2 \u00a3");  
    }  
}
```

Bitwise Operators

- Operator
 - $\&$, $|$, $\ll$, $\gg$, $\ggg$, $\wedge$, $\sim$
 - Operand should be integer type
 - Precedence

Operator	Precedence
$\sim$ $\ll$ $\gg$ $\ggg$ $\&$ $\wedge$ $ $	(H) ↑ ↓ (L)

Ternary Operator

■ Operator

➤ Expr1 ? Expr2 : Expr3 (3 Terms Operator)

```
max = x > y ? x : y ;
```

```
if (x > y) max = x;  
else max = y;
```

```
m = a > b ? (c > a ? c : a) : (c > b ? c : b) ;
```

Assignment Operators

Expr 1 = Expr 1 op Expr2



Expr1 op= Expr 2

■ Operator

➤ Arithmetic operator : + - * / %

➤ Bitwise operator : & | ^ << >> >>>

sum = sum + i ;



sum += i ;

x = x * y + 1;



x *= y + 1;

x = x * (y+1)

Cast Operators

■ Data Type Casting Operator

(Data Type) Expression

➤ Cast operator : ()

(int) 3.75	====>	3
(float) 3	====>	3.0
(float) (1 / 2)	====>	0.0
(float) 1/2	====>	0.5

Control statements are like C

- `if (x < y) smaller = x;`
- `if (x < y) {
 smaller = x; sum += x;
else { smaller = y; sum += y; }`
- `while (x < y) {
 y = y - x; }`
- `do { y = y - x; }
 while (x < y)`
- `for (int i = 0; i < max; i++) sum += i;`
- **BUT: conditions must be boolean !**

Control statements II

```
switch (n + 1) {  
    case 0: m = n - 1; break;  
    case 1: m = n + 1;  
    case 3: m = m * n; break;  
    default: m = -n; break;  
}
```

- Java also introduces the **try** statement, about which more later

Java Keywords

■ 50 Java Keywords

abstract	double	int	super
boolean	else	interface	switch
break	extends	long	synchronized
byte	final	native	this
case	finally	new	throw
catch	float	package	throws
char	for	private	transient*
class	goto*	protected	try
const*	if	public	void
continue	implements	return	volatile
default	import	short	while
do	instanceof	static	strictfp
assert (New in 1.5)		enum (New in 1.5)	

Java isn't C!

- In C, almost everything is in functions
- In Java, almost everything is in classes
- There is often only one class per file
- There *must* be only one **public** class per file
- The file name *must* be the same as the name of that public class, but with a **.java** extension

THE JAVA BUZZWORDS

- Object oriented
- Network Savvy
- Robust
- Secure
- Architecture Neutral
- Portable
- Interpreted
- High Performance
- Multithreaded
- Dynamic

Programming using Java

Java Classes and Objects
Arrays

Objectives:

- Get an introduction to Classes and Objects
- Get a general idea of how a small program is put together

Simple Java Program

A class to display a simple message:

```
public class Hello {  
    public static void main(String[] args)  
    {  
        System.out.println("Hello World!");  
    }  
}
```

Running the Program

Type the program, save as
Hello.java.

In the command line, type:

```
> ls
```

```
Hello.java
```

```
> javac Hello.java
```

```
> ls
```

```
Hello.java, Hello.class
```

```
> java Hello
```

```
Hello World!
```


Explaining the Process

- 1) creating a source file - a source file contains text written in the Java programming language, created using any text editor on any system.
- 2) compiling the source file Java compiler (javac) reads the source file and translates its text into instructions that the Java interpreter can understand. These instructions are called bytecode.
- 3) running the compiled program Java interpreter (java) installed takes as input the bytecode file and carries out its instructions by translating them on the fly into instructions that your computer can understand.

Java Program Explained

Hello is the name of the class:

```
class Hello{
```

main is the method with one parameter args and no results:

```
public static void main(String[] args) {
```

println is a method in the standard System class:

```
System.out.println("First Java program.");  
}}
```

Main Method

The main method must be present in every Java application:

public static void main(String[] args)

- a) public means that the method can be called by any object
- b) static means that the method is shared by all instances
- c) void means that the method does not return any value

Interpreter executes an application-calling its main method

The main method accepts a single argument – a string array- command-line parameters.

public class *SomeClass*

{

□ *Fields*

Attributes / variables that define the object's state. Can hold numbers, characters, strings, other objects. Usually **private**.

□ *Constructors*

Code for constructing a new object and initializing its fields. Usually **public**.

□ *Methods*

Actions that an object can take. Can be **public** or **private**.

}

private: visible only inside this class
public: visible in other classes

```
public class Fraction
```

```
{
```

```
    private int num, denom;
```

Fields

```
    public Fraction ()
```

```
{
```

```
        num = 0;
```

```
        denom = 1;
```

```
}
```

Constructor

The name of a constructor is always the same as the name of the class.

```
    public int getNum ()
```

```
{
```

```
        return num;
```

```
}
```

Methods

```
}
```

Fields

Fields

You name it!

`private` (or `public`) `[static]` `[final]`

Usually
private

`datatype name;`

May be present:
means the field is
shared by all objects
in the class

May be present:
means the field
is a constant

`int`, `double`, etc., or an
object: `String`, `JButton`,
`FallingCube`, `Timer`

Field

□ Access Modifier

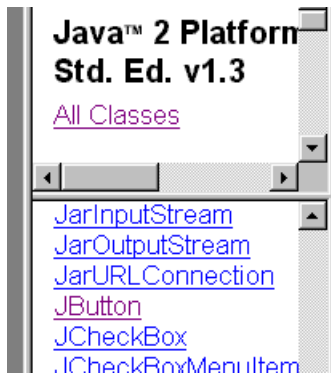
- Access Permission Level from other classes
- public, protected, private
- *Default/no declaration*: package

Modifier	Class	Sub-Class	Same Package	All Class
private	O	X	X	X
package	O	X	O	X
protected	O	O	O	X
public	O	O	O	O

Constructors

Constructors

- Constructors are like methods for **creating objects** of a class. Have the same name as the class.
- Most constructors initialize the object's fields.
- Constructors may take parameters.
- A class may have several constructors that differ in the number or types of their parameters.
- Default constructor- If no other constructor.



Constructors

Constructor Summary

[JButton](#) ()

Creates a button with no set text or icon.

[JButton](#) ([Action](#) a)

Creates a button where properties are taken from the Action supplied.

[JButton](#) ([Icon](#) icon)

Creates a button with an icon.

[JButton](#) ([String](#) text)

Creates a button with text.

[JButton](#) ([String](#) text, [Icon](#) icon)

Creates a button with initial text and an icon.

`go = new JButton("Go");`

Constructors (cont'd)

```
public class Fraction
{
    private int num, denom;

    public Fraction ( )
    {
        num = 0;
        denom = 1;
    }

    public Fraction (int n)
    {
        num = n;
        denom = 1;
    } 
```

"No-args"
constructor



```
    public Fraction (int p, int q)
    {
        num = p;
        denom = q;
        reduce ();
    }

    public Fraction (Fraction other)
    {
        num = other.num;
        denom = other.denom;
    }
    ...
}
```

Copy
constructor

Constructors

- Constructors of a class can call each other using the keyword **this** — a good way to avoid duplicating code:

```
public class Fraction
```

```
{
```

```
    ...
```

```
    public Fraction (int n)
```

```
    {
```

```
        this (n, 1);
```

```
    }
```

```
    ...
```

```
    ...
```

```
    public Fraction (int p,  
    int q)
```

```
    {
```

```
        num = p;
```

```
        denom = q;
```

```
        reduce ();
```

```
    }
```

```
    ...
```

Operator new

- Constructors are invoked using new

Fraction f1 = new Fraction (); ————| 0 / 1

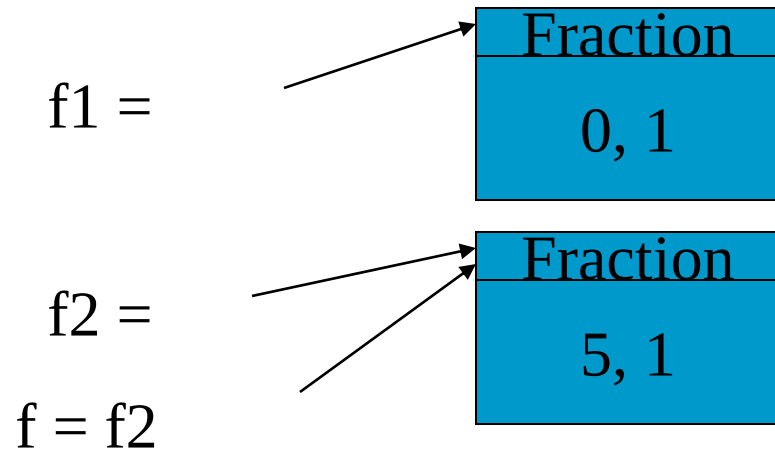
Fraction f2 = new Fraction (5); —————| 5 / 1

Fraction f3 = new Fraction (4, 6); ————| 2 / 3

Fraction f4 = new Fraction (f3); ————| 2 / 3

Object references

- A variable holds a memory location of an object.



null references

- The null value can be assigned to a variable of any reference type
- Ex:
 - `String middleInitial = null;`
 - `Car c = null;`
- Compare objects
 - `if (c == null).....`

Methods

Methods

- Call them for a particular object:

```
frac.getNum();
```

But call *static* (“*class*”) *methods* for the whole class, not a specific object:

```
y = Math.sqrt (x);
```

Methods: Java Style

- Method names start with lowercase letters.
- Method names usually sound like verbs.
- The name of a method that returns the value of a field often starts with **get**:
`getWidth`, `getX`

The name of a method that sets the value of a field often starts with **set**:
`setLocation`, `setText`

Method

□ Method Qualifier

– Access Modifier

- Access Permission Level to Method from Other Class
- Same as that of access modifier in field

– **static**

- static method, class method
- Same role of Global function
- Use only the static field of correspond class or the static method
- Can be referred by only class name

```
ClassName.methodName;
```

Method

– **final**

- Final method
- Method which cannot be redefined in subclass

– **synchronized**

- Synchronization method
- Control the threads so that only one thread can always access the target

– **native**

- To use the implementation written in other programming languages such as C language

Overloaded Methods

- Methods of the same class that have the same name but different numbers or types of arguments are called ***overloaded methods***.
- Use overloaded methods when they perform similar tasks:

```
public void move (int x, int y) { ... }  
public void move (double x, double y)  
{ ... }  
public void move (Point p) { ... }
```

```
public Fraction add (int n) { ... }  
public Fraction add (Fraction other) {  
... }
```

Static

Static Fields

- A *static* field (a.k.a. *class field* or *class variable*) is shared by all objects of the class.
- A static field can hold a constant shared by all objects of the class:
- A *non-static field* (a.k.a. *instance field* or *instance variable*) belongs to an individual object.

Static Fields

- Static fields are stored with the class code, separately from non-static fields that describe an individual object.
- public static fields, usually **global constants**, are referred to in other classes using “dot notation”:
`ClassName.constName`

```
double area = Math.PI * r * r;  
setBackground(Color.blue);  
c.add(btn, BorderLayout.NORTH);  
System.out.println(area);
```

Static Methods

- Static methods can access and manipulate a class's static fields.
- Static methods cannot access non-static fields or call non-static methods of the class.
- Static methods are called using “dot notation”: `ClassName.statMethod(...)`

```
double x = Math.random();  
double y = Math.sqrt(x);  
System.exit();
```

Instance Methods

- Non-static methods are also called *instance methods*.
- An instance method is called for a particular object using “dot notation”:

`objName.instMethod(...);`

- Instance methods can access ALL fields and call ALL methods of their class — both class and instance fields and methods.

Static (Class) vs. Non-Static (Instance)

```
public class MyClass
```

```
{
```

```
    public static final int statConst;
```

```
    private static int statVar;
```

```
    private int instVar;
```

```
    ...
```

```
    ...
```

```
    public static int statMethod(...)
```

```
    {
```

```
        statVar = statConst;
```

```
        statMethod2(...);
```

```
        instVar = ...;
```

```
        instMethod(...);
```

```
    }
```



All OK

OK

Error!



```
    public int instMethod(...)
```

```
    {
```

```
        statVar = statConst;
```

```
        inst Var = statConst;
```

```
        instVar = statMethod(...);
```

```
        statVar = instMethod2(...);
```

```
        ...
```

```
    }
```

```
    public int instMethod2(...)
```

```
    {
```

```
        ...
```

```
    }
```

```
    ...
```

```
}
```

Static vs. Non-Static (cont'd)

- Note: `main` is `static` and therefore cannot access non-static fields or call non-static methods of its class:

```
public class Hello
{
    private String message = "Hello,
World";

    public static void main (String[ ]
args)
    {
        System.out.println (message);
    }
}
```

Error:
non-static
variable
`message` is
used in static
context (`main`)



Static Initialization Statement

- The Statement to be executed at the same time when the system initialize the static variable in the class
- From

```
static { <statement> }
```

Static Initialization Statements

```
public class Test {
```

```
    static{
```

```
        System.out.println("Static");
```

```
    }
```

```
    {
```

```
        System.out.println("Non-static block");
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Test t = new Test();
```

```
        Test t2 = new Test();
```

```
    }
```

```
}
```

Static

Non-static block

Non-static block

Static Initialization Statements

```
public class Main{
    //instance variable initializer
    String s = "abc";
    //constructor
    public Main() {
        System.out.println("constructor called");    }
    static int i, j=4;
    //static initializer
    static {
        System.out.println("static initializer called");
        System.out.println(j);    }
    //instance initializer
    { System.out.println("instance initializer called"); }
    public static void main(String[] args) {
        new Main();
        new Main();    }
}
```

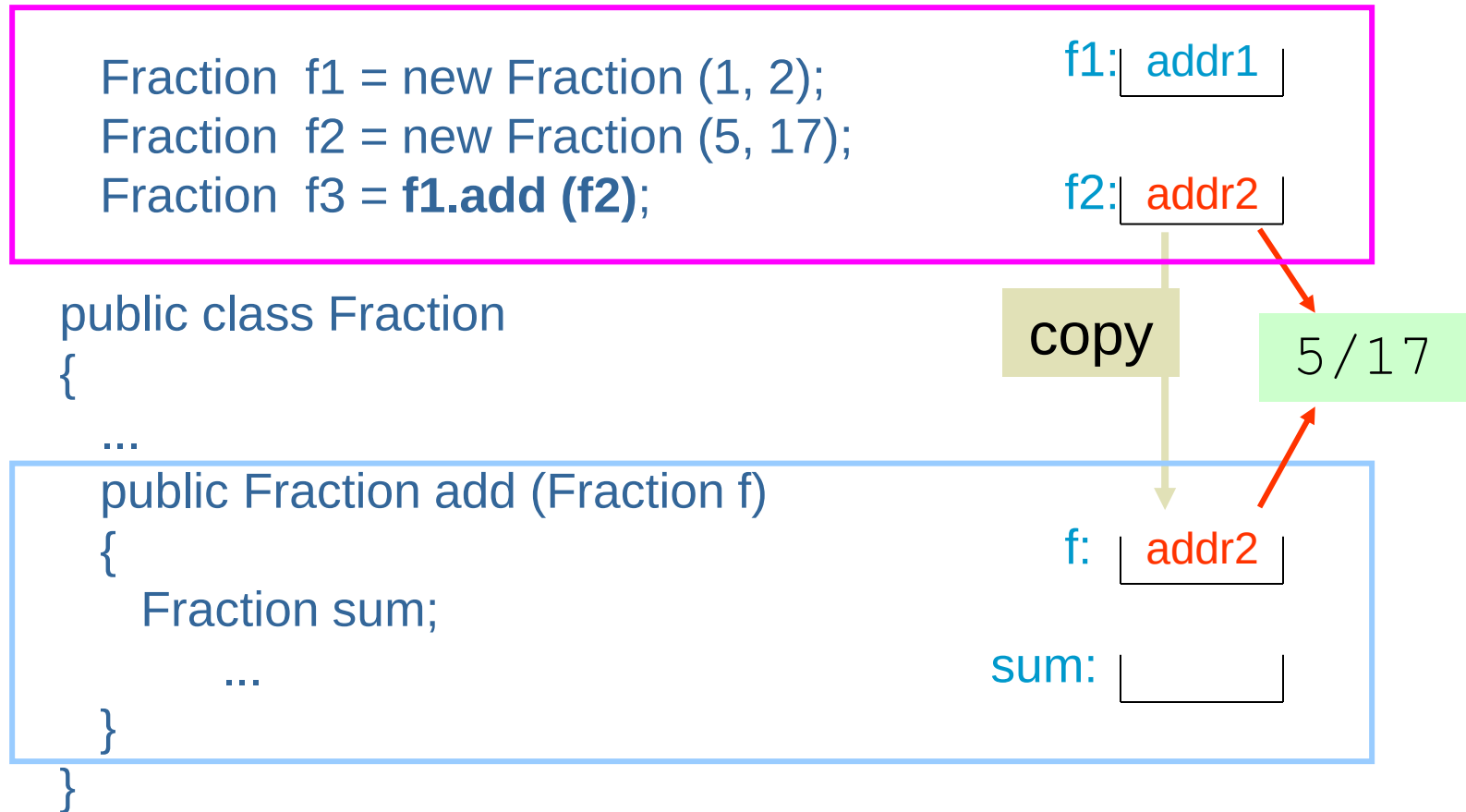
Output:

static initializer called
4

instance initializer called
constructor called
instance initializer called
constructor called

Passing Objects as Arguments

- Objects are always passed as references: the address is copied, not the object.



import

- Full library class names include the package name. For example:

```
java.awt.Color  
javax.swing.JButton
```

- `import` statements at the top of your program let you refer to library classes by their short names:

```
import javax.swing.JButton;  
...  
  
JButton go = new JButton("Click here");
```

*Fully-qualified
name*

import (cont'd)

- You can import names for all the classes in a package by using a wildcard `.*`:

```
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;
```

Imports all classes from `awt`, `awt.event`, and `swing` packages

- `java.lang` is imported automatically into all classes; defines `System`, `Math`, `Object`, `String`, and other commonly used classes.

Fraction- Methods

```
public class Fraction
{
    private int num, denom;

    public Fraction reduce( )
    {

    }

    public Fraction invert(Fraction a){

    }
```



```
    public Fraction (int p, int q)
    {
        num = p;
        denom = q;
        // reduce
    }

    public Fraction add(Fraction a,
        Fraction b){

    }

    public Fraction add (Fraction a, int n)
    { ... }
    }
```

Fraction Methods

```
public class Fraction
```

```
{ int num, denom;  
  public Fraction (int p, int q)  
  {  
    num = p;  
    denom = q;  
  }  
}
```

```
public add ( Fraction f)...
```

```
public Fraction invert (Fraction f)...
```

```
}
```

```
class FractionDemo {  
  public static void main(String args[]) {  
    //Create 3 Fraction objects,  
    // initialize 2 with 4 hardcoded values  
    (i,j,k,l)  
    // Print them on screen  
  
    // add the two Fraction objects  
    and assign it to third Fraction  
  
    //invert method should verify if  
    numerator is zero.  
  }  
}
```

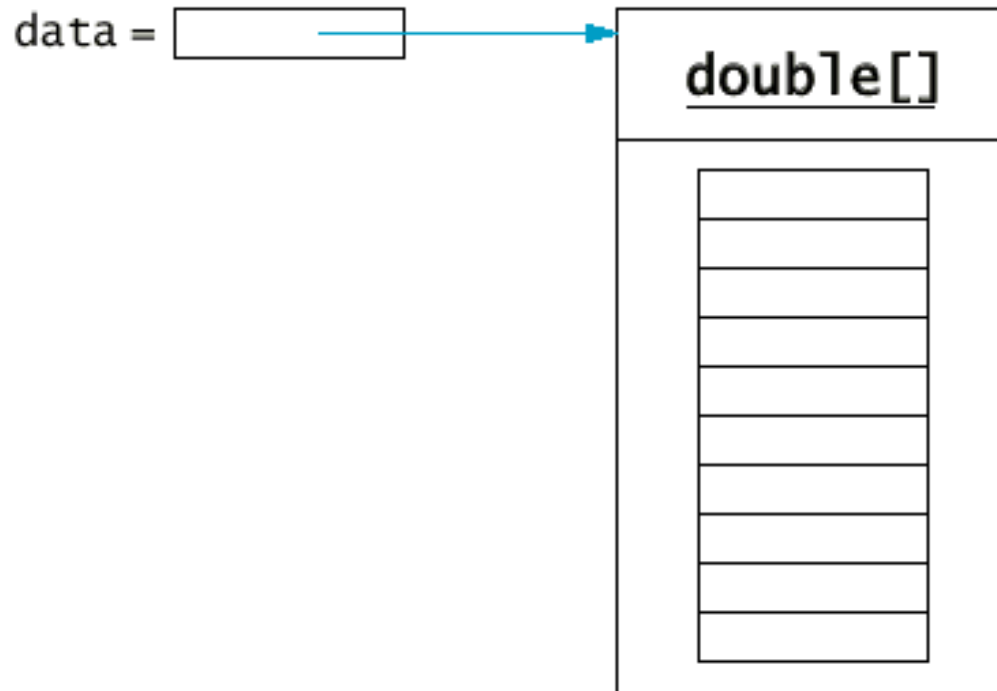
Java Arrays

Introduction

- Array is a useful and powerful aggregate data structure presence in modern programming languages
- Arrays allow easy access and manipulation to the values/objects that they store
- Arrays are indexed by a sequence of integers

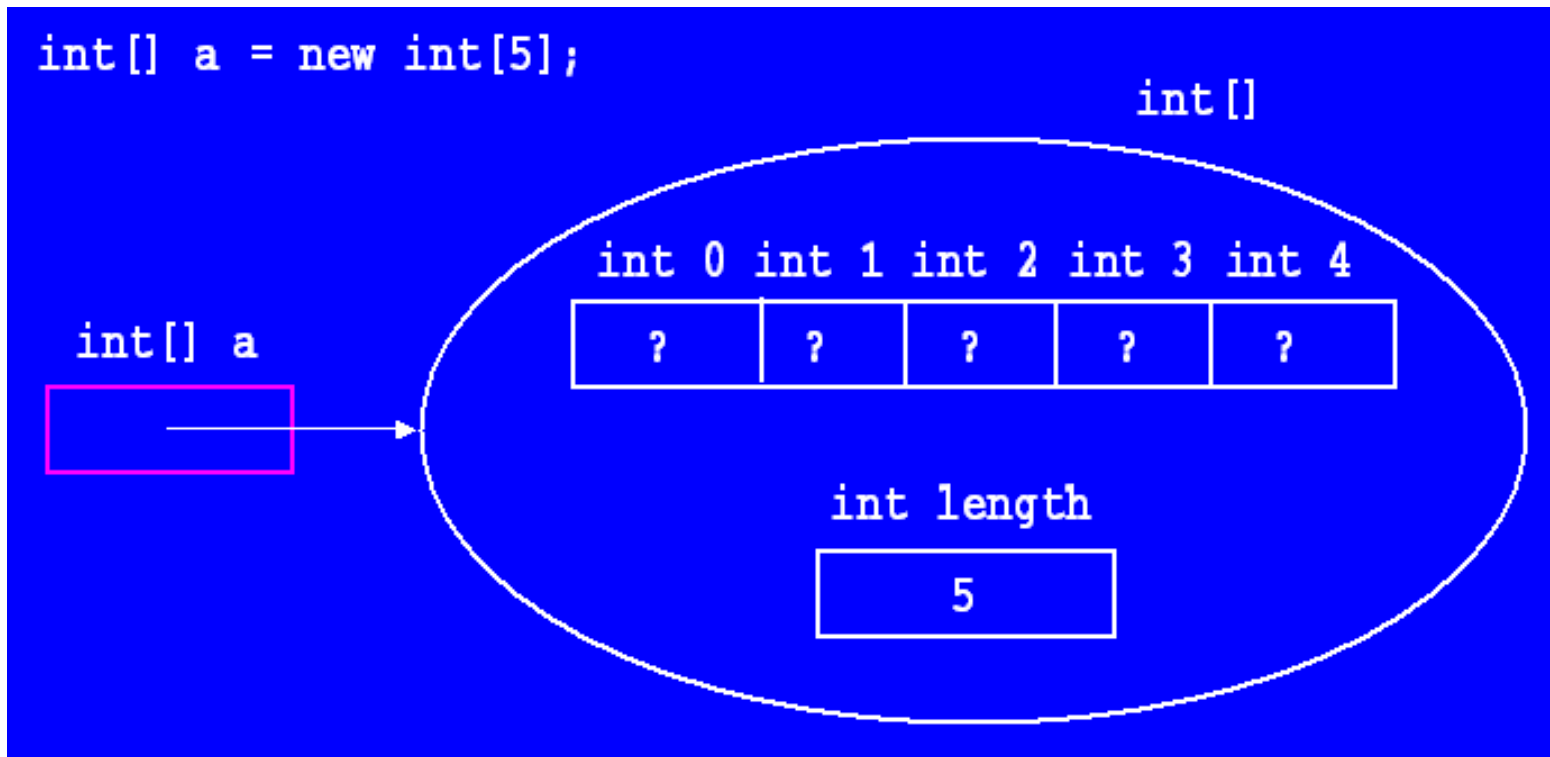
Arrays

- new is used to construct a new array:
new double[10]
- Store 10 double type variables in an array of doubles
double[] data = new double[10];



integer Arrays

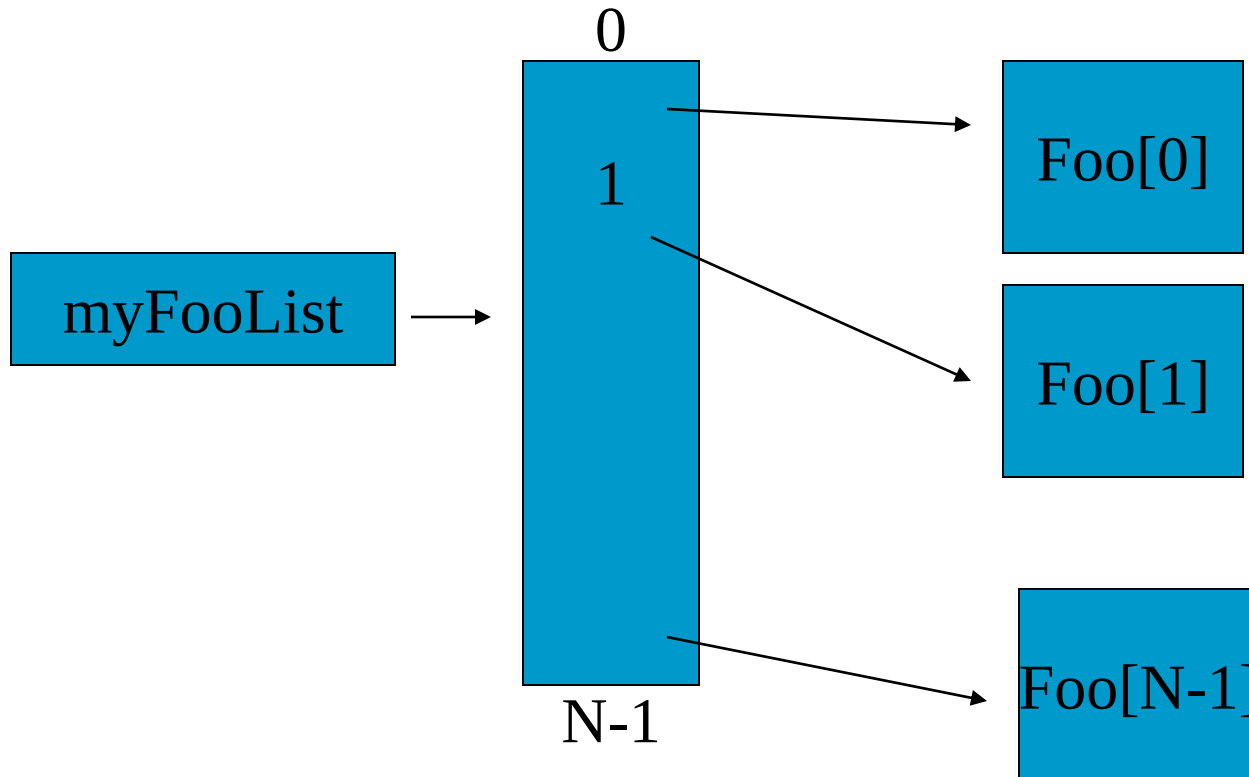
int[] a = new int[5];



Array of Object References

```
class Foo() { ....}
```

```
Foo[ ] myFooList = new Foo[N];
```



Arrays

- *fixed length*
- Element of specific type or references to Objects
- [] is used to access array elements
data[4] = 29.95;
- Use length **attribute** to get array length.
 - data.length
 - (Not a method!)

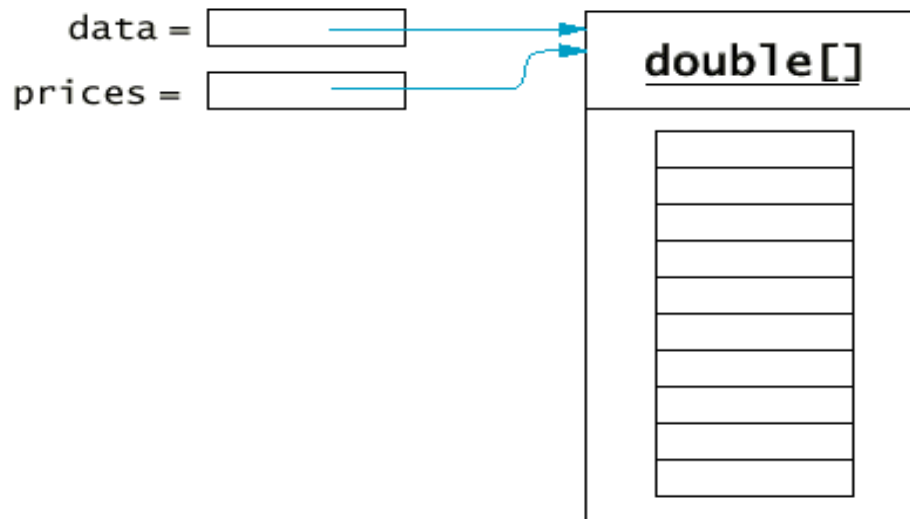
Array

- homogeneous data structure: each of its members stores the same type (either primitive or reference)
- indices go from 0 to one less than the length of the array
- each array object stores a **public final int length** instance variable that stores the length of the array
- we can access the value stored in this field, in the example above, by writing **a.length**

Copying Arrays

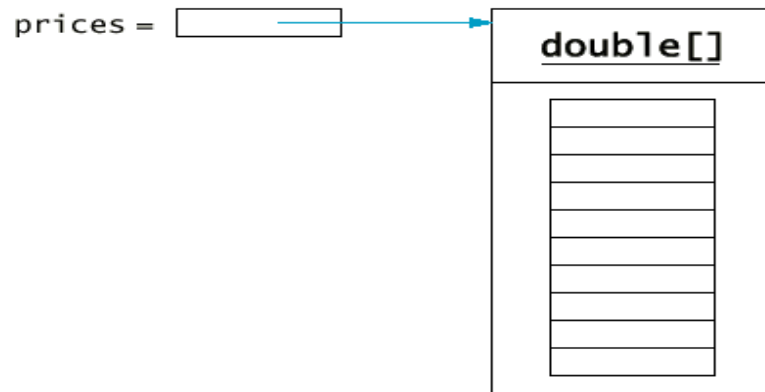
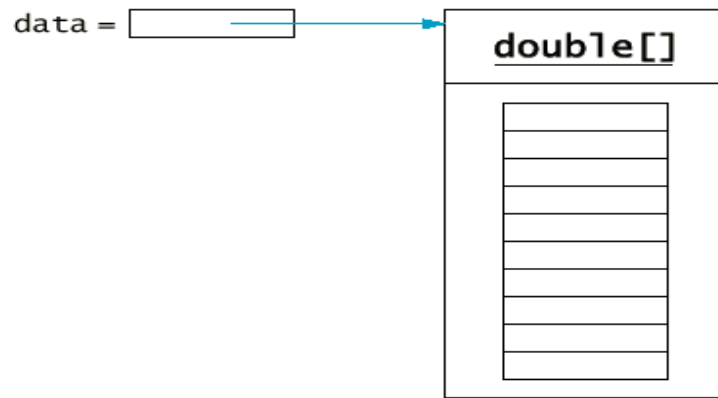
Copying an array reference yields a second reference to the same array

```
double[] data = new double[10];  
    // fill array . . .  
double[] prices = data;
```



Cloning Arrays

- Use `clone` to make true copy
`double[] prices = (double[])data.clone();`



Swapping Array Elements

- Suppose you want to swap two elements in the array, say entries with indices i and j .
Assuming we are dealing with an array of ints
 - `int temp = A[i];` // save a copy of $A[i]$ in temp
 - `A[i] = A[j];` // copy the content of $A[j]$ to $A[i]$
 - `A[j] = temp;` // copy the content of temp to $A[j]$
- Note that : $A[i] = A[j]$ and $A[j] = A[i]$ do not swap content
- Exercise: Reverse an array using swaps

Accessing Arrays

- `int[] a = new int[]{4, 2, 0, 1, 3};`
- `system.out.println( a[0] );`
- `if (a[5] == 0) ...some statement`
- if the value computed for the index is less than 0, or greater than OR EQUAL TO the length of the array
 - trying to access the member at an illegal index causes Java to throw the
 - `ArrayIndexOutOfBoundsException` which contains a message showing what index was attempted to be accessed

The **String** Class

Strings

- ❑ A string is a sequence of characters
- ❑ Strings are objects of the `String` class
- ❑ String constants:

```
"Hello, World!"
```

- ❑ String variables:

```
String message = "Hello, World!";
```

- ❑ String length:

```
int n = message.length();
```

- ❑ Empty string:

```
""
```

Constructing a String:

```
String message = "Welcome to Java!"
```

String is an Object in Java.

Constructing Strings

```
String newString = new String(message);
```

```
String message = new String("Welcome to Java!");
```

Since strings are used frequently, Java provides a **shorthand notation** for creating a string:

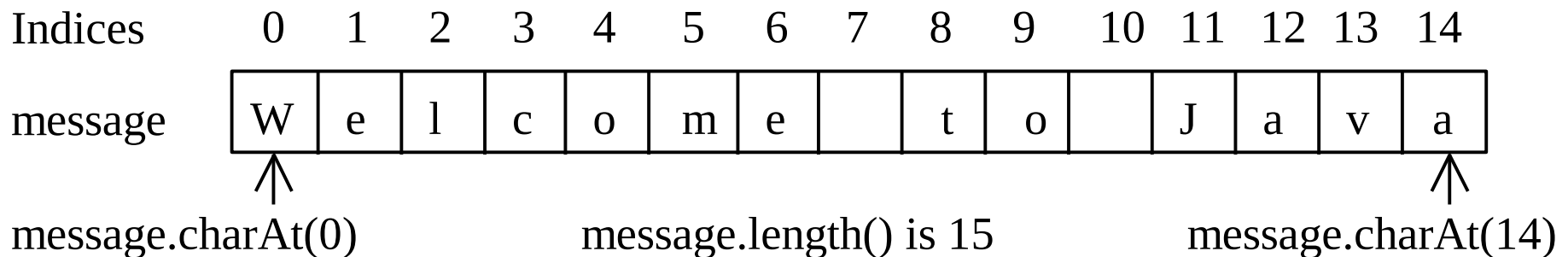
```
String message = "Welcome to Java!";
```

String

- +String()
- +String(value: String)
- +String(value: char[])
- +charAt(index: int): char
- +compareTo(anotherString: String): int
- +compareToIgnoreCase(anotherString: String): int
- +concat(anotherString: String): String
- +endsWith(suffix: String): boolean
- +equals(anotherString: String): boolean
- +equalsIgnoreCase(anotherString: String): boolean
- +indexOf(ch: int): int
- +indexOf(ch: int, fromIndex: int): int
- +indexOf(str: String): int
- +indexOf(str: String, fromIndex: int): int
- +intern(): String
- +regionMatches(toffset: int, other: String, offset: int, len: int): boolean
- +length(): int
- +replace(oldChar: char, newChar: char): String
- +startsWith(prefix: String): boolean
- +substring(beginIndex: int): String
- +substring(beginIndex: int, endIndex: int): String
- +toCharArray(): char[]
- +toLowerCase(): String
- +toString(): String

Retrieving Individual Characters in a String

- ❑ Do not use `message[0]`
- ❑ Use `message.charAt(index)`
- ❑ Index starts from 0



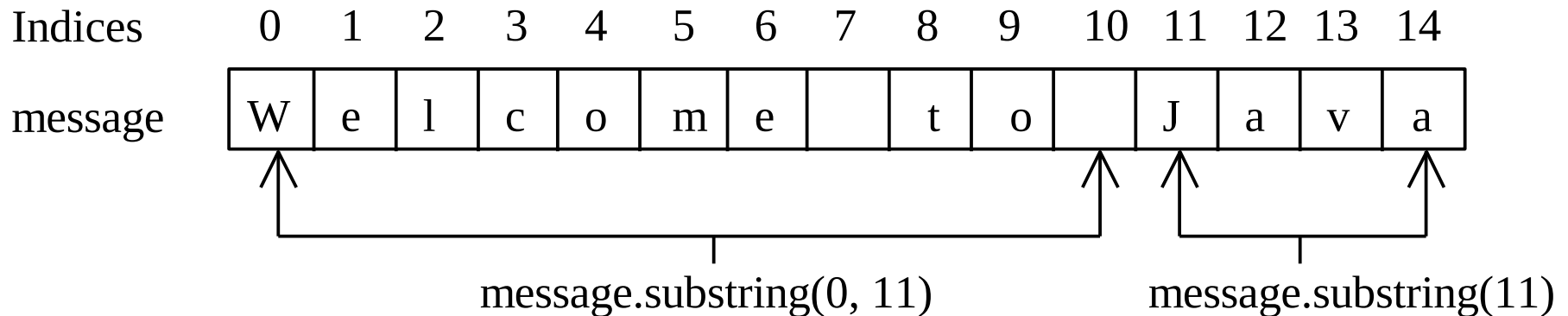
String Concatenation

```
String s3 = s1.concat(s2);
```

```
String s3 = s1 + s2;
```

Extracting Substrings

```
String s1 = "Welcome to Java";  
String s2 = s1.substring(0, 11) + "HTML";
```



Strings Are Immutable

A String object is **immutable**, whose contents cannot be changed.

To improve efficiency and save memory, Java Virtual Machine stores two String objects into the same object, if the two String objects are created with the same string literal using the shorthand notation.

Therefore, the shorthand notation is preferred to create strings.

String ==

```
String s = "Welcome to Java!";  
String s1 = new String("Welcome to Java!");  
String s2 = s1.intern();  
System.out.println("s1 == s is " + (s1 == s));  
System.out.println("s2 == s is " + (s2 == s));  
System.out.println("s1 == s2 is " + (s1 == s2));
```

displays

s1 == s is false

s2 == s is true

s1 == s2 false

String Comparisons: equals

```
String s1 = "Welcome";
```

```
String s2 = "Welcome";
```

```
if (s1.equals(s2)){
```

```
    // s1 and s2 have the same contents
```

```
}
```

```
if (s1 == s2) {
```

```
    // s1 and s2 have the same reference
```

```
}
```

compareTo(Object object)

```
if (s1.compareTo(s2) > 0) {  
    // s1 is greater than s2  
}
```

String Conversions

The contents of a string cannot be changed once the string is created. But you can convert a string to a new string using the following methods:

- ❑ `toLowerCase`
- ❑ `toUpperCase`
- ❑ `trim`
- ❑ `replace(oldChar, newChar)`

Finding a Character or a Substring in a String

"Welcome to Java!".indexOf('W')) returns 0.

"Welcome to Java!".indexOf('x')) returns -1.

"Welcome to Java!".indexOf('o', 5)) returns 9.

"Welcome to Java!".indexOf("come")) returns 3.

"Welcome to Java!".indexOf("Java", 5)) returns 11.

"Welcome to Java!".indexOf("java", 5)) returns -1.

Convert Character and Numbers to Strings

`valueOf` methods for converting a character, an array of characters, and numeric values to strings. These methods have the same name `valueOf` with different argument types `char`, `char[]`, `double`, `long`, `int`, and `float`.

Convert a double value to a String
`String.valueOf(5.44).`

The Character Class

Character

+Character(value: char)
+charValue(): char
+compareTo(anotherCharacter: Character): int
+equals(anotherCharacter: Character): boolean
+isDigit(ch: char): boolean
+isLetter(ch: char): boolean
+isLetterOrDigit(ch: char): boolean
+isLowerCase(ch: char): boolean
+isUpperCase(ch: char): boolean
+toLowerCase(ch: char): char
+toUpperCase(ch: char): char

Examples: Given a Character Object with value 'b'

`charObject.compareTo(new Character('a'))` returns 1

`charObject.compareTo(new Character('b'))` returns 0

`charObject.compareTo(new Character('c'))` returns -1

`charObject.compareTo(new Character('d'))` returns -2

`charObject.equals(new Character('b'))` returns true

`charObject.equals(new Character('d'))` returns false

Counting Each Letter in a String

Write a program that counts the number of occurrence of each letter in a string. Assume the letters are not case-sensitive.

String and String pools

What will be the output for :

```
String s1 = new String("Welcome to Java!");  
String s = "Welcome to Java!";  
String s2 = s1.intern();  
System.out.println("s1 == s is " + (s1 == s));  
System.out.println("s2 == s is " + (s2 == s));  
System.out.println("s1 == s2 is " + (s1 == s2));
```

```
String s1 = new String("Welcome to Java!");  
String s = "Welcome to Java!";  
String s2 = s1.intern();  
System.out.println("s1 == s is " + (s1 == s));  
System.out.println("s2 == s is " + (s2 == s));  
System.out.println("s1 == s2 is " + (s1 == s2));
```

The **StringBuffer** Class

The StringBuffer class is an alternative to the String class. In general, a string buffer can be used wherever a string is used.

StringBuffer is more flexible than String. You can add, insert, or append new contents into a string buffer. However, the value of a string is fixed once the string is created.

StringBuffer

- +append(data: char[]): StringBuffer
- +append(data: char[], offset: int, len: int): StringBuffer
- +append(v: *aPrimitiveType*): StringBuffer
- +append(str: String): StringBuffer
- +capacity(): int
- +charAt(index: int): char
- +delete(startIndex: int, endIndex: int): StringBuffer
- +deleteCharAt(int index): StringBuffer
- +insert(index: int, data: char[], offset: int, len: int): StringBuffer
- +insert(offset: int, data: char[]): StringBuffer
- +insert(offset: int, b: *aPrimitiveType*): StringBuffer
- +insert(offset: int, str: String): StringBuffer
- +length(): int
- +replace(int startIndex, int endIndex, String str): StringBuffer
- +reverse(): StringBuffer
- +setCharAt(index: int, ch: char): void
- +setLength(newLength: int): void
- +substring(start: int): StringBuffer
- +substring(start: int, end: int): StringBuffer

StringBuffer Constructors

- ❑ `public StringBuffer()`
No characters, initial capacity 16 characters.
- ❑ `public StringBuffer(int length)`
No characters, initial capacity specified by the length argument.
- ❑ `public StringBuffer(String str)`
Represents the same sequence of characters as the string argument. Initial capacity 16 plus the length of the string argument.

Appending New Contents into a String Buffer

```
StringBuffer strBuf = new StringBuffer();  
strBuf.append("Welcome");  
strBuf.append(' ');  
strBuf.append("to");  
strBuf.append(' ');  
strBuf.append("Java");
```

Self Check

- ❑ Assuming the `String` variable `s` holds the value `"Agent"`, what is the effect of the assignment `s = s + s.length()`?
- ❑ Assuming the `String` variable `river` holds the value `"Mississippi"`, what is the value of `river.substring(1, 2)`? Of `river.substring(2, river.length() - 3)`?

Answers

- ❑ `s` is set to the string `Agent5`
- ❑ The strings `"i"` and `"ssissi"`

String practice problem

Given Strings s1, s2, s3, Give code using the String class methods

- Order the strings Lexicographically Gr 1-4
- Checking whether a string is a palindrome Gr 5-8
- Convert s1 into substrings by breaking where there are spaces Gr 9-12
- Create a string from s1 inserting "ab" wherever there is a 'a' Gr 13-16
- Set a boolean array barry[] based on whether these strings start with "at". Gr 17-20

Object Oriented Programming

Introduction

Object Oriented Programming

- You have learnt – Procedural programming
- Here we learn a new approach

Programming progression...

- Programming has progressed through:
 - machine code
 - assembly language
 - machine-independent programming languages
 - procedures & functions
 - objects

Procedural Approach

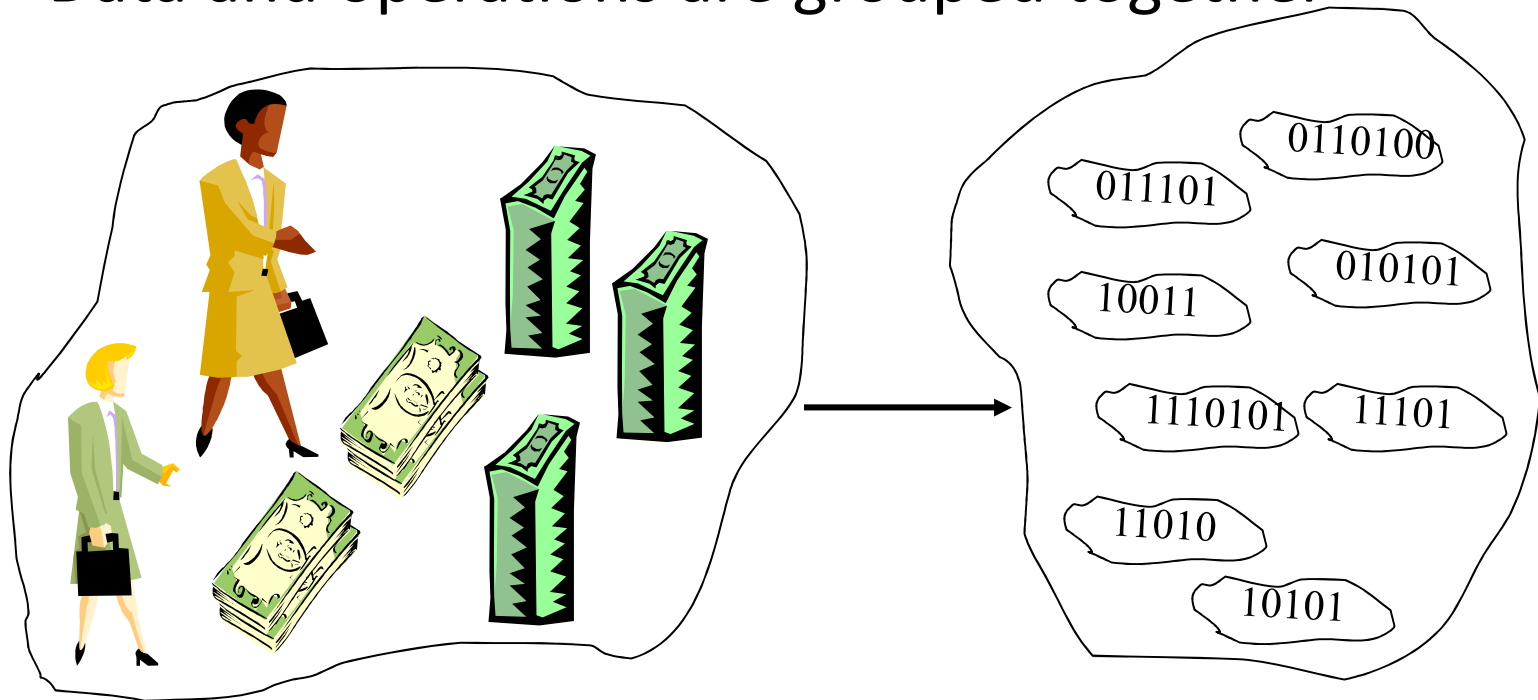
- Focus is on procedures
- All data is shared: no protection
- More difficult to modify
- Hard to manage complexity

History of Object-Oriented Programming

- Started out for simulation of complex man-machine systems, but was soon realized that it was suitable for all complex programming projects
- SIMULA I (1962-65) and Simula 67 (1967) were the first two object-oriented languages
 - Developed at the Norwegian Computing Center, Oslo, Norway by Ole-Johan Dahl and Kristen Nygaard

OOP: Mapping the world to software

- Objects in the problem domain are mapped to objects in software
- Data and operations are grouped together



Old Paradigm

Computer system consists of data and programs.

Programs manipulate data.

Programs organized by

- functional decomposition
- dataflow
- modules

OOP Paradigm

Computer system: a set of objects.

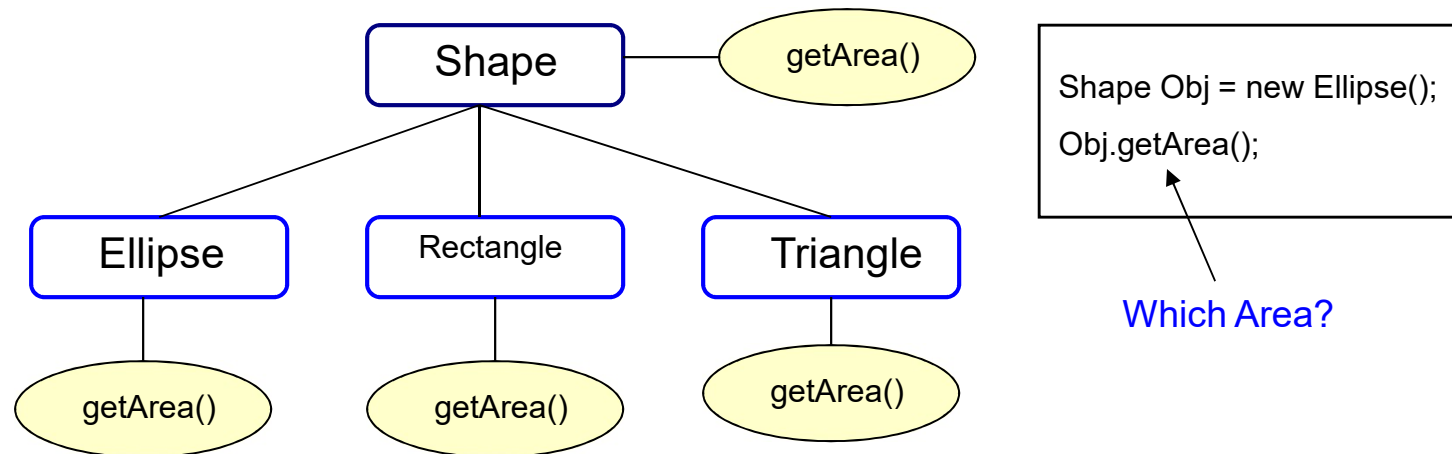
Objects: know and do certain things.

Objects collaborate, responsibilities.

Programs organized by classes, inheritance
hierarchies and subsystems

The 3 Principles of Object Oriented Programming

- **Encapsulation**
 - Control access to the data and methods
- **Inheritance**
 - Inherit the data and methods of parent class
- **Polymorphism**



Agenda

Object

Object Collaboration

Message

Class

OO Basics - Object

Real-world objects share two characteristics :-

They all have **state** and **behavior**.

Bicycles have **state** (current gear, current pedal cadence, two wheels, number of gears)
and **behavior** (braking, accelerating, slowing down, changing gears).

OO Basics - Object

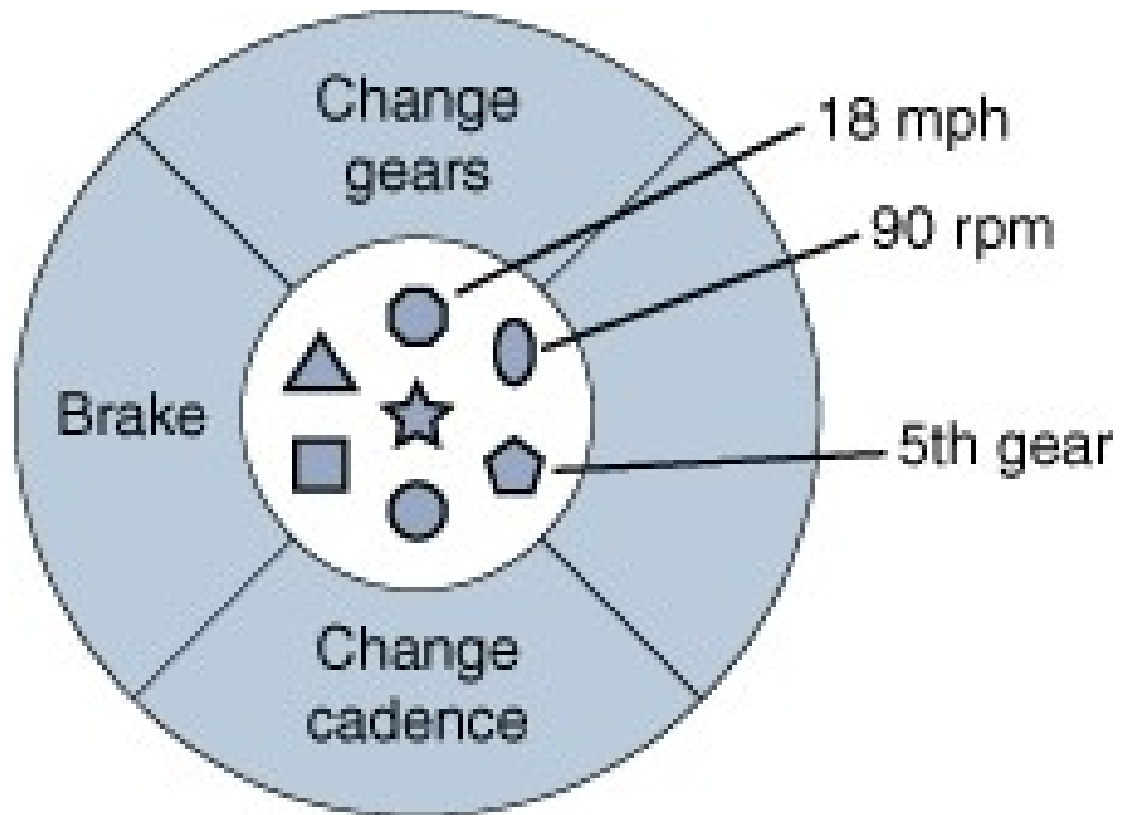
Software objects are modeled after real-world objects in that they too have state and behavior.

State  Variables.

Behavior  Methods.

OO Basics - Object

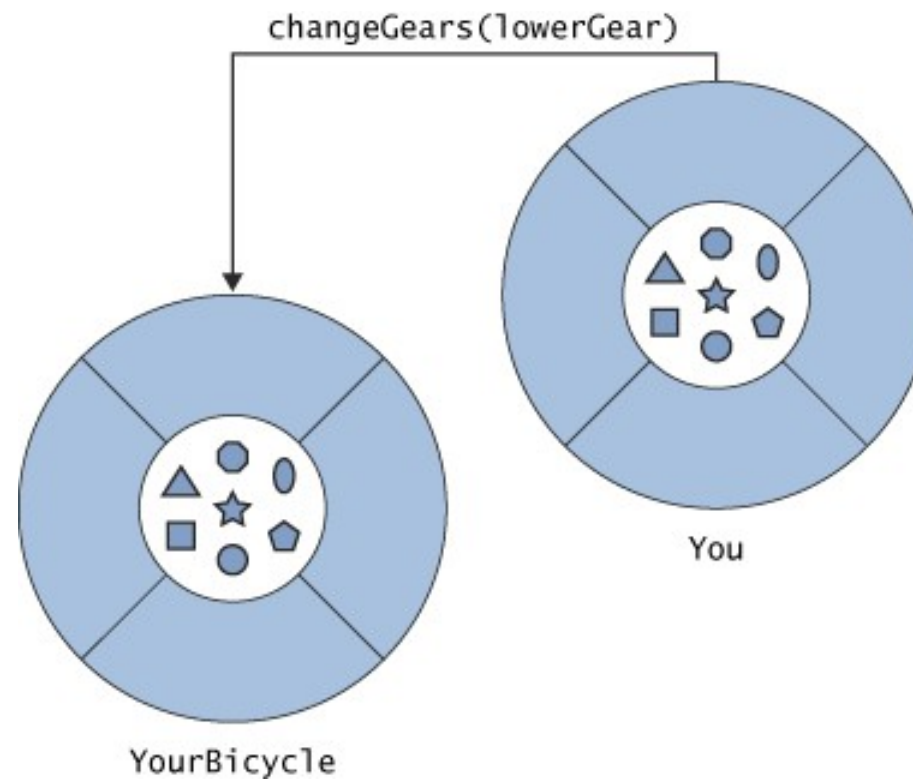
The following figure illustrates a bicycle modeled as a software object.



OO Basics - Message

Software objects interact and communicate with each other using messages.

OO Basics - Message



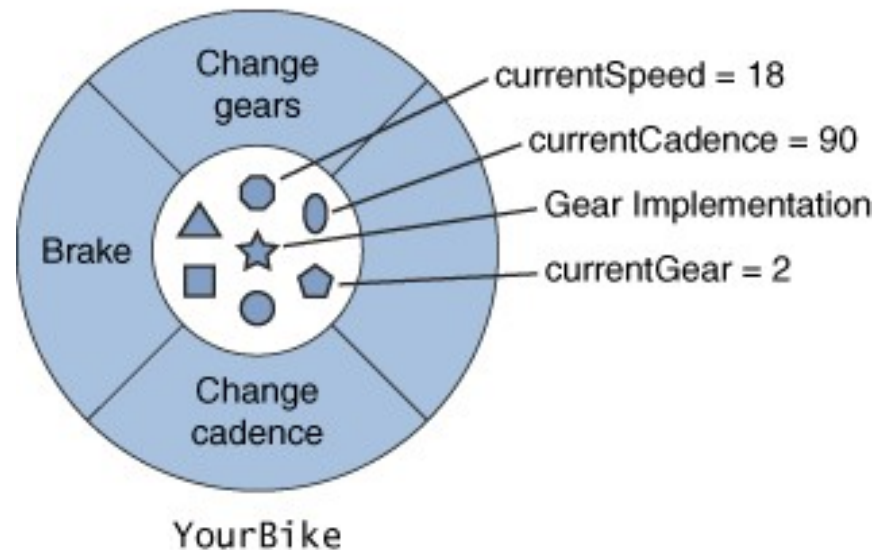
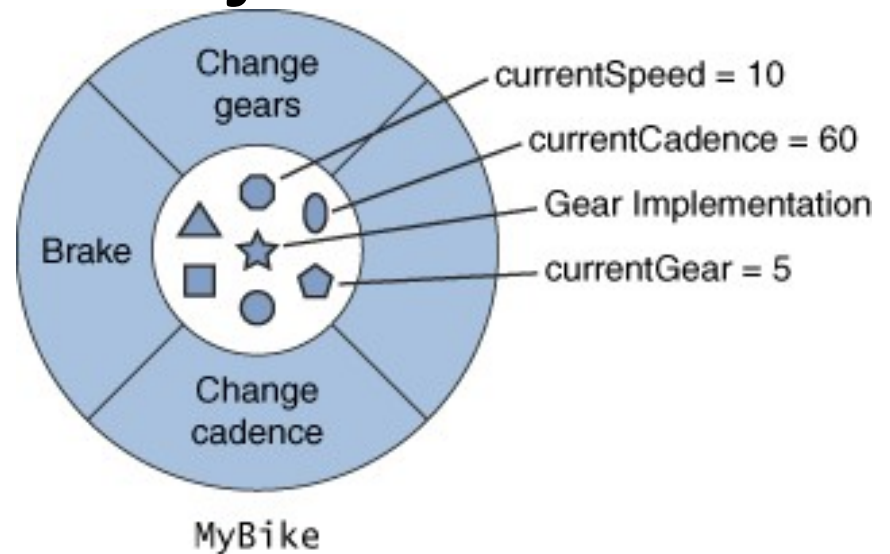
OO Basics - Class

A class is a blueprint or prototype that defines the variables and the methods common to all objects of a certain kind.

OO Basics - Object

MyBike and **YourBike** are two different instances of the Bike class.

Each instance has its own copy of the instance variables defined in the Bike class but has different values for these variables.



OO Basics – Static Variables

In addition to instance variables, classes can define class variables.

A class variable contains information that is shared by all instances of the class.

A class can also declare class methods .

You can invoke a class method directly from the class, whereas you must invoke instance methods on a particular instance.

Programming using Java

Java: final, static

Math class, Wrapper
classes, Object

Constants: `final`

- A `final` variable is a constant
- Once its value has been set, it cannot be changed
- Named constants make programs easier to read and maintain
- Convention: use all-uppercase names for constants

Constants: final

```
final double QUARTER_VALUE = 0.25;
final double DIME_VALUE = 0.1;
final double NICKEL_VALUE = 0.05;
final double PENNY_VALUE = 0.01;
payment = dollars + quarters * QUARTER_VALUE
        + dimes * DIME_VALUE
        + nickels * NICKEL_VALUE
        + pennies * PENNY_VALUE;
```

Constants: `static final`

- If constant values are needed in several methods, declare them together with the instance fields of a class and tag them as `static` **and** `final`
- **Give** `static final` constants public access to enable other classes to use them

Constants: `static final`

```
public class Math
{
    . . .
    public static final double E =
        2.7182818284590452354;
    public static final double PI =
        3.14159265358979323846;
}

double circumference = Math.PI * diameter;
```

Class Math

The Math class

- Math class: contains static methods like `sqrt` and `pow`
- To compute x^n , you write `Math.pow(x, n)`
- However, to compute x^2 it is significantly more efficient simply to compute `x * x`
- To take the square root of a number, use the `Math.sqrt`; for example,
`Math.sqrt(x)`

Mathematical Methods in Java

<code>Math.sqrt(x)</code>	square root
<code>Math.pow(x, y)</code>	power x^y
<code>Math.exp(x)</code>	e^x
<code>Math.log(x)</code>	natural log
<code>Math.sin(x)</code> , <code>Math.cos(x)</code> , <code>Math.tan(x)</code>	sine, cosine, tangent (x in radian)
<code>Math.round(x)</code>	closest integer to x
<code>Math.min(x, y)</code> , <code>Math.max(x, y)</code>	minimum, maximum

Primitives & Wrappers

- Java has a *wrapper* class for each of the eight primitive data types:

Primitive Type	Wrapper Class	Primitive Type	Wrapper Class
boolean	Boolean	float	Float
byte	Byte	int	Integer
char	Character	long	Long
double	Double	short	Short

Use of the Wrapper Classes:

Methods for conversion

- Value => Object `valueOf`
- Object => Value `intValue, booleanValue`
- String => value `parseInt, parseBoolean, ...`

```
Integer i1 = Integer.valueOf(42) ;  
Integer i2 = Integer.valueOf("42") ;
```

```
Boolean b1 = Boolean.valueOf(true) ;  
Boolean b2 = Boolean.valueOf("true") ;
```

```
System.out.println(i1.intValue()) ;  
System.out.println(b1.booleanValue()) ;
```

```
Integer.parseInt("42")           => 42  
Boolean.parseBoolean("true")     => true  
Double.parseDouble("2.71")       => 2.71
```

Many useful utility methods: Integer

```
        int    hashCode()  
static int    numberOfLeadingZeros(int i)  
static int    numberOfTrailingZeros(int i)  
static int    reverse(int i)  
static int    reverseBytes(int i)  
static int    rotateLeft(int i, int distance)  
static int    rotateRight(int i, int distance)  
static String toBinaryString(int i)  
static String toHexString(int i)  
static String toOctalString(int i)  
static String toString(int i, int radix)
```


Double & Float: Utilities for Arithmetic Operations:

- Constants

POSITIVE_INFINITY, NEGATIVE_INFINITY

- Constant NaN = Not-a-Number (NaN) value.
- Methods isNaN(), isFinite()

Class Object

Class Object

- **Object** is the root of the class hierarchy
 - Every *class* has **Object** as a superclass
- All classes inherit the methods of **Object**
 - But may override them

Class Object

TABLE 3.2

Methods of Class `java.lang.Object`

Method	Behavior
<code>Object clone()</code>	Makes a copy of an object.
<code>boolean equals(Object obj)</code>	Compares this object to its argument.
<code>int hashCode()</code>	Returns an integer hash code value for this object.
<code>String toString()</code>	Returns a string that textually represents the object.

The Method `toString`

- You should always override `toString` method if you want to print object state

- If you do *not* override it:

- `Object.toString` will return a `String`
- Just not the `String` you want!

Example: `ArrayBasedPD@ef08879`

... The name of the class, @, instance's hash code

Programming using Java

Java: Inheritance

Objectives:

- Get an introduction to Inheritance
- Get a general idea of how a hierarchy of classes is put together

Inheritance: Definition

- **inheritance**: a parent-child relationship between classes
- allows sharing of the behavior of the parent class into its child classes
 - one of the major benefits of object-oriented programming (OOP) is this code sharing between classes through inheritance
- child class can add new behavior or override existing behavior from parent

Inheritance terms

- **superclass, base class, parent class:** terms to describe the parent in the relationship, which shares its functionality
- **subclass, derived class, child class:** terms to describe the child in the relationship, which accepts functionality from its parent
- **extend, inherit, derive:** become a subclass of another class

Inheritance in Java

- in Java, you specify another class as your parent by using the keyword `extends`
 - `public class Rectangle`
`extends Shape {`
 - `public class CheckingAccount`
`extends BankAccount {`
- Java forces a class to have exactly one parent ("single inheritance")
 - other languages (C++) allow multiple inheritance

Inheritance in Java

- the objects of your class will now receive all of the state (fields) and behavior (methods) of the parent class
- constructors and static methods/fields are not inherited
- by default, a class's parent is `Object`

Inheritance Example

```
class BankAccount {  
    private double myBal;  
    public BankAccount() { myBal = 0; }  
    public double getBalance() { return myBal; }  
}
```

```
class CheckingAccount extends BankAccount {  
    private double myInterest;  
    public CheckingAccount(double interest) { }  
    public double getInterest() { return  
        myInterest; }  
    public void applyInterest() { }  
}
```

- CheckingAccount objects have myBal and myInterest fields, and getBalance(), getInterest(), and applyInterest() methods

Multiple layers of inheritance

- it is possible to extend a class that itself is a child class; inheritance chains like this can be arbitrarily deep

```
public class TransactionFeeCheckingAccount
    extends CheckingAccount {
    private static final double FEE = 2.00;

    public void chargeFee() {
        withdraw(FEE);
    }
}
```

"Has-a" Relationships

- "Has-a" relationship: when one object contains another as a field

```
public class BankAccountManager {  
    private List myAccounts;  
    // ...  
}
```

- a `BankAccountManager` object "has-a" `List` inside it, and therefore can use it

"Is-a" relationships

- "Is-a" relationships represent sets of abilities; implemented through interfaces and inheritance

```
public class CheckingAccount
    extends BankAccount {
    // ...
}
```

- a CheckingAccount **object** "is-a" BankAccount
 - therefore, it can do anything an BankAccount can do
 - it can be substituted wherever a BankAccount is needed
 - a variable of type BankAccount may refer to a CheckingAccount **object**

Using the account classes

- CheckingAccount inherits BankAccount's methods

```
CheckingAccount c = new
```

```
CheckingAccount(0.10);
```

```
System.out.println(c.getBalance());
```

```
c.applyInterest();
```

```
□ );
```


Using the account classes

- a `BankAccount` variable can refer to a `CheckingAccount` object

```
BankAccount b2 = new CheckingAccount(0.06);  
System.out.println(b2.getBalance());
```

- an `Object` variable can point to either account type

```
Object o = new BankAccount();  
Object o2 = new CheckingAccount(0.09);
```

Some code that won't compile

- CheckingAccount variable can't refer to BankAccount (not every BankAccount "is-a" CheckingAccount)

```
CheckingAccount c = new BankAccount();
```

- cannot call a CheckingAccount method on a variable of type BankAccount (can only use BankAccount behavior)

```
BankAccount b = new CheckingAccount(0.10);  
b.applyInterest();
```

- cannot use any account behavior on an Object variable

```
Object o = new CheckingAccount(0.06);  
System.out.println(o.getBalance());  
o.applyInterest();
```

Interfaces

Interface

- is a way to describe what classes should do,
 - without specifying how they should do it.
- NOT a class but a set of requirements
- class - implements an interface

Concept

```
public interface Comparable  
{  
    int compareTo(Object otherObject);  
}
```

- Any class implementing the `Comparable` interface contains a `compareTo` method,
- and this method must take an `Object` parameter and return an integer

Interface declarations

- The declaration consists of a keyword `interface`, its name, and the members
- Similar to classes, interfaces can have three types of members
 - constants (fields)
 - methods
 - nested classes and interfaces

Interface member – constants

- An interface can define named constants, which are `public`, `static` and `final` (**these modifiers are omitted by convention**) automatically. Interfaces never contain instance fields.
- All the named constants MUST be initialized

An example interface

```
interface Verbose {  
    int SILENT = 0;  
    int TERSE = 1;  
    int NORMAL = 2;  
    int VERBOSE = 3;  
  
    void setVerbosity (int level);  
    int getVerbosity();  
}
```

To implement interfaces in a class

- Two steps to make a class implement an interface
 1. declare that the class intends to implement the given interface by using the `implements` keyword

```
class Employee implements Comparable  
{ . . . }
```

2. supply definitions for **all** methods in the interface

```
public int compareTo(Object otherObject) {
```

```
    Employee other = (Employee) otherObject;  
    if (salary < other.salary) return -1;  
    if (salary > other.salary) return 1;  
    return 0; }
```

note: in the `Comparable` interface declaration, the method `compareTo()` is `public` implicitly but this modifier is omitted. But in the `Employee` class design, you cannot omit the `public` modifier, otherwise, it will be assumed to have package accessibility

To implement interfaces in a class

- If a class leaves any method of the interface undefined, the class becomes `abstract` class and must be declared `abstract`
- A single class can implement multiple interfaces. Just separate the interface names by comma

```
class Employee implements Comparable,  
    Cloneable { . . . }
```

Instantiation properties of interfaces

- Interfaces are not classes. You can never use the `new` operator to instantiate an interface.

```
public interface Comparable {  
    . . .  
}  
  
Comparable x = new Comparable( );
```

- You can still declare interface variables

```
Comparable x;
```

but they must refer to an object of a class that implements the interface

```
class Employee implements Comparable {  
    . . .  
}  
  
x = new Employee( );
```

Extending interfaces

- Interfaces support **multiple** inheritance – an interface can extend more than one interface
- Superinterfaces and subinterfaces

Example

```
public interface SerializableRunnable extends  
java.io.Serializable, Runnable {  
    . . .  
}
```

Abstract Classes

Abstract classes

- **abstract class:** a hybrid between an interface and a class
 - used to define a generic parent type that can contain method declarations (like an interface) and/or method bodies (like a class)
 - like interfaces, abstract classes that cannot be instantiated (cannot use `new` to create any objects of their type)

What goes in an abstract class?

- implement common state and behavior that will be inherited by subclasses (parent class role)
- declare generic behaviors that subclasses must implement (interface role)

Abstract class syntax

- put `abstract` keyword on class header and on any generic (abstract) methods
 - A class can be declared `abstract` even though it has no abstract methods
 - Any class with abstract methods *must* be declared `abstract`, or it will not compile
- Variables of abstract types may be declared, but *objects* of abstract types cannot be constructed

Abstract class example

```
public abstract class Shape implements Shape2D
{
    private int myX, myY;

    public Shape(int x, int y) {
        myX = x;    myY = y;
    }
    public int getX() { return myX; }
    public int getY() { return myY; }

    public abstract double getArea();
    public abstract double getPerimeter();
}
```

- Shape class cannot be instantiated
- all classes that extend Shape **must** implement getArea and getPerimeter or else must also be declared abstract

Extending an abstract class

```
public class Rectangle extends Shape {  
    private int myWidth, myHeight;  
  
    public Rectangle(int x, int y, int w, int h)  
    {  
        super(x, y);  
        myWidth = w;    myHeight = h;  
    }  
  
    public double getArea() {  
        return myWidth * myHeight;  
    }  
    public double getPerimeter() {  
        return 2*myWidth + 2*myHeight;  
    }  
}  
  
// ... example usage ...  
Shape rect = new Rectangle(1, 2, 10, 5);
```


Interface / abstract class chart

TABLE 3.1

Comparison of Actual Classes, Abstract Classes, and Interfaces

Property	Actual Class	Abstract Class	Interface
Instances (objects) of this can be created	Yes	No	No
This can define instance variables and methods	Yes	Yes	No
This can define constants	Yes	Yes	Yes
The number of these a class can extend	0 or 1	0 or 1	0
The number of these a class can implement	0	0	Any number
This can extend another class	Yes	Yes	No
This can declare abstract methods	No	Yes	Yes
Variables of this type can be declared	Yes	Yes	Yes

Understanding Packages

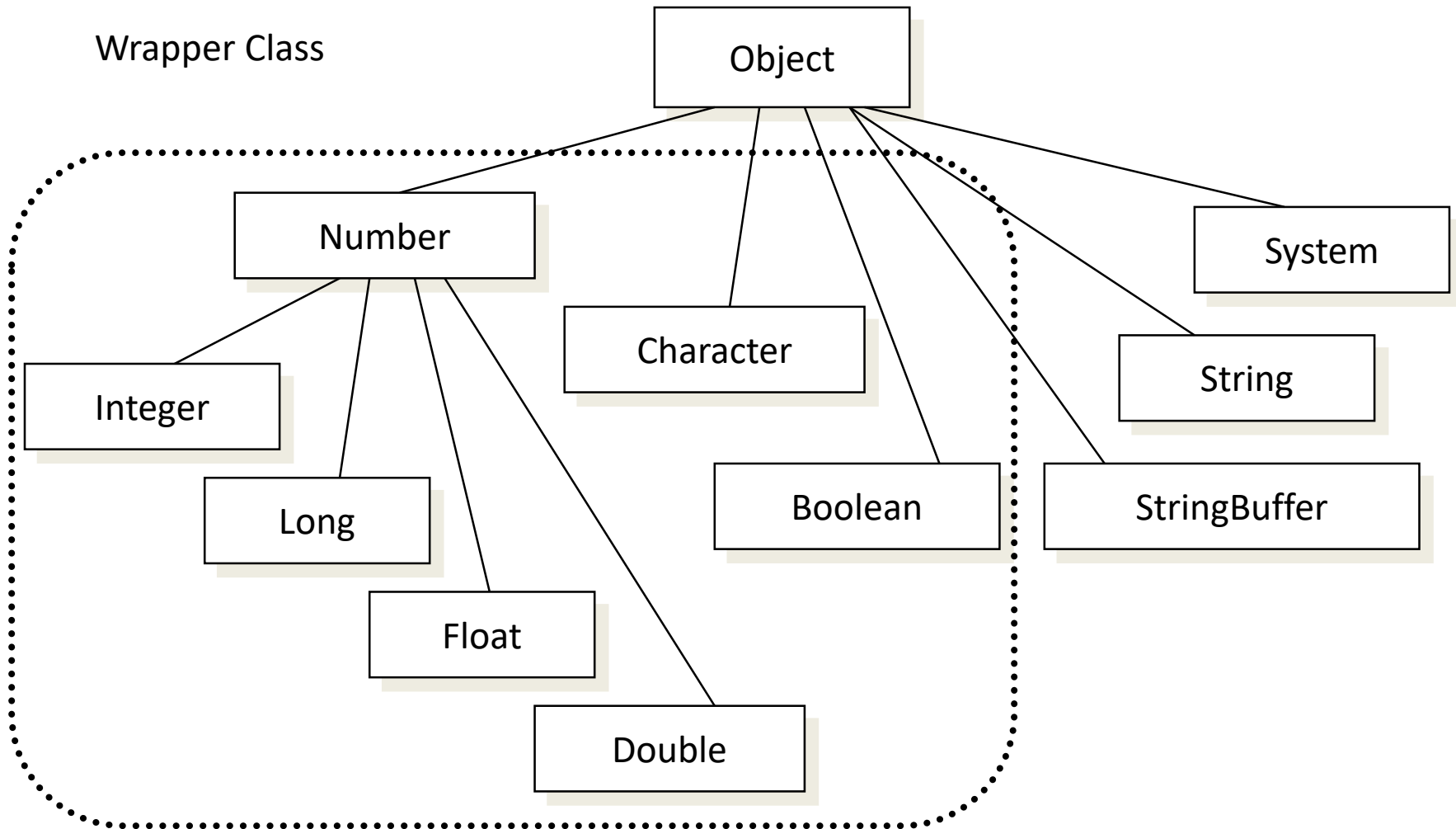
Packages

- A Package is a unique named collection of classes
- The purpose of grouping classes is to make it easy to add any or all of the classes
- Names of the classes within the package should be unique, but, same name can be repeated across the packages – as a class name is qualified with the package name

Package in Java

- A package is a namespace that organizes a set of related classes and interfaces.
- A java package is a group of related classes similar to a class library.
- Keyword 'import'
- All of the standard classes are contained within the packages.
- If you put multiple types in a single source file, only one can be public, and it must have the same name as the source file.
- *java.lang* is special

Package in Java



Selected packages

- java.lang -- String, wrapper classes, Math
- java.util – Calendar, Date, Vector
- java.applet – Applet
- java.text – DateFormat
- java.awt – Graphics, Button, Label..
- javax.swing – JButton
- java.io – InputStream, OutputStream

Naming Conventions

- Package names are written in all lowercase to avoid conflict with the names of classes or interfaces. Companies use their reversed Internet domain name to begin their package name
 - For example, `com.example.orion` for a package named `orion` created by a programmer at `example.com`.
- Name collisions that occur within a single company need to be handled by convention within that company, perhaps by including the region or the project name after the company name
 - (for example, `com.company.region.package`).
- Packages in the Java language itself begin with `java.` or `javax.`

Naming conventions

- In some cases, the internet domain name may not be a valid package name.
 - This can occur if the domain name contains a hyphen or other special character, if the package name begins with a digit or other character that is illegal to use as the beginning of a Java name, or if the package name contains a reserved Java keyword, such as "int".
 - In this event, the suggested convention is to add an underscore.
 - Ex: clipart-open.org → org.clipart_open

Using Package Members

- The types that comprise a package are known as the *package members*. To use a public package member from outside its package, you must do one of the following:
 - Refer to the member by its fully qualified name
 - *graphics.Rectangle myRect = new graphics.Rectangle();*
 - Import the package member
 - *import graphics.Rectangle;*
Rectangle myRectangle = new Rectangle();
 - Import the member's entire package
 - *import java.util.*;*
 - *import graphics.*;*
 - *import graphics.A*; //does not work*

Packaging your classes

- Add a package statement as the first statement in the source file containing the class definition
- It must always be the first statement
- Ex: `package geometry;`


keyword


necessary

```
//Sphere.java  
package geometry;  
public class Sphere  
{  
//details of the class definition  
}
```

Packaging your classes

- Directory : geometry
- File: *Sphere.java*
- *Compiling: Ensure that you are out of the directory*
javac /geometry/Sphere.java

To execute

java geometry.Sphere

For a large number of files in a package you can use
javac /geometry/.java*

All interdependencies will be figured out by the compiler

Packages and directory structure

- A package is intimately related to the directory structure in which it is stored
- A class with a *Classname* must be stored in a file called *Classname.java*
- Similarly, all the files for classes within a package, *packageName* must be included in a directory with the name, *packageName*.
- A package need not have a single name. it can be a sequence of names separated by a period

package geometry.shapes3D;

package geometry.shapes2D;

- shapes3D and shapes2D are sub directories of geometry directory.

Setting classpath

- From the command prompt,
set CLASSPATH = .;c:\MySource; c:\MyPackages
- Unless a class path is set, or set incorrectly, java will not be able to find the classes in any new packages you might create.

Adding classes from a Package to your Program

- Classes with 'public' are accessible to your program by using 'import' statement.
 - *import geometry.shapes3D.*;* //includes all the classes from this package
 - '*' selects all the classes in the package
 - You can refer any public class in the package
 - If you have to add a specific class, specify explicitly like,

import geometry.shapes3D.Sphere;

- '*' can be only used to select all the classes in a package. You cannot use *geometry.** to select all the packages in the directory *geometry*.

Encapsulation of classes into a package

- Add a class into a package — two steps:
 1. put the name of the package at the top of your source file

```
package com.hostname.corejava;  
public class Employee {  
    . . .  
}
```

2. put the files in a package into a subdirectory which matches the full package name

→ stored in the file "Employee.java" which is stored under
"somePath/com/hostname/corejava/"

Setting the class path

E.g.

- current classpath:

`/home/user/classdir../home/user/archives/archive.jar`

- to search for the class file of the

`com.horstmann.corejava.Employee` class

first searches in the system class files that are stored in archives in the `jre/lib` and `jre/lib/ext` directories.

If it can't find the class there it will search in the order:

- 1) `/home/user/classdir/com/horstmann/corejava/Employee.class`

- 2) `./com/horstmann/corejava/Employee.class`

- 3) `com/horstmann/corejava/Employee.class` inside `/home/user/archives/archive.jar`

- if found the interpreter stops searching process

Naming conventions

- **Package names:** start with lowercase letter
 - E.g. java.util, java.net, java.io ...
- **Class names:** start with uppercase letter
 - E.g. File, Math ...
 - avoid name conflicts with packages
 - avoid name conflicts with standard keywords in java system
- **Variable, field and method names:** start with lowercase letter
 - E.g. x, out, abs ...
- **Constant names:** all uppercase letters
 - E.g. PI ...
- **Multi-word names:** capitalize the first letter of each word after the first one
 - E.g. HelloWorldApp, getName ...
- **Exception class names:** (1) start with uppercase letter (2) end with “Exception” with normal exception and “Error” with fatal exception
 - E.g. OutOfMemoryError, FileNotFoundException

Nested Classes

Nested Classes and Interfaces

- ❧ Classes and interfaces can be declared inside other classes and interfaces, either as members or within blocks of code.

Some Definitions

- ⌘ **Nested** = a class or interface definition is somewhere inside another one
- ⌘ **Top-level class or interface** = an instance of a type **does not** need to be instantiated with a reference to any enclosing instance
- ⌘ **Inner class** = an instance of a class **does** need to be instantiated with an enclosing instance
- ⌘ **Inner Member class** = defined inside another class, but not inside any methods.
- ⌘ **Inner Local class** = defined inside a method
- ⌘ **Named Inner Local class** = has a class name
- ⌘ **Anonymous Inner Local class** = does not have a class name

Nested Classes

```
class TheEnclosingClass{  
    ...  
    class ANestedClass {  
        ...  
    }  
}
```

- ⌘ **Definition:** A **nested class** is a class that is a member of another class.
- ⌘ **Reason for making nested classes:**
 - ⌘ the nested class **makes sense only in the context of its enclosing class**
 - ⌘ the nested class **needs the enclosing class to have the right functionality.**

Nested Static Classes

- ⌘ A nested class that is declared static **is attached to the enclosing class** and not to objects of the enclosing class. Instance fields and methods can not be directly accessed.

Example: *Linked List*

```
public class LinkedList {  
    private Node first;  
    .....  
    public static class Node{  
        public Node next;  
        public Object data;  
    }  
    .....  
}
```

Static Nested Classes/Interfaces — Overview

- ✧ A nested class/interface which is declared as `static` acts just like any non-nested class/interface, except that its name and accessibility are defined by its enclosing type.
- ✧ Static nested types are members of their enclosing type
 - ✧ They can access all other members of the enclosing type including the private ones.
 - ✧ Inside a class, the static nested classes/interfaces can have private, package, protected or public access; while inside an interface, all the static nested classes/interfaces are implicitly public.
 - ✧ They serve as a structuring and scoping mechanism for logically related types

Static Inner Classes

- ✎ Since a static inner class has no connection to an object of the outer class, within an inner class method
 - ✎ Instance variables of the outer class cannot be referenced
 - ✎ Nonstatic methods of the outer class cannot be invoked
- ✎ To invoke a static method or to name a static variable of a static inner class within the outer class, preface each with the name of the inner class and a dot

Static Nested Classes/Interfaces (cont.)

⌘ Static nested classes

- ⌘ If a class is nested in an interface, it's always static (omitted by convention)
- ⌘ It can extend any other class, implement any interface and itself be extended by any other class to which it's accessible
- ⌘ Static nested classes serve as a mechanism for defining logically related types within a context where that type makes sense.

Static Nested Classes/Interfaces (cont.)

↳ Nested interfaces

- ↳ Nested interfaces are always static (omitted by convention) since they don't provide implementation

Static Nested Classes/Interfaces (cont.)

```
public class BankAccount {  
    private long number;    //account number  
    private long balance;   //current balance  
  
    public static class Permissions {  
        public boolean canDeposit, canWithdraw,  
canClose;  
    }  
    // . . .  
}
```

& Code outside the BankAccount class must use
BankAccount.Permissions to refer to this class

```
BankAccount.Permissions perm = new  
BankAccount.Permissions();
```

Home Work: Modify the LinkedList and Node classes on Slide 6

1. Give code for a CreateLL class which will have main() to take input from the user- No of LL to be created, number of nodes for each
2. LinkedList should have an int variable numNodes which will keep a count of ALL nodes created

Inner Classes

- ⌘ A **nested class** that is not static is called an **inner class**.
- ⌘ An **inner class** is associated with an object of its enclosing class and it has **direct and unlimited access** to that object's instance variables and methods.
- ⌘ A **nested class** can be **declared at the top level** inside a class **or inside any block of code**.

Inner classes

```
class Outer {  
    int n;  
  
    class Inner {  
        int ten = 10;  
        void setNToTen( ) { n = ten; }  
    }  
  
    void setN ( ) {  
        new Inner( ).setNToTen( );  
    }  
}
```

Inner Class

🔗 Name Reference

🔗 inside OuterClass : Use InnerClass Simple name

🔗 outside OuterClass : OuterClass.InnerClass

```
public static void main(String[] args) {  
    OuterClass outObj = new OuterClass();  
    OuterClass.InnerClass inObj = outObj.new InnerClass();  
}
```

🔗 Access Modifier

🔗 public, private, protected

Inner class cannot have static variables

Nesting Inner Classes

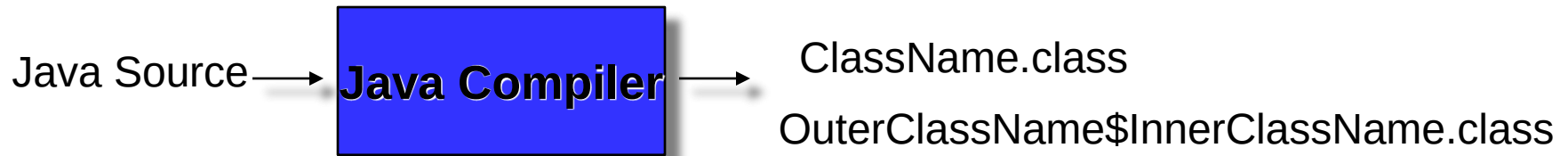
- It is legal to nest inner classes within inner classes
 - The rules are the same as before, but the names get longer
 - Given class **A**, which has public inner class **B**, which has public inner class **C**, then the following is valid:

```
A aObject = new A();
```

```
A.B bObject = aObject.new B();
```

```
A.B.C cObject = bObject.new C();
```

Name of Inner Class



```
class Outer {  
    class Inner1 {  
        class Inner2 { // ...  
        }  
        // ...  
    }  
    // ...  
}
```

⇒

- Outer.class
- Outer\$Inner1.class
- Outer\$Inner1\$Inner2.class

Simple inner class example

```
class Outer{  
    private int x1;
```

```
    Outer(int x1){  
        this.x1 = x1;  
    }
```

```
    public void foo(){ System.out.println("fooing");}
```

```
    public class Inner{  
        private int x1 = 0;  
        void foo(){  
            System.out.println("Outer value of x1: " + Outer.this.x1);  
            System.out.println("Inner value of x1: " + this.x1);  
        }  
    }
```

Simple example, cont -- driver

- Rules for instantiation

```
public class TestDrive{  
  
    public static void main(String[] args){  
        Outer outer = new Outer();  
        Inner inner = outer.new Inner(); //must call new through  
                                         //outer object handle  
        inner.foo();  
  
        // note that this can only be done if inner is visible  
        // according to the regular scoping rules  
    }  
}
```

Non-static Classes — Inner classes

✂ *Inner classes* are associated with instances of its enclosing class.

```
public class BankAccount {  
    private long number;        // account number  
    private long balance;       // current balance  
    private Action lastAct;     //last action performed  
  
    public class Action {  
        private String act;  
        private long amount;  
  
        Action(String act, long amount) {  
            this.act = act;  
            this.amount = amount;  
        }  
    }  
}
```

Non-static Classes — Inner classes

```
public void deposit(long amount) {  
    balance += amount;  
    lastAct = new Action("deposit", amount);  
}  
  
public void withdraw(long amount) {  
    balance -= amount;  
    lastAct = new Action("withdraw", amount);  
}  
// . . .  
}
```

Inner classes (cont.)

- ⌘ When an inner class object is created, it MUST be associated with an object of its enclosing class.
- ⌘ Usually, inner class objects are created inside instance methods of the enclosing class. When this occurs, the current enclosing object `this` is associated with the inner object by default.

```
lastAct = this.new Action("deposit", amount);
```

- ⌘ When `deposit` creates an `Action` object, a reference to the enclosing `BankAccount` object is automatically stored in the `Action` object.

Inner classes (cont.)

method for `transfer` is added

```
public void transfer(BankAccount other, long
amount) {
```

```
other.withdraw(amount);
deposit(amount);
```

```
lastAct = this.new Action("transfer", amount);
```

[illegible]

Inner classes (cont.)

- ✧ The enclosing class can also access the private members of its inner class, but only via explicit reference to an inner class object.
- ✧ An object of the enclosing class need not have any inner class objects associated with it, or it could have many.
- ✧ An inner class acts as a top-level class except that it can't have static members (except for final static fields).
- ✧ Inner classes can also be extended.

Inheritance, Scoping and Hiding

- ⌘ All members declared within the enclosing class are said to be in *scope* inside the inner class.
- ⌘ An inner class's own fields and methods can hide those of the enclosing object. Two possible ways:

1). A member with same name is declared in the inner class

- ⌘ Any direct use of the name refers to the version inside the inner class

```
class Host {  
    int x;  
    class Helper {  
        void increment()  
        {int x=0; x++;}  
    }  
}
```



- ⌘ Access to the enclosing object's members needs be preceded by `this` explicitly

Inheritance, Scoping and Hiding

2). A member with the same name is inherited by the inner class

⌘ The direct use of the name is not allowed

```
class Host {  
    int x;  
    class Helper extends Unknown {  
//Unknown class has a field x  
        void increment () {x++;}  
    }  
}
```

⌘ Use `enclosingClassName.this.name` to refer to the version in the outer class

⌘ or `super.name` to refer to the version in the super class

⌘ Use `this.name` in the inner class

Specifying which scope you want:

```
public class ScopeConflict {  
    String s = "outer";
```

```
    class Inner extends SuperClass {
```

```
        String s = "inner";
```

```
        void foo(){  
            System.out.println(this.s);  
            System.out.println(super.s);  
            System.out.println(ScopeConflict.this.s);  
        }  
    }  
}
```

```
class SuperClass {  
    String s = "super";  
}
```

output:

inner (from this.s)

super (from super.s)

outer (from ScopeConflict.this.s)

Inheritance, Scoping and Hiding (cont.)

- ⌘ A method within an inner class which has the same name as an enclosing **method hides all overloaded forms** of the enclosing method, even if the inner class itself does not declare those overloaded forms.

```
class Outer {  
    void print() {}  
    void print(int value) {}  
    class Inner {  
        void print() {}  
        void show() {  
            print();  
            Outer.this.print();  
            print(1); // no Inner.print(int)  
        }  
    }  
}
```

Local Inner Classes

- ⌘ You can define inner classes in code blocks. They are called *local inner classes*.
- ⌘ You can actually declare an inner class inside of a method, just like you could declare a local variable.
- ⌘ Local classes do not get an access modifier – they are automatically restricted to the method they are defined in
- ⌘ Can only refer to final members of the enclosing class
 - ⌘ They are NOT members of the class which contains the code but are local to that block, as a local variable.
 - ⌘ They are completely inaccessible outside of the block.
 - ⌘ Only one modifier is allowed—final—which makes them unextendable

Anonymous Inner classes

- ⌘ When using a local inner class, if you only want to make one instance of it, you don't even need to give it a name
- ⌘ This is known as an anonymous inner class
- ⌘ These are convenient for event programming
- ⌘ However, the syntax is extremely cryptic.

Anonymous Inner classes

- ⌘ You have to look very carefully to see a difference between construction of a new object, and construction of a new inner class extending a class.

//A person object from **Person** class

Person queen=new Person("Mary"); //Person Object

//An object of an inner class extending **Person** interface

Person count = new **Person() {**
 //class code here
 };

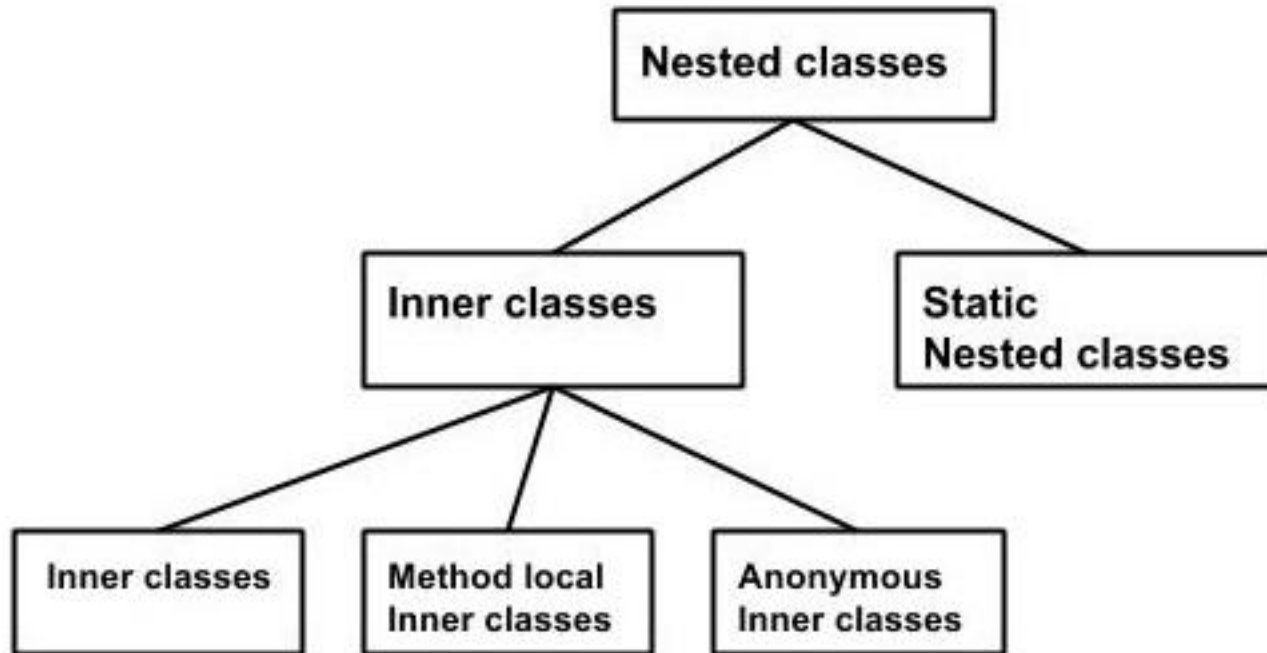
Anonymous Inner classes

- ⌘ Anonymous Inner classes cannot have constructors, since constructors have to have the same name as the class, and these classes have no names.
- ⌘ As you can see, the syntax for these is confusing – both for people writing and reading the code. Use this with care, if at all.

Anonymous Inner classes

```
public void start(final double rate)
{
    ActionListener adder = new ActionListener()
    {
        public void actionPerformed(ActionEvent evt)
        {
            double interest = balance * rate / 100;
            balance += interest;
        }
    };
};
```

- ✂ This is saying, construct a new object of a class that implements the ActionListener interface, where the one required method (actionPerformed) is defined inside the brackets.



Abstract Data Types

Abstract Data Types

- An **abstract data type** is a mathematical set of data, along with operations defined on that kind of data
- Examples:
 - ♦ **int**: it is the set of integers (up to a certain magnitude), with operations $+$, $-$, $/$, $*$, $\%$
 - ♦ **double**: it's the set of decimal numbers (up to a certain magnitude), with operations $+$, $-$, $/$, $*$

Data Structures

- A data structure is a user-defined abstract data type
- Examples:
 - ◆ **Complex numbers:** with operations $+$, $-$, $/$, $*$, *magnitude*, *angle*, etc.
 - ◆ **Stack:** with operations *push*, *pop*, *peek*, *isempty*
 - ◆ **Queue:** *enqueue*, *dequeue*, *isempty* ...
 - ◆ **Binary Search Tree:** *insert*, *delete*, *search*.
 - ◆ **Heap:** *insert*, *min*, *delete-min*.

Data Structure Design

- Specification
 - ◆ A set of data
 - ◆ Specifications for a number of operations to be performed on the data
- Design
 - ◆ A lay-out organization of the data
 - ◆ Algorithms for the operations
- Goals of Design: **fast** operations

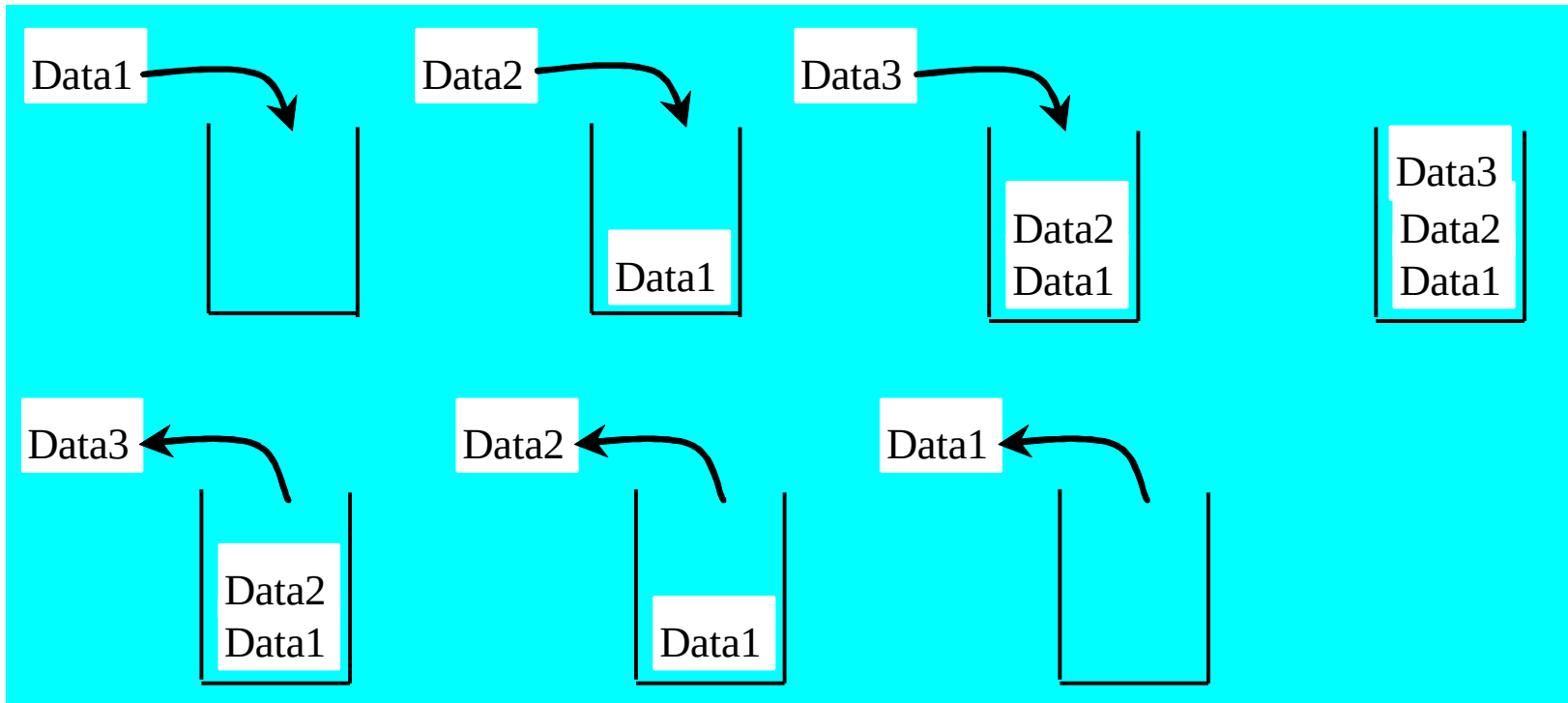
Implementation of a Data Structure

- Representation of the data using built-in data types of the programming language (such as int, double, char, strings, arrays, structs, classes, pointers, etc.)
- Language implementation (code) of the algorithms for the operations
- In OOP languages both the data representation and the operations are aggregated together into what is called **objects**
- The data type of such objects are called **classes**.
- Classes are blue prints, objects are instances.

Stack, Queue and List

Stacks

A stack can be viewed as a special type of list, where the elements are accessed, inserted, and deleted only from the end, called the top, of the stack.

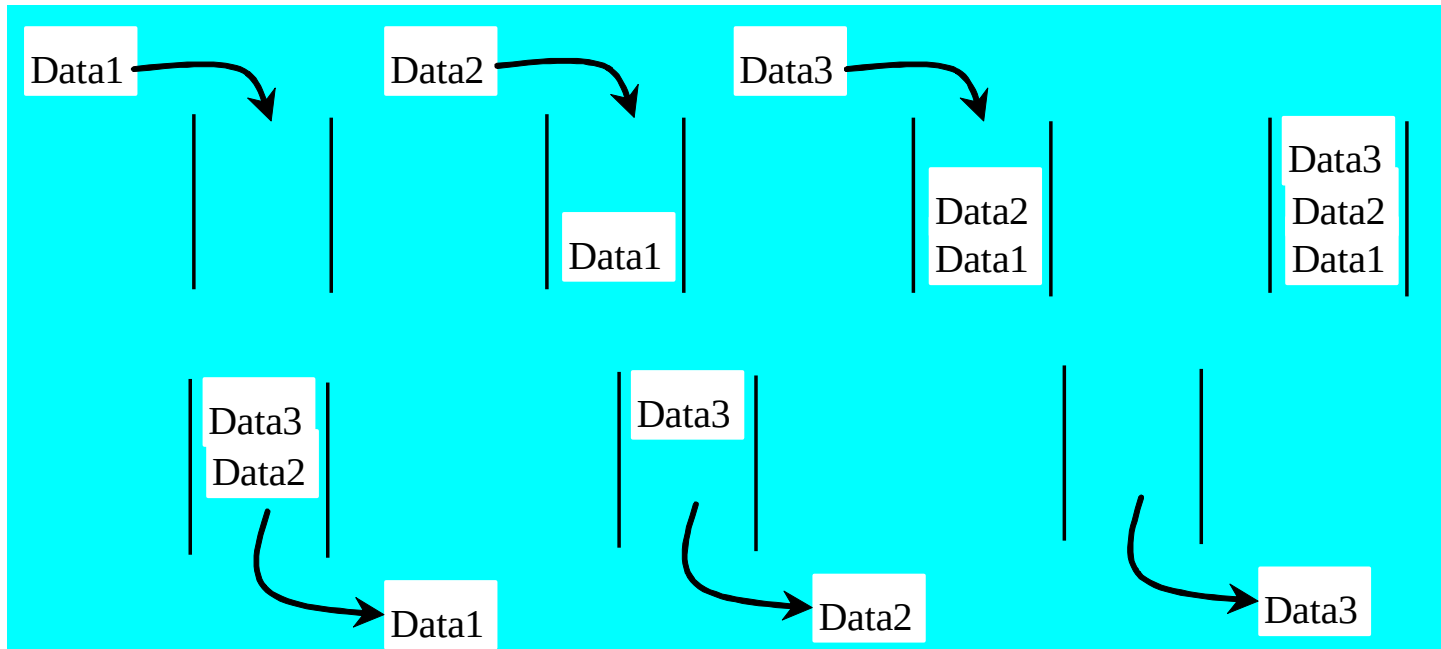


Stack

- A stack is maintained Last-In-First-Out (not unlike a stack of plates in a cafeteria)
- Standard operations
 - ◆ `isEmpty()` : returns `true` or `false`
 - ◆ `top()` : returns copy of value at top of stack (without removing it)
 - ◆ `push(v)` : adds a value `v` at the top of the stack
 - ◆ `pop()` : removes and returns value at top

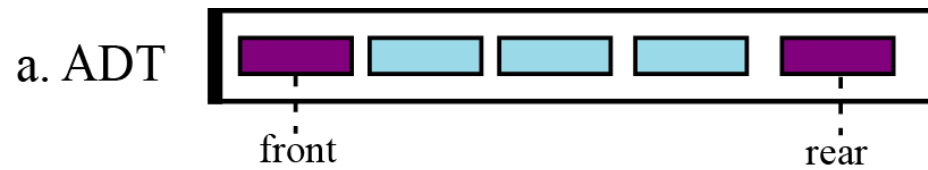
Queues

A queue represents a waiting list. A queue can be viewed as a special type of list, where the elements are inserted into the end (tail) of the queue, and are accessed and deleted from the beginning (head) of the queue.

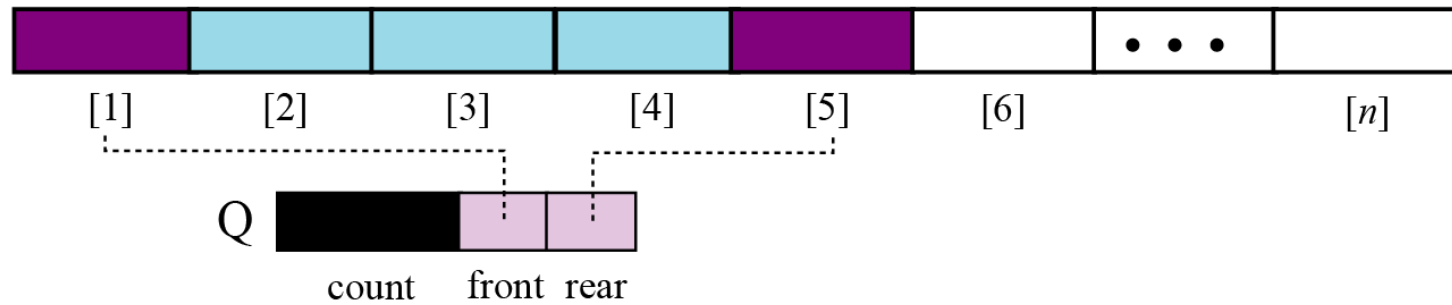


Queues

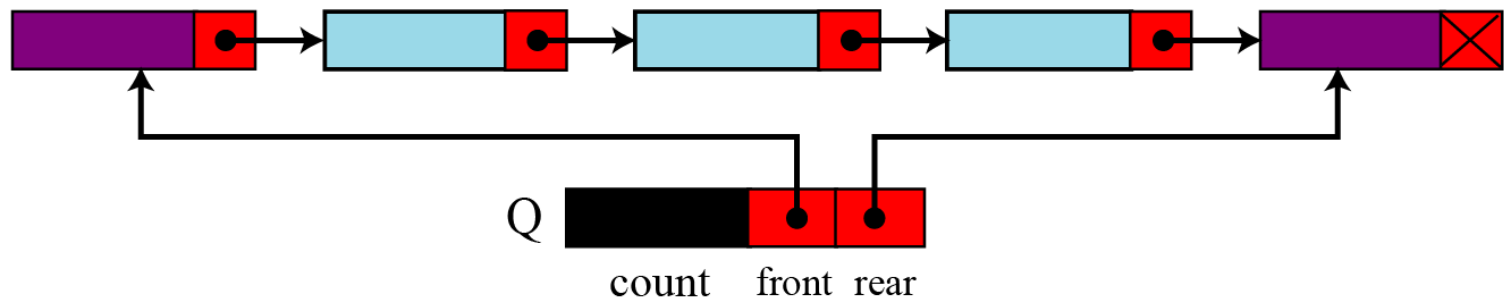
- Queue Manipulation Operations
 - ◆ `isEmpty()`: returns `true` or `false`
 - ◆ `first()`: returns copy of value at front
 - ◆ `add(v)`: adds a new value at rear of queue **Enqueue**
 - ◆ `remove()`: removes, returns value at front **Dequeue**



b. Array
implementation



c. Linked list
implementation



Queue implementation

Implementing Stacks and Queues

- Using an Arraylist to implement Stack
- Use a Linked list to implement Queue

Since the insertion and deletion operations on a stack are made only at the end of the stack, using an array list to implement a stack is more efficient than a linked list.

Since deletions are made at the beginning of the list, it is more efficient to implement a queue using a linked list than an array list.

Design of the Stack and Queue Classes

There are two ways to design the stack and queue classes:

- ◆ Using inheritance: You can declare the stack class by extending the array list class, and the queue class by extending the linked list class.



- Using composition: You can declare an array list as a data field in the stack class, and a linked list as a data field in the queue class.



Composition is Better

Both designs are fine, but using composition is better because it enables you to declare a complete new stack class and queue class without inheriting the unnecessary and inappropriate methods from the array list and linked list.

MyStack and MyQueue

MyStack

-list: MyArrayList

+isEmpty(): boolean

+getSize(): int

+peek(): Object

+pop(): Object

+push(o: Object): Object

+search(o: Object): int

MyStack

Returns true if this stack is empty.

Returns the number of elements in this stack.

Returns the top element in this stack.

Returns and removes the top element in this stack.

Adds a new element to the top of this stack.

Returns the position of the specified element in this stack.

MyQueue

-list: MyLinkedList

+enqueue(element: Object): void

+dequeue(): Object

+getSize(): int

MyQueue

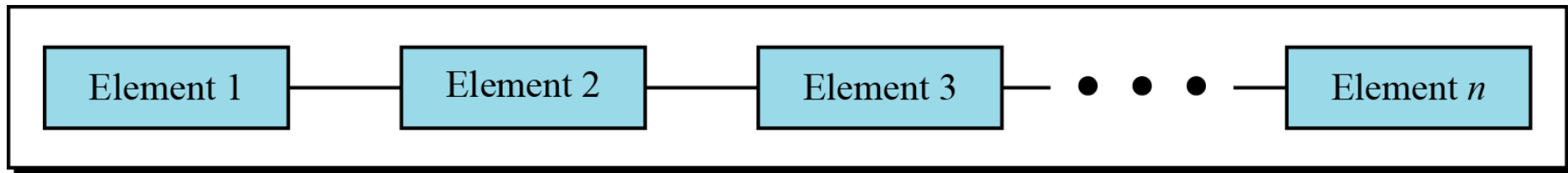
Adds an element to this queue.

Removes an element from this queue.

Returns the number of elements from this queue.

GENERAL LINEAR LISTS

- Stacks and queues defined in the two previous sections are **restricted linear lists**.
- A **general linear list** is a list in which operations, such as insertion and deletion, can be done anywhere in the list—at the beginning, in the middle or at the end. Figure shows a general linear list.



General linear list

Operations on general linear lists

Six common operations: *list*, *insert*, *delete*, *retrieve*, *traverse* and *empty*.

The *list* operation

The list operation creates an empty list. The following shows the format:

```
list (listName)
```

List features

- **ORDERING:** maintains order elements were added (new elements are added to the end by default)
- **DUPLICATES:** yes (allowed)
- **OPERATIONS:** add element to end of list, insert element at given index, clear all elements, search for element, get element at given index, remove element at given index, get size
 - some of these operations are inefficient! (seen later)
- list manages its own size; user of the list does not need to worry about overfilling it

Java's **List** interface

- Java also has an interface `java.util.List` to represent a list of objects:
(a partial list)

```
public void add(int index, Object o)
```

Inserts the specified element at the specified position in this list.

```
public Object get(int index)
```

Returns the element at the specified position in this list.

```
public int indexOf(Object o)
```

Returns the index in this list of the first occurrence of the specified element, or `-1` if the list does not contain it.

List interface, cont'd.

```
public int lastIndexOf(Object o)
```

Returns the index in this list of the last occurrence of the specified element, or -1 if the list does not contain it.

```
public Object remove(int index)
```

Removes the object at the specified position in this list.

```
public Object set(int index, Object o)
```

Replaces the element at the specified position in this list with the specified element.

- Notice that the methods added to `Collection` by `List` all deal with indexes; a list has indexes while a general collection may not

Some list questions

- all of the list operations on the previous slide could be performed using an array instead!
- open question: What are some reasons why we might want to use a list class, rather than an array, to store our data?
- thought question: How might a List be implemented, under the hood?
- why do all the List methods use type `Object`?

List example

List Iterations

A particularly slow idiom

```
// print every element of linked list
for (int i = 0; i < list.size(); i++) {
    Object element = list.get(i);
    System.out.println(i + ": " +
        element);
}
```

- This code executes an $O(n)$ operation (`get`) every time through a loop that runs n times!
 - Its runtime is $O(n^2)$, which is much worse than $O(n)$
 - this code will take prohibitively long to run for large data sizes

The problem of position

- The code on the previous slide is wasteful because it throws away the position each time
 - every call to `get` has to re-traverse the list!
- it would be much better if we could somehow keep the list in place at each index as we looped through it
- Java uses special objects to represent a position of a collection as it's being examined...
 - these objects are called "iterators"

List Iterator

- `ListIterator` type
- Gives access to elements inside a linked list
- Encapsulates a position anywhere inside the linked list
- Protects the linked list while giving access

A List Iterator

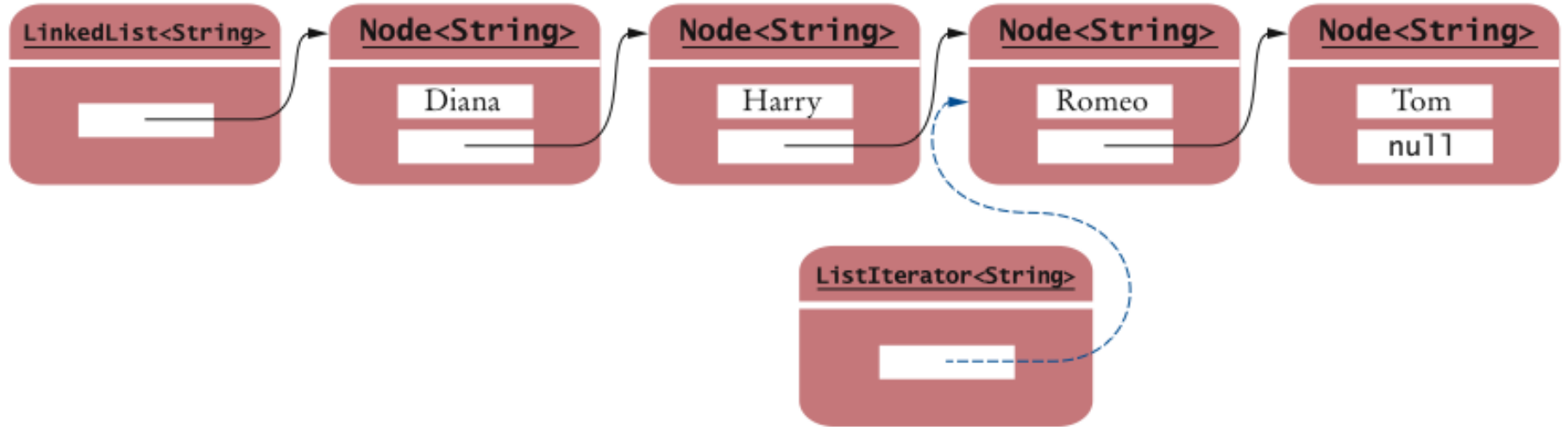


Figure 2 A List Iterator

A Conceptual View of the List Iterator

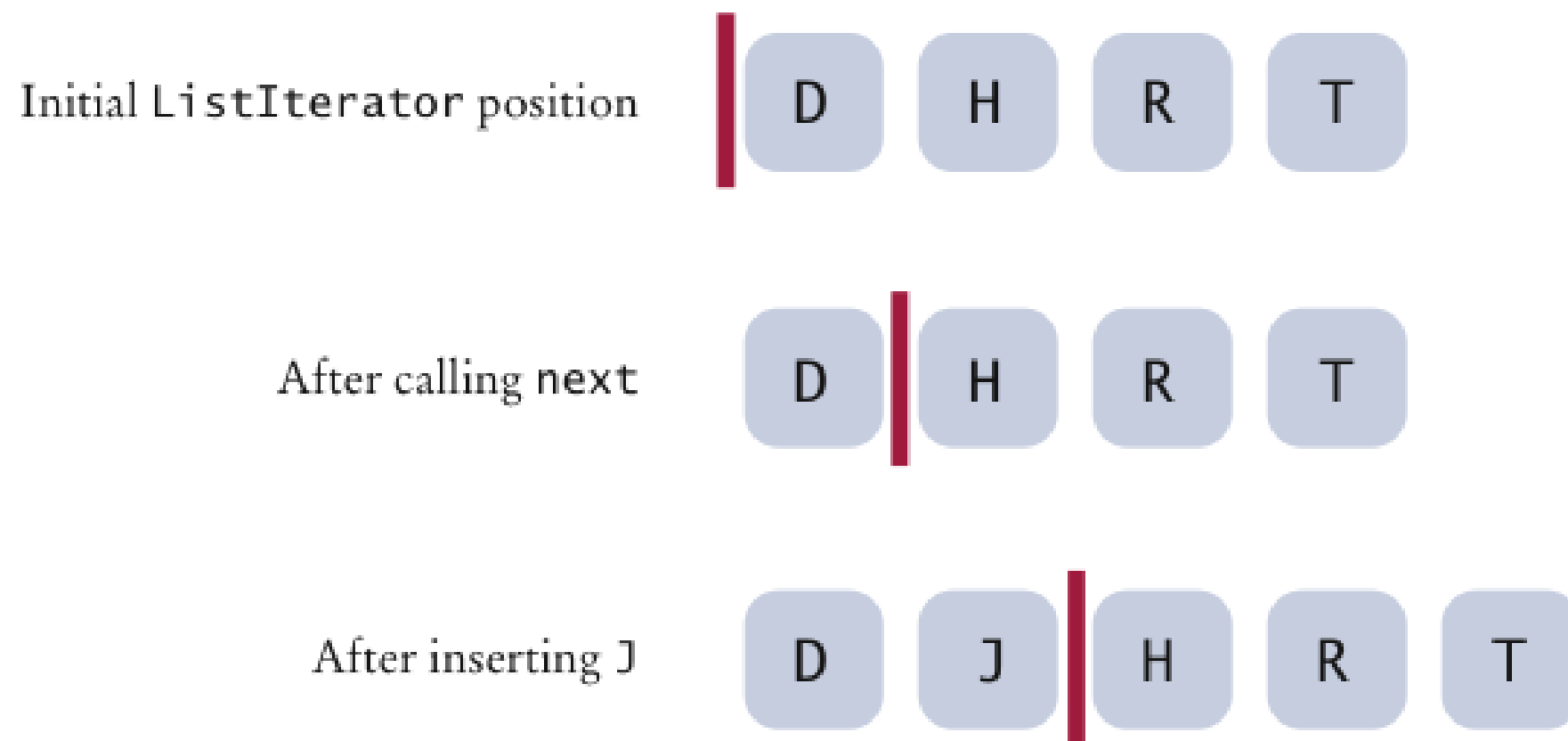


Figure 3 A Conceptual View of the List Iterator

List Iterator

- Think of an iterator as pointing between two elements
 - *Analogy: Like the cursor in a word processor points between two characters*
- The listIterator method of the LinkedList class gets a list iterator

```
LinkedList<String> employeeNames = ...;
```

```
ListIterator<String> iterator =  
    employeeNames.listIterator();
```


List Iterator

- Initially, the iterator points before the first element
- The `next` method moves the iterator:

`iterator.next();`

- `next` throws a `NoSuchElementException` if you are already past the end of the list
- `hasNext` returns true if there is a next element:

**`if (iterator.hasNext())
 iterator.next();`**

List Iterator

- The `next` method returns the element that the iterator is passing:

```
while iterator.hasNext()  
{  
    String name = iterator.next();  
    //Do something with name  
}
```

List Iterator

- Shorthand for loop:

```
for (String name : employeeNames)
{
    // Do something with name
}
```

Behind the scenes, the `for` loop uses an iterator to visit all list elements

List Iterator

- LinkedList is a *doubly linked list*
 - *Class stores two links:*
 - *One to the next element, and*
 - *One to the previous element*
- To move the list position backwards, use:
 - *hasPrevious*
 - *previous*

Adding and Removing from a LinkedList

- The add method:
 - *Adds an object after the iterator*
 - *Moves the iterator position past the new element:*
iterator.add("Juliet");

Adding and Removing from a LinkedList

- The remove method
 - *Removes and*
 - *Returns the object that was returned by the last call to next or previous*

//Remove all names that fulfill a certain condition

while (iterator.hasNext())

{

String name = iterator.next();

if (name fulfills condition)

iterator.remove(); }

Adding and Removing from a LinkedList

- Be careful when calling `remove`:

- *It can be called **only once** after calling `next` or `previous`:*

```
iterator.next();
```

```
iterator.next();
```

```
iterator.remove();
```

```
iterator.remove();
```

```
// Error: You cannot call remove twice.
```

- ***You cannot call it immediately after a call to `add`:***

```
iter.add("Fred");
```

```
iter.remove(); // Error: Can only call remove after  
               // calling next or previous
```

- *If you call it improperly, it throws an `IllegalStateException`*

Methods of the `ListIterator` Interface

Table 2 Methods of the `ListIterator` Interface

<code>String s = iter.next();</code>	Assume that <code>iter</code> points to the beginning of the list <code>[Sally]</code> before calling <code>next</code> . After the call, <code>s</code> is "Sally" and the iterator points to the end.
<code>iter.hasNext()</code>	Returns <code>false</code> because the iterator is at the end of the collection.
<code>if (iter.hasPrevious()) { s = iter.previous(); }</code>	<code>hasPrevious</code> returns <code>true</code> because the iterator is not at the beginning of the list.
<code>iter.add("Diana");</code>	Adds an element before the iterator position. The list is now <code>[Diana, Sally]</code> .
<code>iter.next(); iter.remove();</code>	<code>remove</code> removes the last element returned by <code>next</code> or <code>previous</code> . The list is again <code>[Diana]</code> .

Sample Program

- `ListTester` is a sample program that
 - *Inserts strings into a list*
 - *Iterates through the list, adding and removing elements*
 - *Prints the list*

ListTester.java

```
1  import java.util.LinkedList;
2  import java.util.ListIterator;
3
4  /**
5   A program that tests the LinkedList class
6   */
7  public class ListTester
8  {
9      public static void main(String[] args)
10     {
11         LinkedList<String> staff = new LinkedList<String>();
12         staff.addLast("Diana");
13         staff.addLast("Harry");
14         staff.addLast("Romeo");
15         staff.addLast("Tom");
16
17         // | in the comments indicates the iterator position
18
19         ListIterator<String> iterator = staff.listIterator(); // |DHRT
20         iterator.next(); // D|HRT
21         iterator.next(); // DH|RT
22
```

Continued

ListTester.java (cont.)

```
23      // Add more elements after second element
24
25      iterator.add("Juliet"); // DHJ|RT
26      iterator.add("Nina"); // DHJN|RT
27
28      iterator.next(); // DHJNR|T
29
30      // Remove last traversed element
31
32      iterator.remove(); // DHJN|T
33
34      // Print all elements
35
36      for (String name : staff)
37          System.out.print(name + " ");
38      System.out.println();
39      System.out.println("Expected: Diana Harry Juliet Nina Tom");
40  }
41 }
```

Continued

ListTester.java (cont.)

Program Run:

```
Diana Harry Juliet Nina Tom
```

```
Expected: Diana Harry Juliet Nina Tom
```

Self Check

Do linked lists take more storage space than arrays of the same size?

Answer: Yes, for two reasons. You need to store the node references, and each node is a separate object. (There is a fixed overhead to store each object in the virtual machine.)

Self Check

Why don't we need iterators with arrays?

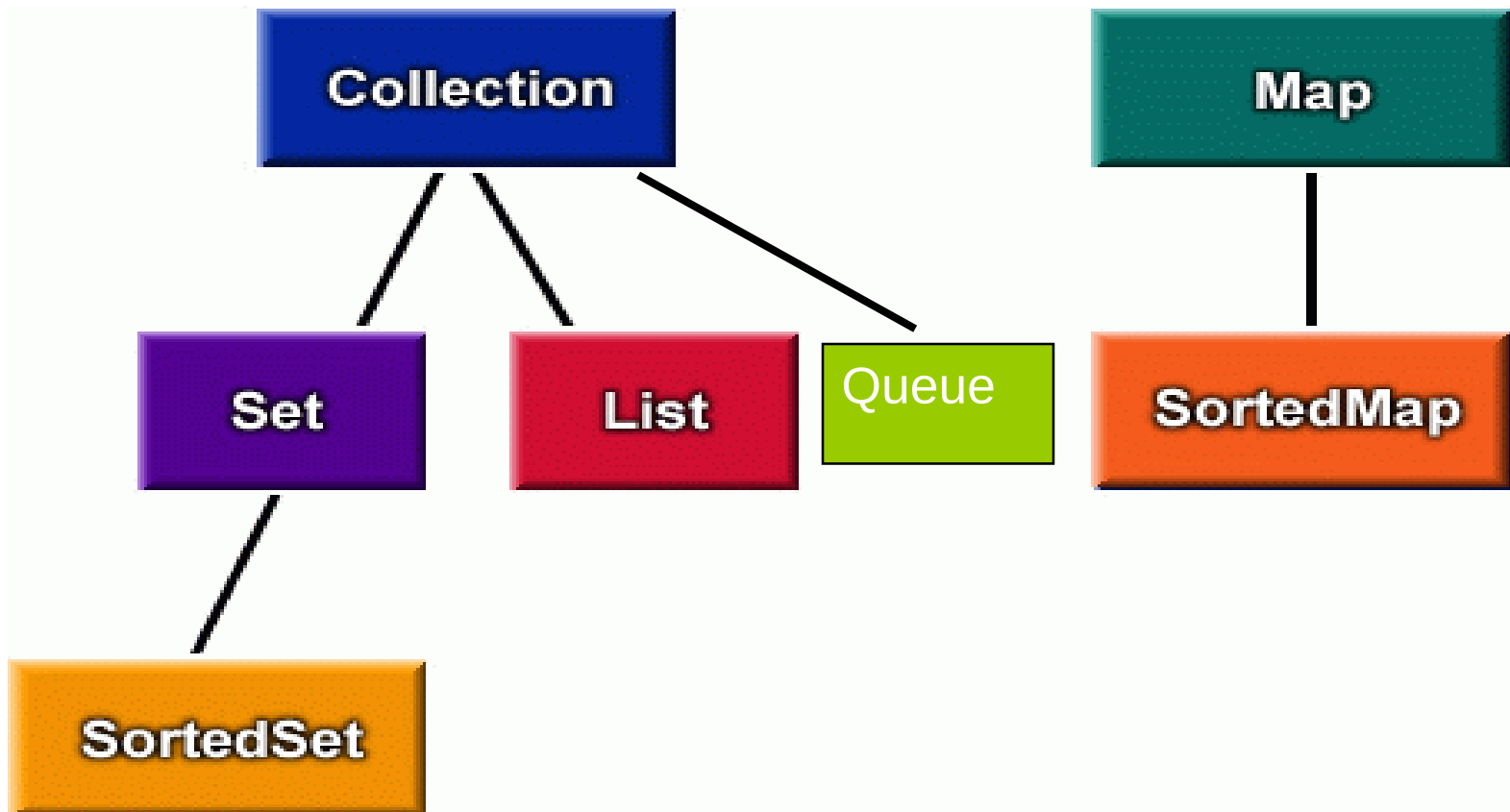
Answer: An integer index can be used to access any array location.

Java Collection Framework

Collection Framework

- ◆ A *collection framework* is a unified architecture for representing and manipulating collections. It has:
 - **Interfaces:** abstract data types representing collections
 - **Implementations:** concrete implementations of the collection interfaces
 - **Algorithms:** methods that perform useful computations, such as searching and sorting
 - These algorithms are said to be *polymorphic*: the same method can be used on different implementations

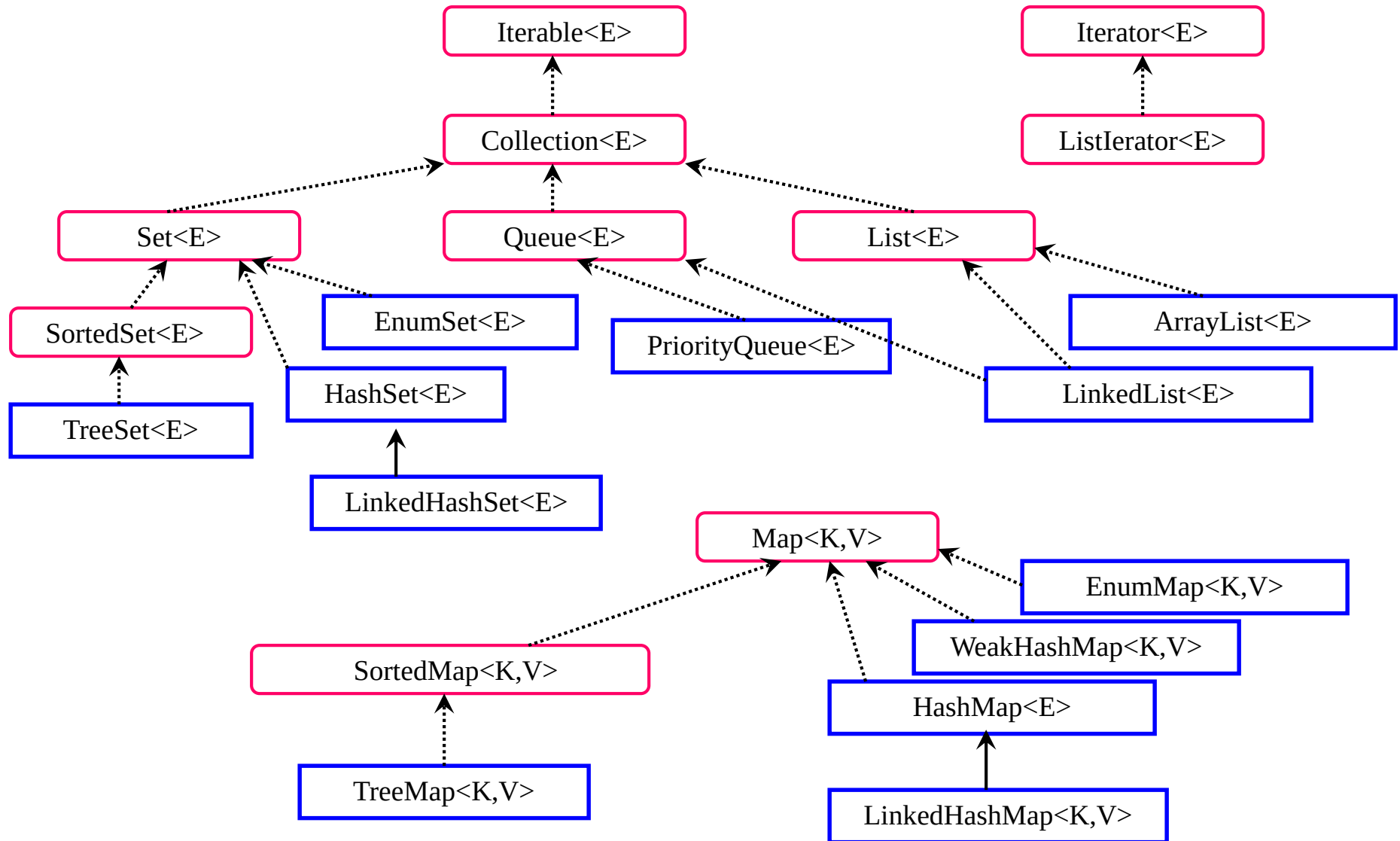
Collection interfaces



Collection Interface continued

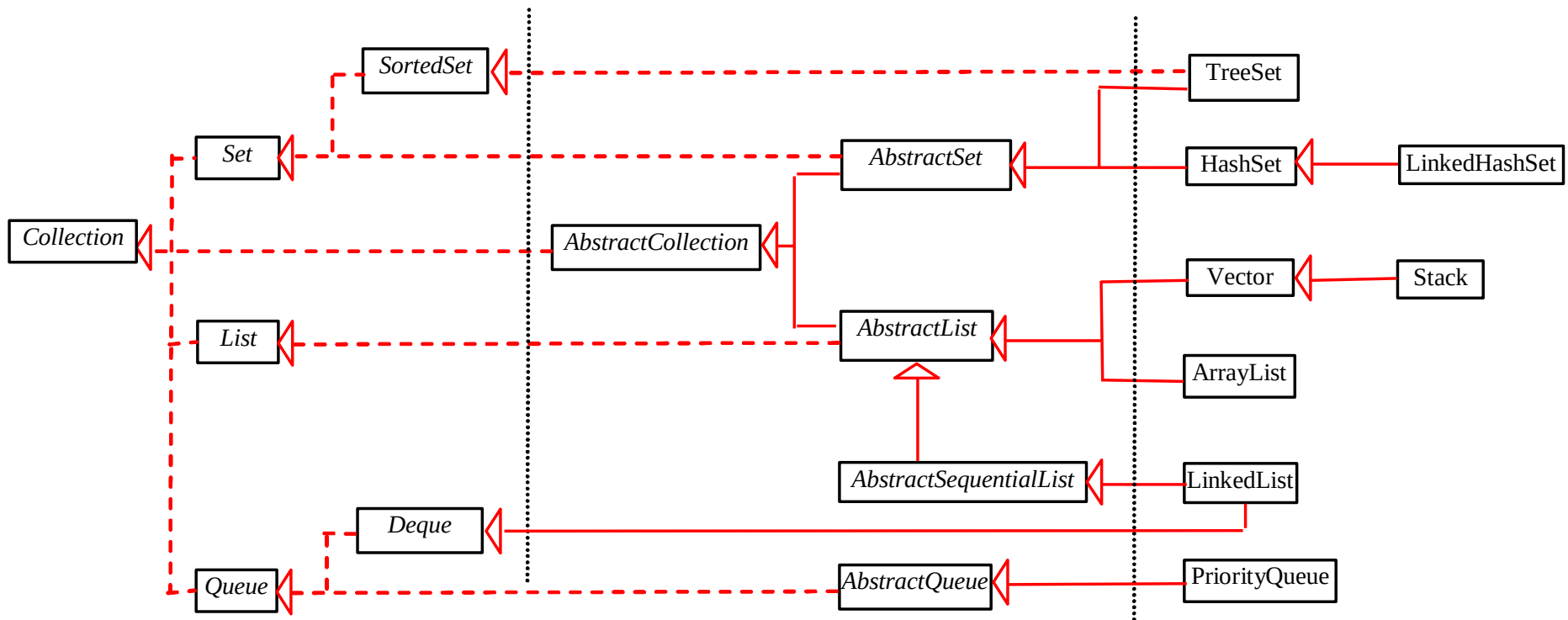
- **Set** →
 - ◆ The familiar set abstraction.
 - ◆ No duplicates; May or may not be ordered.
- **List** →
 - ◆ Ordered collection, also known as a sequence.
 - ◆ Duplicates permitted; Allows positional access
- **Map** →
 - ◆ A mapping from keys to values.
 - ◆ Each key can map to at most one value (function).
 - ◆ The keys are like indexes. In List, the indexes are integer. In Map, the keys can be any objects.
- **Queue** →
 - ◆ Ordered collection. FIFO (First In First Out)

Type Trees for Collections



Java Collection Framework hierarchy, cont.

Set and List are subinterfaces of Collection.

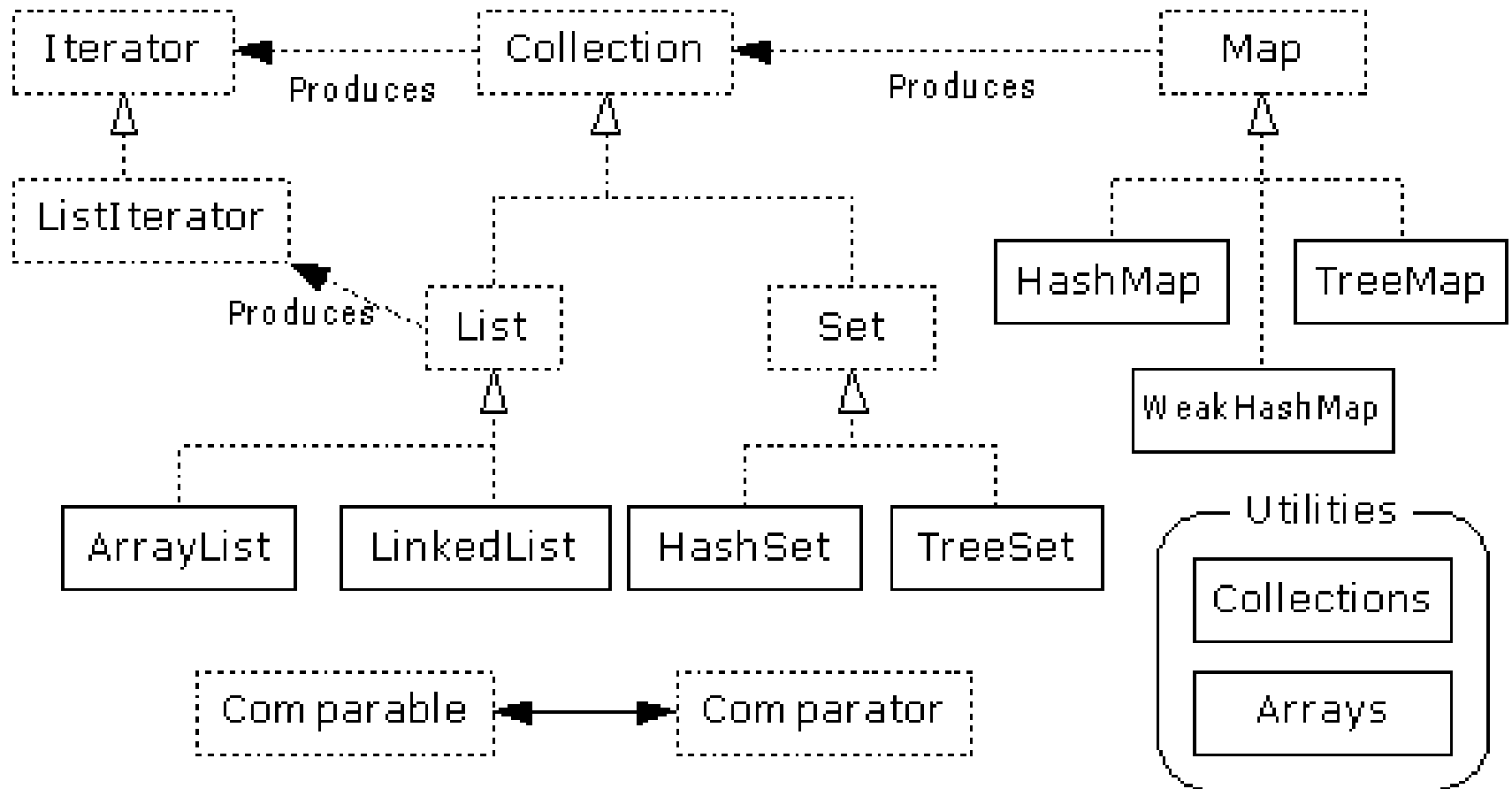


Interfaces

Abstract Classes

Concrete Classes

Collections Framework Diagram



Collection Interface

- Defines fundamental methods
 - ♦ **int size();**
 - ♦ **boolean isEmpty();**
 - ♦ **boolean contains(Object element);**
 - ♦ **boolean add(Object element); // Optional**
 - ♦ **boolean remove(Object element); // Optional**
 - ♦ **Iterator iterator();**
- These methods are enough to define the basic behavior of a collection
- Provides an Iterator to step through the elements in the Collection

Interface Collection

• add(o)	Add a new element
• addAll(c)	Add a collection
• clear()	Remove all elements
• contains(o)	Membership checking.
• containsAll(c)	Inclusion checking
• isEmpty()	Whether it is empty
• iterator()	Return an iterator
• remove(o)	Remove an element
• removeAll(c)	Remove a collection
• retainAll(c)	Keep the elements
• size()	The number of elements

Iterator Interface

- Defines three fundamental methods
 - ◆ **Object next()**
 - ◆ **boolean hasNext()**
 - ◆ **void remove()**
- These three methods provide access to the contents of the collection
- An Iterator knows position within collection
- Each call to next() “reads” an element from the collection
 - ◆ Then you can use it or remove it

Iterator Position

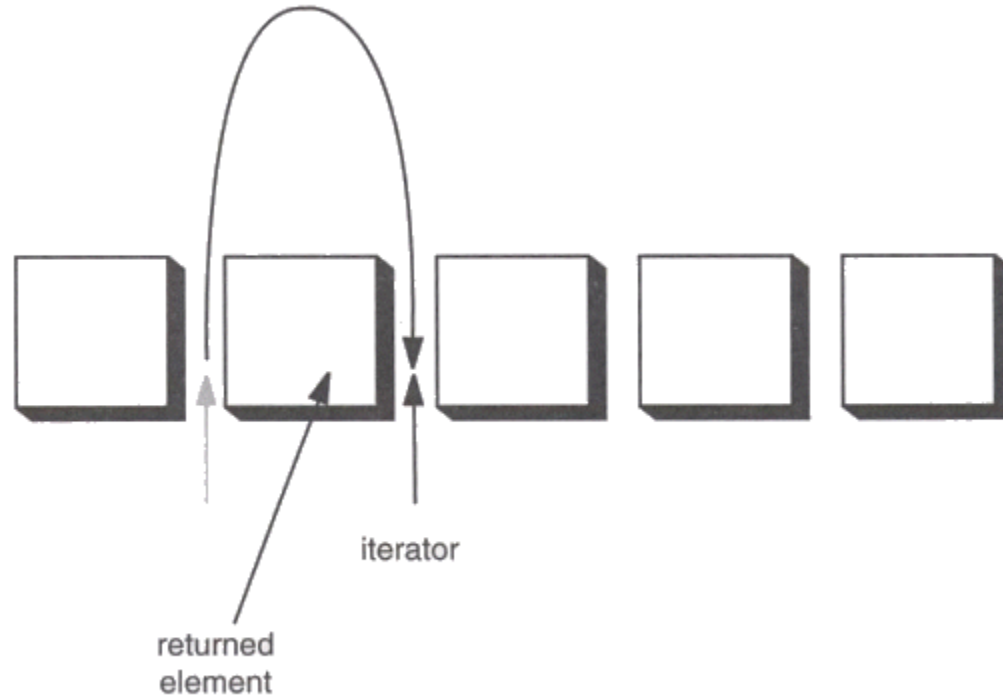
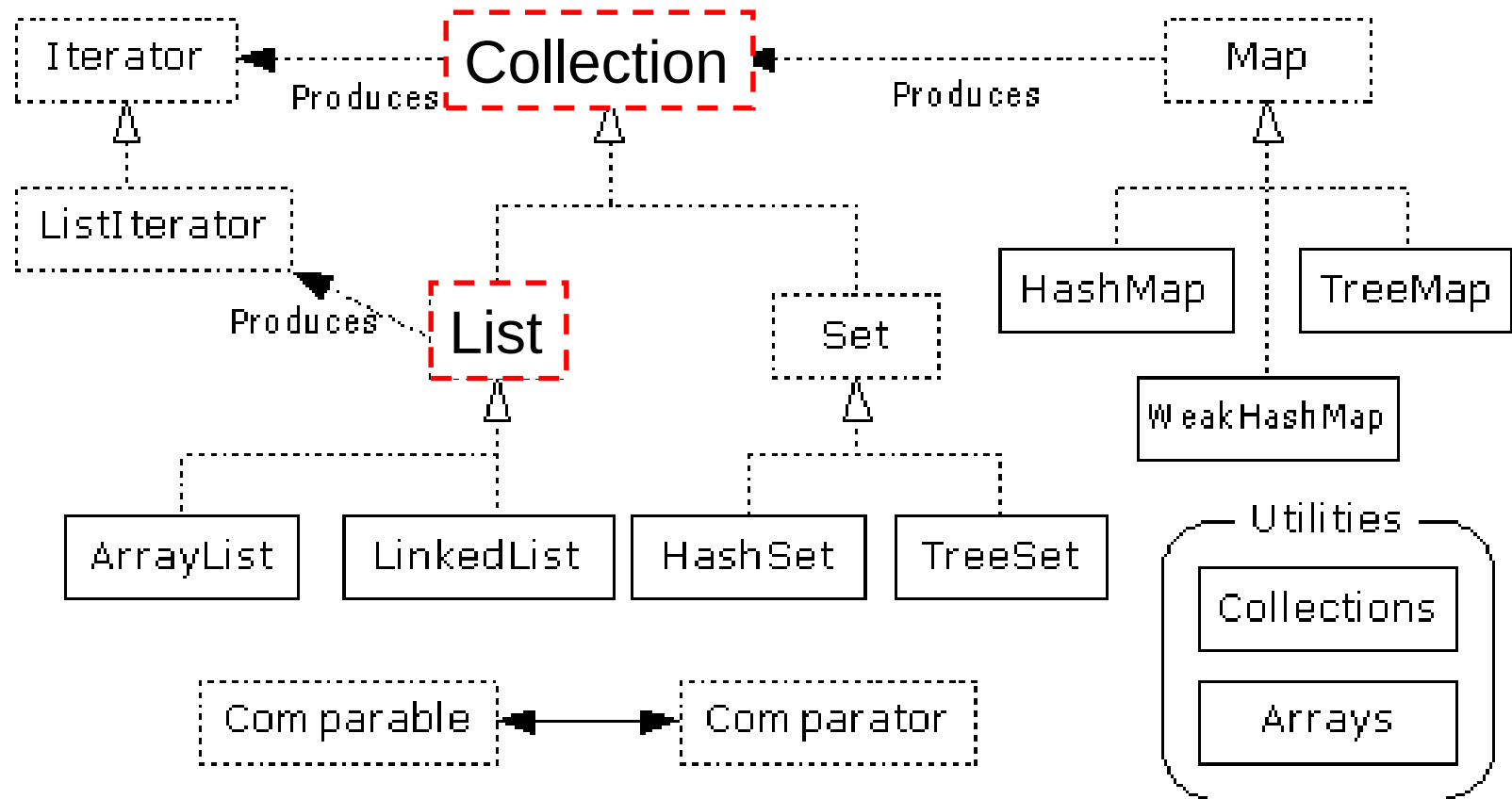


Figure 2-3: Advancing an iterator

Example - SimpleCollection

```
public class SimpleCollection {  
    public static void main(String[] args) {  
        Collection c;  
        c = new ArrayList();  
        System.out.println(c.getClass().getName());  
        for (int i=1; i <= 10; i++) {  
            c.add(i + " * " + i + " = "+i*i);  
        }  
        Iterator iter = c.iterator();  
        while (iter.hasNext())  
            System.out.println(iter.next());  
    }  
}
```

List Interface Context



List Interface

- The List interface adds the notion of *order* to a collection
- The user of a list has control over where an element is added in the collection
- Lists typically allow *duplicate* elements
- Provides a **ListIterator** to step through the elements in the list.

ListIterator Interface

- Extends the Iterator interface
- Defines three fundamental methods
 - ◆ **void add(Object o)** - before current position
 - ◆ **boolean hasPrevious()**
 - ◆ **Object previous()**
- The addition of these three methods defines the basic behavior of an ordered list
- A ListIterator knows position within list

Iterator Position - `next()` , `previous()`

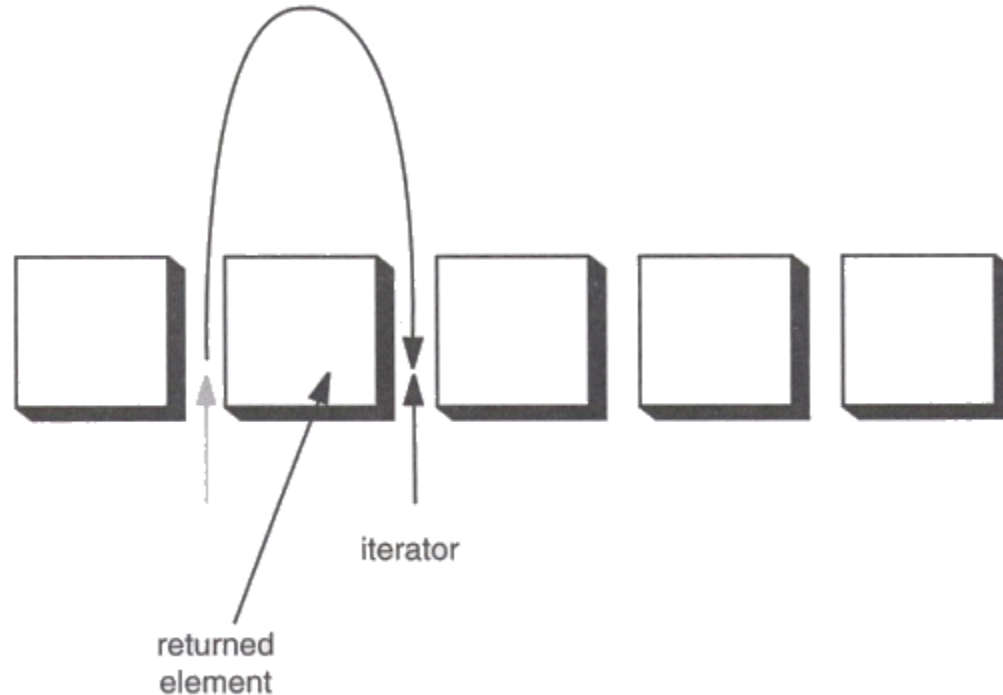
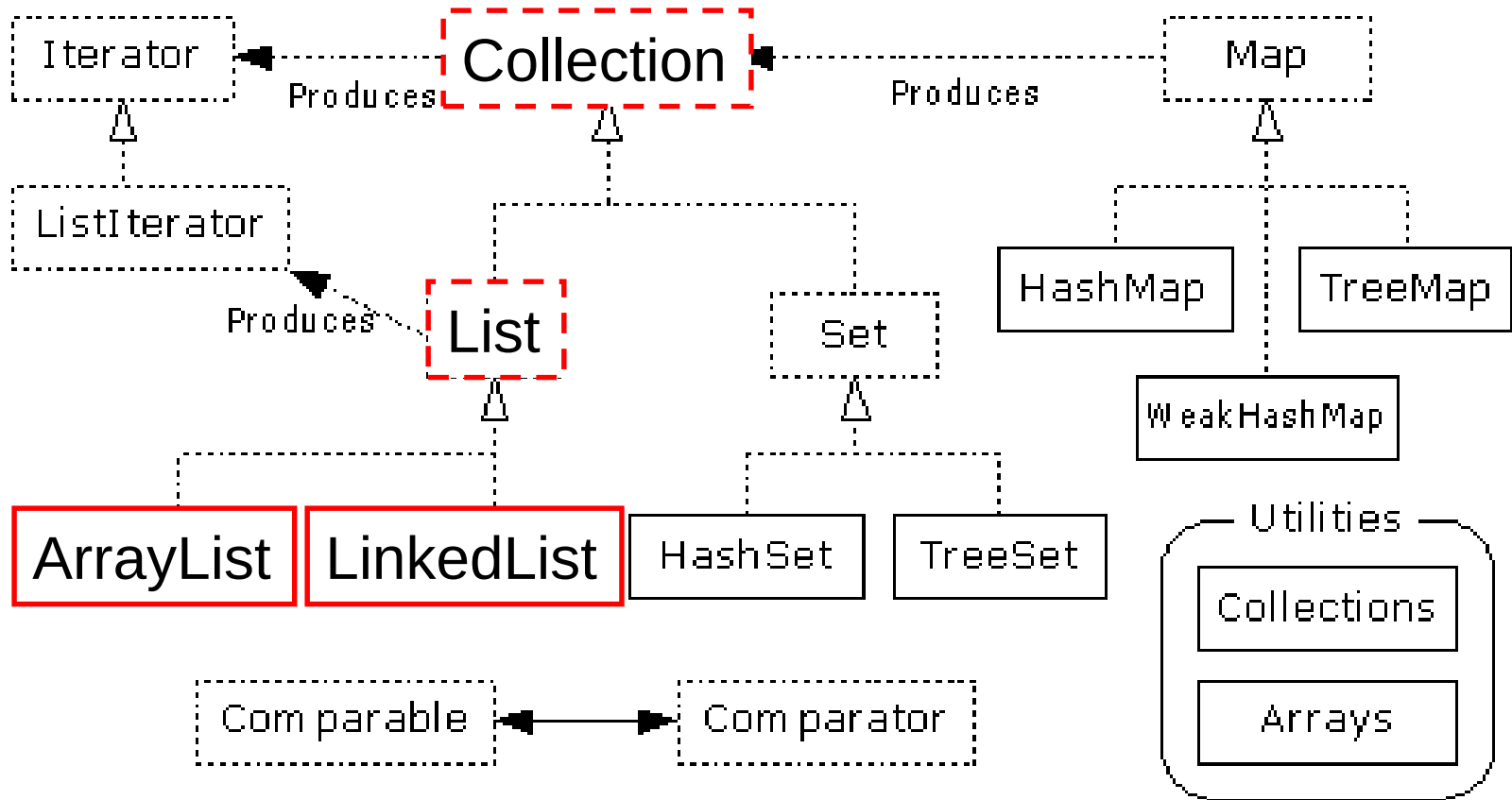


Figure 2-3: Advancing an iterator

ArrayList and LinkedList Context



List Implementations

- **ArrayList**
 - ◆ low cost random access
 - ◆ high cost insert and delete
 - ◆ array that resizes if need be
- **LinkedList**
 - ◆ sequential access
 - ◆ low cost insert and delete
 - ◆ high cost random access

ArrayList overview

- Constant time positional access (it's an array)
- One tuning parameter, the initial capacity

```
public ArrayList(int initialCapacity) {  
    super();  
    if (initialCapacity < 0)  
        throw new IllegalArgumentException(  
            "Illegal Capacity: "+initialCapacity);  
    this.elementData = new Object[initialCapacity];  
}
```

ArrayList methods

- The indexed get and set methods of the List interface are appropriate to use since ArrayLists are backed by an array
 - ♦ **Object get(int index)**
 - ♦ **Object set(int index, Object element)**
- Indexed add and remove are provided, but can be costly if used frequently
 - ♦ **void add(int index, Object element)**
 - ♦ **Object remove(int index)**
- May want to resize in one shot if adding many elements
 - ♦ **void ensureCapacity(int minCapacity)**

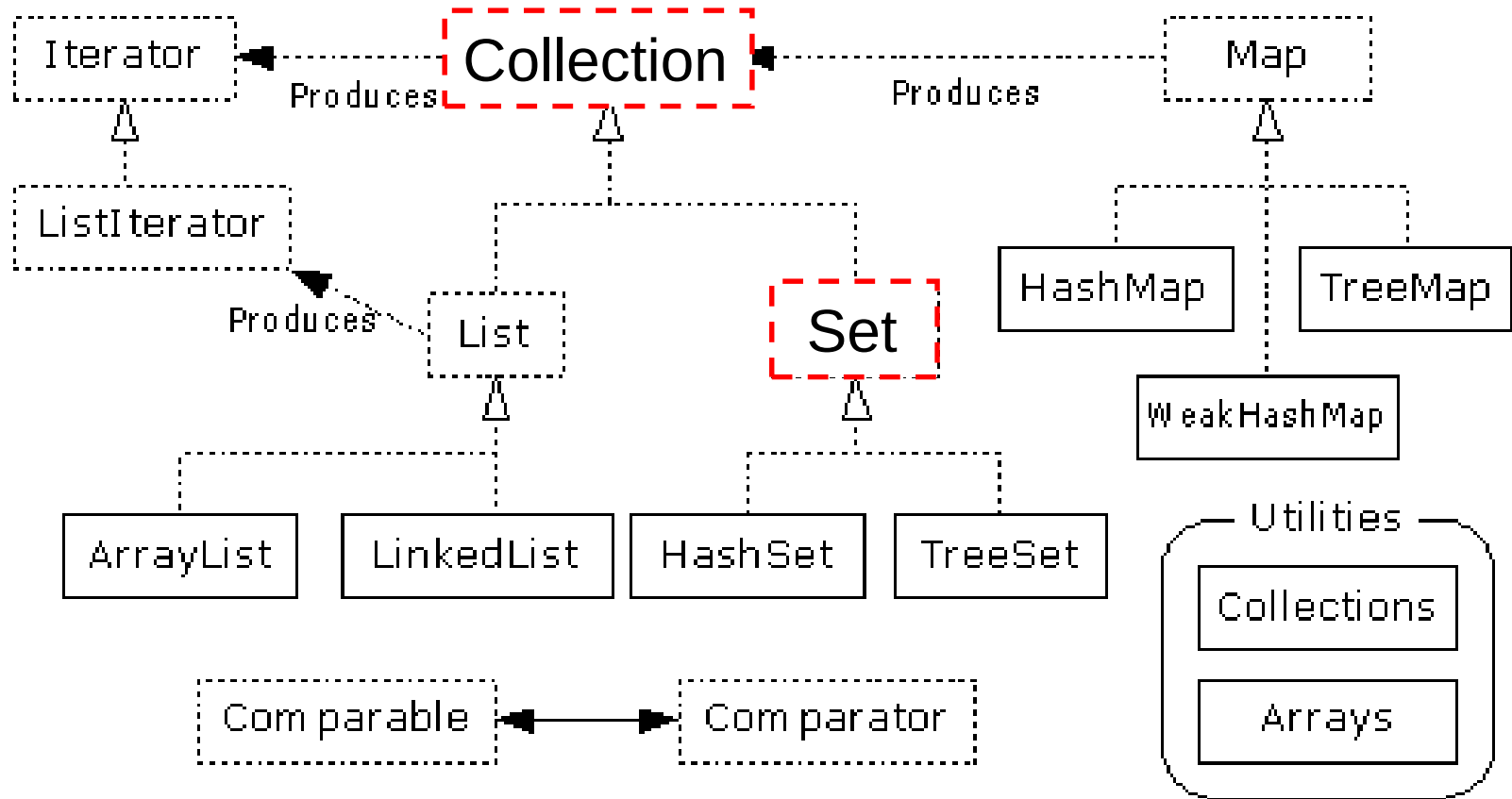
LinkedList overview

- Stores each element in a node
- Each node stores a link to the next and previous nodes
- Insertion and removal are inexpensive
 - ♦ just update the links in the surrounding nodes
- Linear traversal is inexpensive
- Random access is expensive
 - ♦ Start from beginning or end and traverse each node while counting

LinkedList methods

- The list is sequential, so access it that way
 - ♦ **ListIterator listIterator()**
- ListIterator knows about position
 - ♦ use **add()** from ListIterator to add at a position
 - ♦ use **remove()** from ListIterator to remove at a position
- LinkedList knows a few things too
 - ♦ **void addFirst(Object o), void addLast(Object o)**
 - ♦ **Object getFirst(), Object getLast()**
 - ♦ **Object removeFirst(), Object removeLast()**

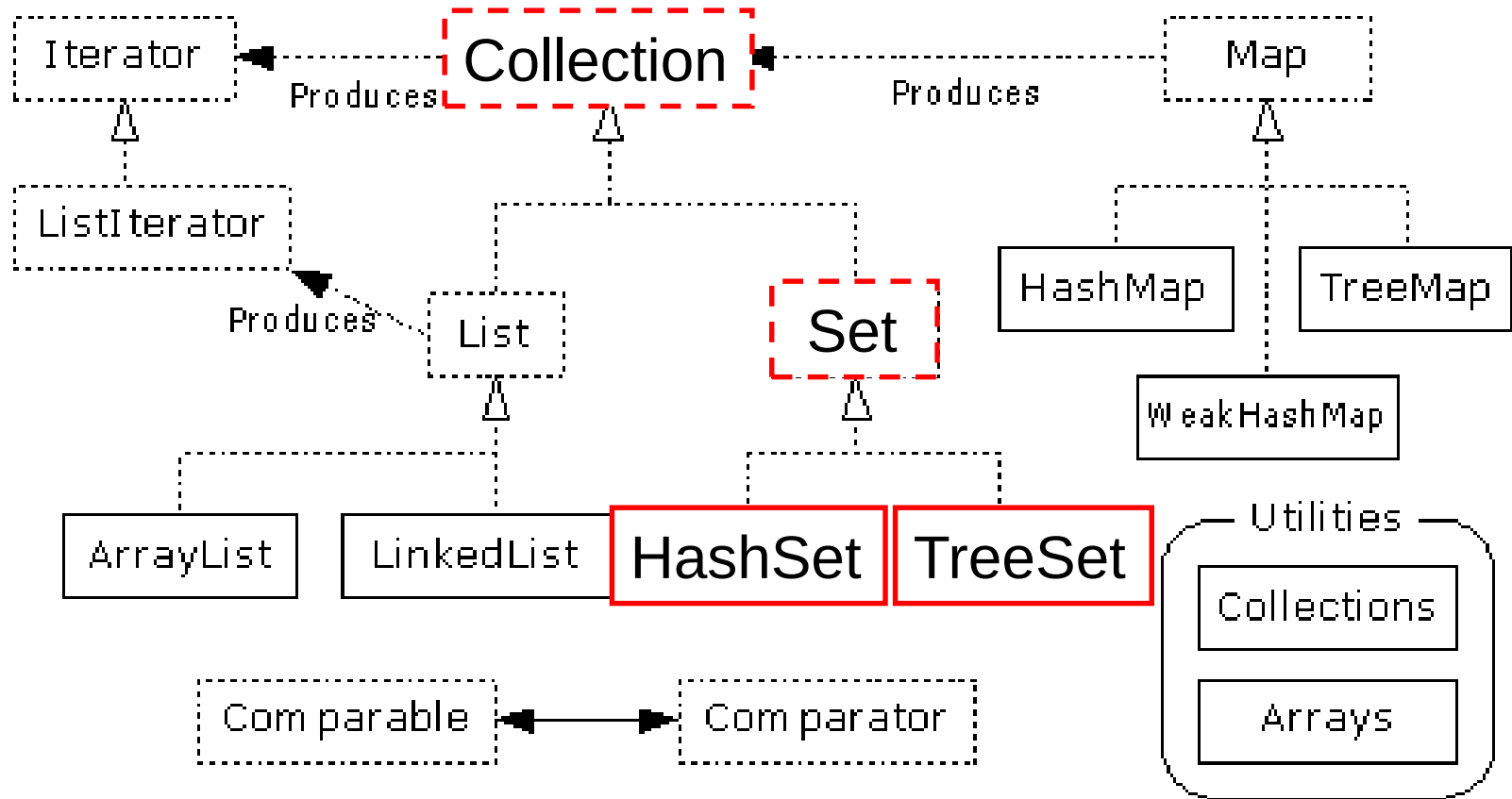
Set Interface Context



Set Interface

- Same methods as Collection
 - ◆ different contract - no duplicate entries
- Defines two fundamental methods
 - ◆ **boolean add(Object o)** - reject duplicates
 - ◆ **Iterator iterator()**
- Provides an Iterator to step through the elements in the Set
 - ◆ No guaranteed order in the basic Set interface
 - ◆ There is a SortedSet interface that extends Set

HashSet and TreeSet Context



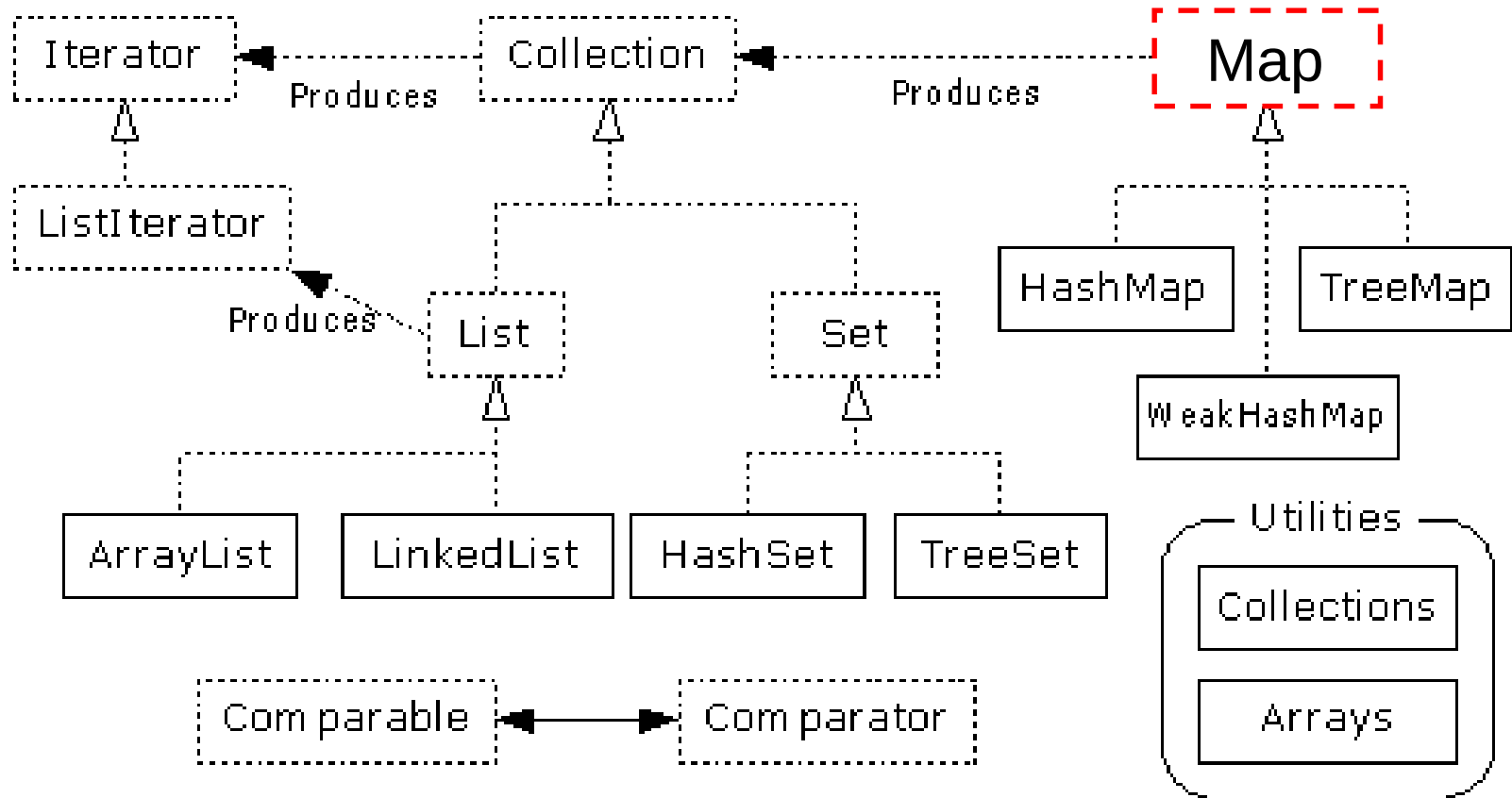
HashSet

- Find and add elements very quickly
 - ◆ uses hashing implementation in HashMap
- Hashing uses an array of linked lists
 - ◆ The **hashCode()** is used to index into the array
 - ◆ Then **equals()** is used to determine if element is in the (short) list of elements at that index
- No order imposed on elements
- The **hashCode()** method and the **equals()** method must be compatible
 - ◆ if two objects are equal, they must have the same **hashCode()** value

TreeSet

- Elements can be inserted in any order
- The TreeSet stores them in order
- An iterator always presents them in order
- Default order is defined by natural order
 - ◆ objects implement the Comparable interface
 - ◆ TreeSet uses **compareTo(Object o)** to sort

Map Interface Context



Map Interface

- Stores **key/value** pairs
- Maps from the key to the value
- Keys are unique
 - ◆ a single key only appears once in the Map
 - ◆ a key can map to only one value
- Values do not have to be unique

Map methods

Object put(Object key, Object value)

Object get(Object key)

Object remove(Object key)

boolean containsKey(Object key)

boolean containsValue(Object value)

int size()

boolean isEmpty()

Object Oriented Programming

Generics

Why generics?

- `Person[] people = new Person[25];` // you must say what's in the array
`people[0] = "Sally";` // syntax error
- `ArrayList people = new ArrayList();` // but *anything* could go in the ArrayList!
`people.add("Sally");`
 // sometime later...
`Person p = (Person)people.get(0);` // runtime error
- `ArrayList<Person> people = new ArrayList<Person>();` // say what's in it
`people.add("Sally");` // syntax error
- Since Java 5, collections should be used only with generics

Generics

- A **generic** is a method that is recompiled with different types as the need arises
- The bad news:
 - Instead of saying: `List words = new ArrayList();`
 - You'll have to say:
`List<String> words = new ArrayList<String>();`
- The good news:
 - Replaces runtime type checks with compile-time checks
 - No casting; instead of
`String title = (String) words.get(i);`
you use
`String title = words.get(i);`
- Some classes and interfaces that have been “genericized” are: `Vector`, `ArrayList`, `LinkedList`, `Hashtable`, `HashMap`, `Stack`, `Queue`, `PriorityQueue`, `Dictionary`, `TreeMap` and `TreeSet`

Genericized types are still types

- ArrayList myList = new ArrayList();
- ArrayList<String> myList = new ArrayList<String> ();
 ↑ this is the type ↑ this is the type again
- You can use generic types as method parameters:
`String findLongest(ArrayList<String> myList) { ... }`
 - But you ***don't*** mention types when you call a method:
`String longestString = findLongest(myList);`
- You can return a generic type from a method:
`ArrayList<String> readList() { ... }`

Generic Iterators

- To iterate over generic collections, it's a good idea to use a generic iterator
 - `List<String> listOfStrings = new LinkedList<String>();`
 - ...
 - `for (Iterator<String> i =
listOfStrings.iterator(); i.hasNext();) {
 String s = i.next();
 System.out.println(s);
}`

Type wildcards

- Here's a simple (no generics) method to print out any list:

```
– private void printList(List list) {  
    for (Iterator i = list.iterator(); i.hasNext(); ) {  
        System.out.println(i.next());  
    }  
}
```

- The above still works in Java, but now it generates warning messages
- You should eliminate *all* errors and warnings in your final code, so you need to *tell* Java that any type is acceptable:

```
– private void printListOfStrings(List<?> list) {  
    for (Iterator<?> i = list.iterator(); i.hasNext(); ) {  
        System.out.println(i.next());  
    }  
}
```

Creating a ArrayList the new way

- Specify, in angle brackets after the name, the type of object that the class will hold
- Examples:
 - `ArrayList<String> vec1 = new ArrayList<String>();`
 - `ArrayList<String> vec2 = new ArrayList<String>(10);`
- To get the old behavior, but without the warning messages, use the `<?>` wildcard
 - Example: `ArrayList<?> vec1 = new ArrayList<?>();`

Accessing with and without generics

- `Object get(int index)`
 - Returns the component at position *index*
- Using `get` the old way:
 - `ArrayList myList = new ArrayList();`
`myList.add("Some string");`
`String s = (String)myList.get(0);`
- Using `get` the new way:
 - `ArrayList<String> myList = new ArrayList<String>();`
`myList.add("Some string");`
`String s = myList.get(0);`
- Notice that casting is no longer necessary when we retrieve an element from a “genericized” `ArrayList`

Generics and Inheritance

- Suppose you want to restrict the type parameter to express some restriction on the type parameter
- This can be done with a notion of subtypes
- expressed in Java using inheritance
- So it's a natural combination to combine inheritance with generics
- A few examples follow

Parameterized Classes in Methods

- A parameterized class is a type just like any other class.
- It can be used in method input types and return types.

Parameterized Classes in Methods

- If a class is parameterized, that type parameter can be used for any type declaration in that class, e.g:

```
public class Box<E>
```

```
{E data;
```

```
public Box(E data) {this.data = data;}
```

```
public E getData() {return data;}
```

```
public void copyFrom(Box<E> b)
```

```
{this.data = b.getData();}
```

Bounded Parameterized Types

- Sometimes we want restricted parameterization of classes.
- We want a box, called MathBox that holds only Number objects.
- We can't use `Box<E>` because E could be anything.
- We want E to be a subclass of Number.

Bounded Parameterized Types

```
public class MathBox<E extends Number> extends  
    Box<Number>  
{  
    public MathBox(E data)  
    {  
        super(data);  
    }  
    public double sqrt()  
    {  
        return Math.sqrt(getData().doubleValue())  
    }  
}
```

Bounded Parameterized Types

- The `<E extends Number>` syntax means that the type parameter of `MathBox` must be a subclass of the `Number` class
 - We say that the type parameter is **bounded**

new `MathBox<Integer>(5); //Legal`

new `MathBox<Double>(32.1); //Legal`

new `MathBox<String>("No good!"); //Illegal`

Bounded Parameterized Types

- Java allows multiple inheritance in the form of implementing multiple interfaces, so multiple bounds may be necessary to specify a type parameter. The following syntax is used then:

<T extends A & B & C & ...>

- Example**

```
interface A {...}
```

```
interface B {...}
```

```
class MultiBounds<T extends A & B> {
```

```
...
```

```
}
```

Implementing generics

```
// a parameterized (generic) class  
public class name<Type> {  
or  
public class name<Type, Type, ..., Type> {
```

- By putting the **Type** in `< >`, you are demanding that any client that constructs your object must supply a type parameter.
 - You can require multiple type parameters separated by commas.
- The rest of your class's code can refer to that type by name.
 - The convention is to use a 1-letter name such as:
T for Type, E for Element, N for Number, K for Key, or V for Value.
- The type parameter is *instantiated* by the client. (e.g. `E → String`)

Generics and arrays

```
public class Foo<T> {  
    private T myField;           // ok  
    private T[] myArray;        // ok  
  
    public Foo(T param) {  
        myField = new T();      // error  
        myArray = new T[10];    // error  
    }  
}
```

- You cannot create objects or arrays of a parameterized type.

Generics/arrays, fixed

```
public class Foo<T> {  
    private T myField;           // ok  
    private T[] myArray;        // ok  
  
    public Foo(T param) {  
        myField = param;        // ok  
        T[] a2 = (T[]) (new Object[10]); // ok  
    }  
}
```

- But you can create variables of that type, accept them as parameters, return them, or create arrays by casting `Object[]`.
 - Casting to generic types is not type-safe, so it generates a warning.

Generic methods

```
public static <Type> returnType  
name (params) {
```

- When you want to make just a single (often static) method generic in a class, precede its return type by type parameter(s).

```
public class Collections {  
    ...  
    public static <T> void copy(List<T> dst,  
List<T> src) {  
        for (T t : src) {  
            dst.add(t);  
        }  
    }  
}
```

Bounded type parameters

<Type extends SuperType>

- An upper bound; accepts the given supertype or any of its subtypes.
- Works for multiple superclass/interfaces with & :

<Type extends ClassA & InterfaceB & InterfaceC & ...>

<Type super SuperType>

- A lower bound; accepts the given supertype or any of its supertypes.

- Example:

```
// tree set works for any comparable type
public class TreeSet<T extends Comparable<T>>
{
    . . .
}
```


Complex bounded types

- `public static <T extends Comparable<T>>
 T max(Collection<T> c)`
 - Find max value in any collection, if the elements can be compared.
- `public static <T> void copy(
 List<T2 super T> dst, List<T3 extends T> src)`
 - Copy all elements from src to dst. For this to be reasonable, dst must be able to safely store anything that could be in src. This means that all elements of src must be of dst's element type or a subtype.
- `public static <T extends Comparable<T2 super T>>
 void sort(List<T> list)`
 - Sort any list whose elements can be compared to the same type or a broader type.

Generics and subtyping

- Is `List<String>` a subtype of `List<Object>`?
- Is `Set<Giraffe>` a subtype of `Collection<Animal>`?
- **No.** That would violate the Liskov Substitutability Principle.
 - If we could pass a `Set<Giraffe>` to a method expecting a `Collection<Animal>`, that method could add other animals.

```
Set<Giraffe> set1 = new HashSet<Giraffe>();  
Set<Animal>  set2 = set1;                // illegal  
...  
set2.add(new Zebra());  
Giraffe geoffrey = set1.get(0);         // error
```

Wildcards

- ? indicates a *wild-card* type parameter, one that can be any type.
 - `List<?> list = new List<?>();` `// anything`
- Difference between `List<?>` and `List<Object>` :
 - ? can become any particular type; `Object` is just one such type.
 - `List<Object>` is restrictive; wouldn't take a `List<String>`
- Difference btwn. `List<Foo>` and `List<? extends Foo>`:
 - The latter binds to a particular `Foo` subtype and allows ONLY that.
 - e.g. `List<? extends Animal>` might store only Giraffes but not Zebras
 - The former allows anything that is a subtype of `Foo` in the same list.
 - e.g. `List<Animal>` could store both Giraffes and Zebras

Generics with subclass

```
public double areaOfCollection (Collection<?  
extends Shape> c)  
{  
    double sum = 0.0;  
    for (Shape s : c)  
        sum += s.getArea();  
}
```

Generics with Comparator

Comparator interface is also generic

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
    boolean equals(Object o);  
}
```

Create a comparator `CompareByLength` to sort Strings by length in x

Generics with Comparator

```
public class CompareByLength implements  
Comparator<String> {  
    int compare(String o1, String o2)  
    {return o1.length() - o2.length();  
  
}
```

Generics with Comparator

- Method that takes an array of objects and a collection and puts all objects in the array into the collection

```
static <T> void fromArrayToCollection(T[] a,  
Collection<T> c) {  
    for (T o : a) {  
        c.add(o);  
    }  
}
```

Generics and casting

- Casting to generic type results in a warning.

```
List<?> l = new ArrayList<String>();  
// ok  
List<String> ls = (List<String>) l;  
// warn
```

- The compiler gives an unchecked warning, since this isn't something the runtime system is going to check for you.
- Usually, if you think you need to do this, you're doing it wrong.

- The same is true of type variables:

```
public static <T> T badCast(T t,  
Object o) {  
    return (T) o;    // unchecked warning  
}
```


Java I/O and Files

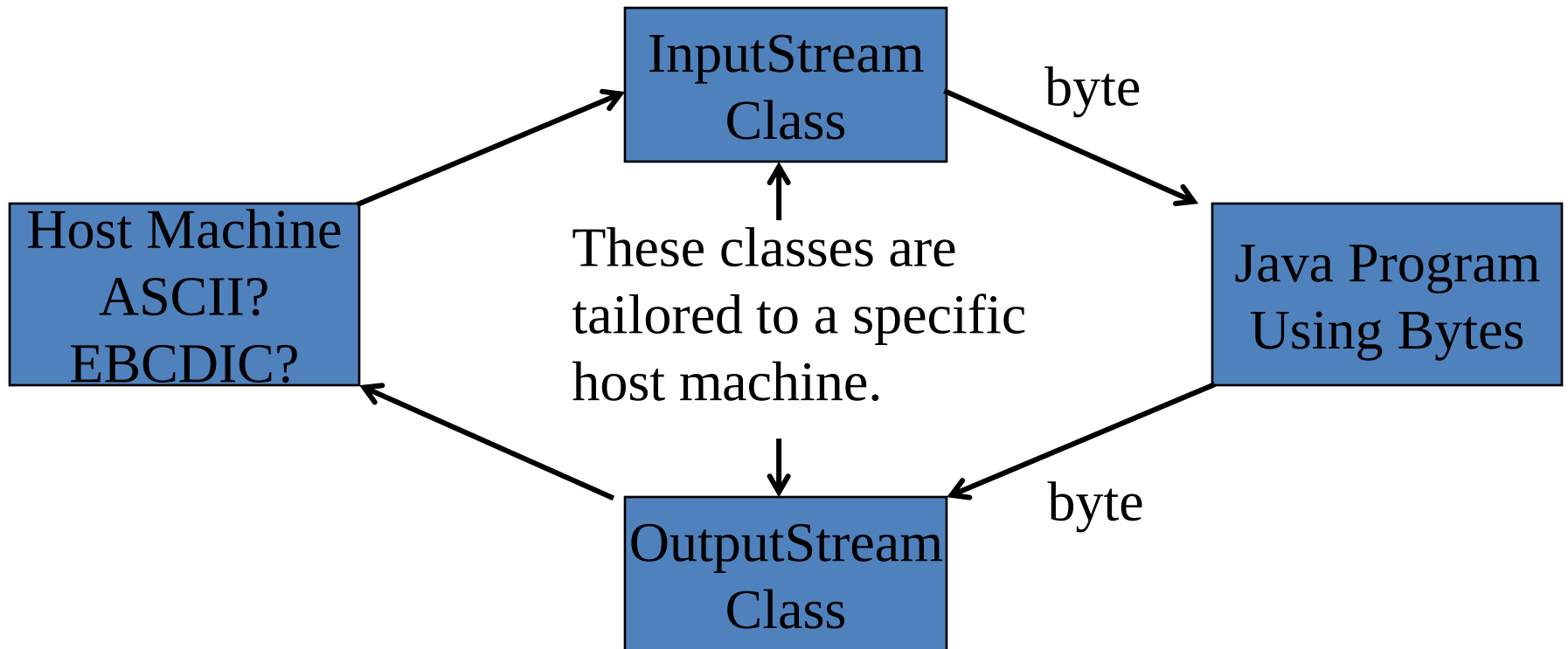
Why Is Java I/O Hard?

- Java is intended to be used on many very different machines, having
 - different character encodings (ASCII, EBCDIC, 7- 8- or 16-bit...)
 - different internal numerical representations
 - different file systems, so different filename & pathname conventions
 - different arrangements for EOL, EOF, etc.
- The Java I/O classes have to “stand between” your code and all these different machines and conventions.

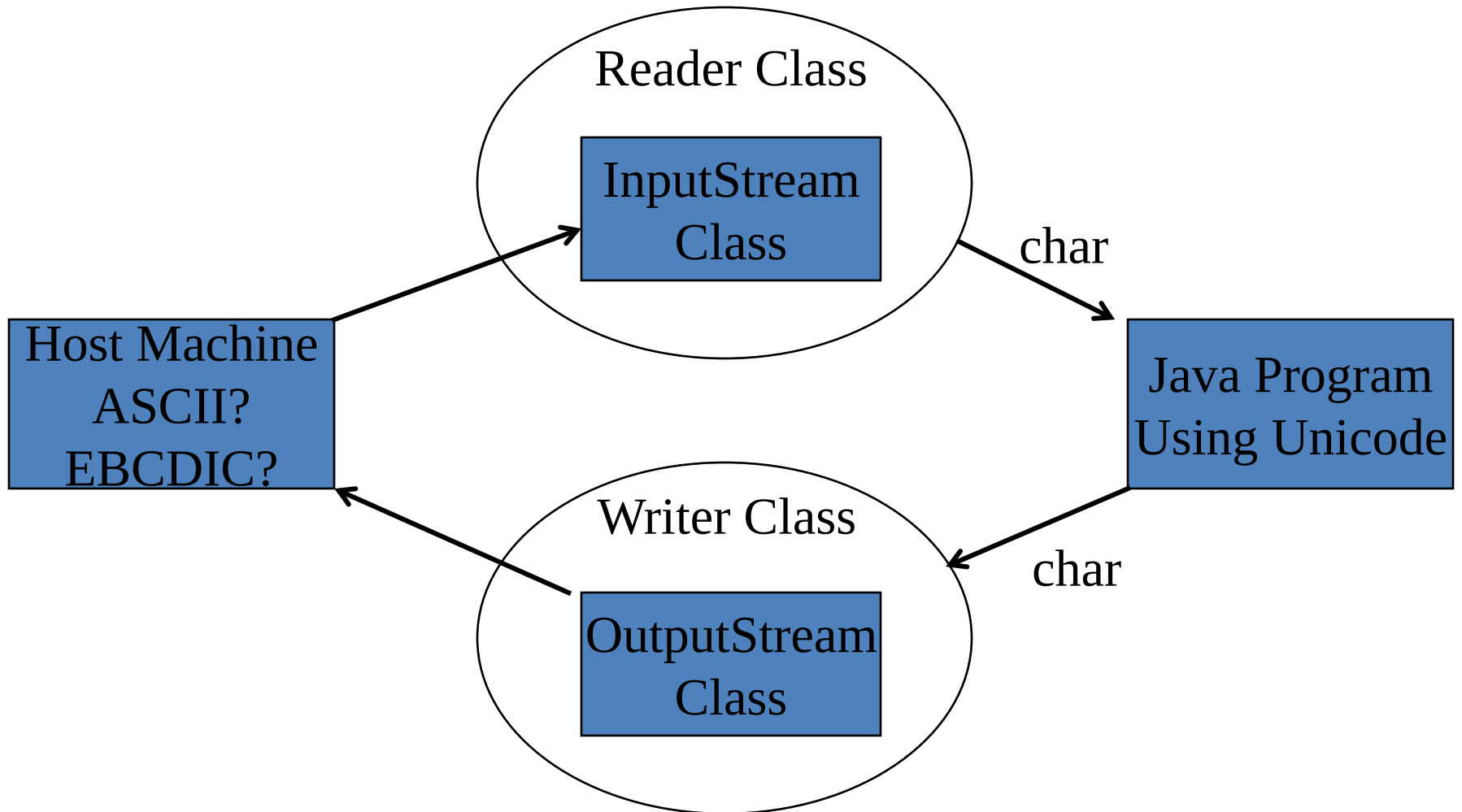
Java's Internal Characters

- Unicode. 16-bit. Good idea.
- primitive type **char** is 16-bit.
- Reading /writing from a file using 8-bit ASCII characters (for example) requires conversion.
- But binary files (e.g., graphics) are “byte-sized”, so there is a primitive type **byte**.
- So Java has two systems to handle the two different requirements.
- Both are in **java.io**, so import this *always*!

Streams



Readers and Writers



Streams

- A “stream” is an abstraction derived from sequential input or output devices.
- Streams apply not just to files, but also to actual IO devices, Internet streams, and so on.

- In reality streams are *buffered* : it is not practical to read or write one character at a time.

java.io

BufferedInputStream	InputStream	
BufferedOutputStream	InputStreamReader	RandomAccessFile
BufferedReader	LineNumberInputStream	Reader
BufferedWriter	LineNumberReader	SequenceInputStream
ByteArrayInputStream	ObjectInputStream	SerializablePermission
ByteArrayOutputStream	ObjectInputStream.GetField	StreamTokenizer
CharArrayReader	ObjectOutputStream	StringBufferInputStream
CharArrayWriter	ObjectOutputStream.PutField	StringReader
DataInputStream	ObjectStreamClass	StringWriter
DataOutputStream	ObjectStreamField	Writer
File	OutputStream	
FileDescriptor	OutputStreamWriter	
FileInputStream	PipedInputStream	
FileOutputStream	PipedOutputStream	
FilePermission	PipedReader	
FileReader	PipedWriter	
FileWriter	PrintStream	
FilterInputStream	PrintWriter	
FilterOutputStream	PushbackInputStream	
FilterReader	PushbackReader	
FilterWriter		

java.io

- Uses four hierarchies of classes rooted at `Reader`, `Writer`, `InputStream`, `OutputStream`.
- Has a special stand-alone class `RandomAccessFile`.

java.io

- `BufferedReader` and `RandomAccessFile` are the only classes that have a method to read a line of text, **`readLine`**.
- `readLine` returns a `String` or `null` if the end of file has been reached.

“Wrapping”

- Input comes in through a stream (bytes)
- we want to read characters, so “wrap” the stream in a Reader to get characters.

```
public static void main(String[] args) {  
    InputStreamReader isr = new InputStreamReader(System.in);  
    int c;  
    try {  
        while ((c = isr.read()) != -1)  
            System.out.println((char) c);  
    }  
    catch(IOException e) {  
    }  
}
```

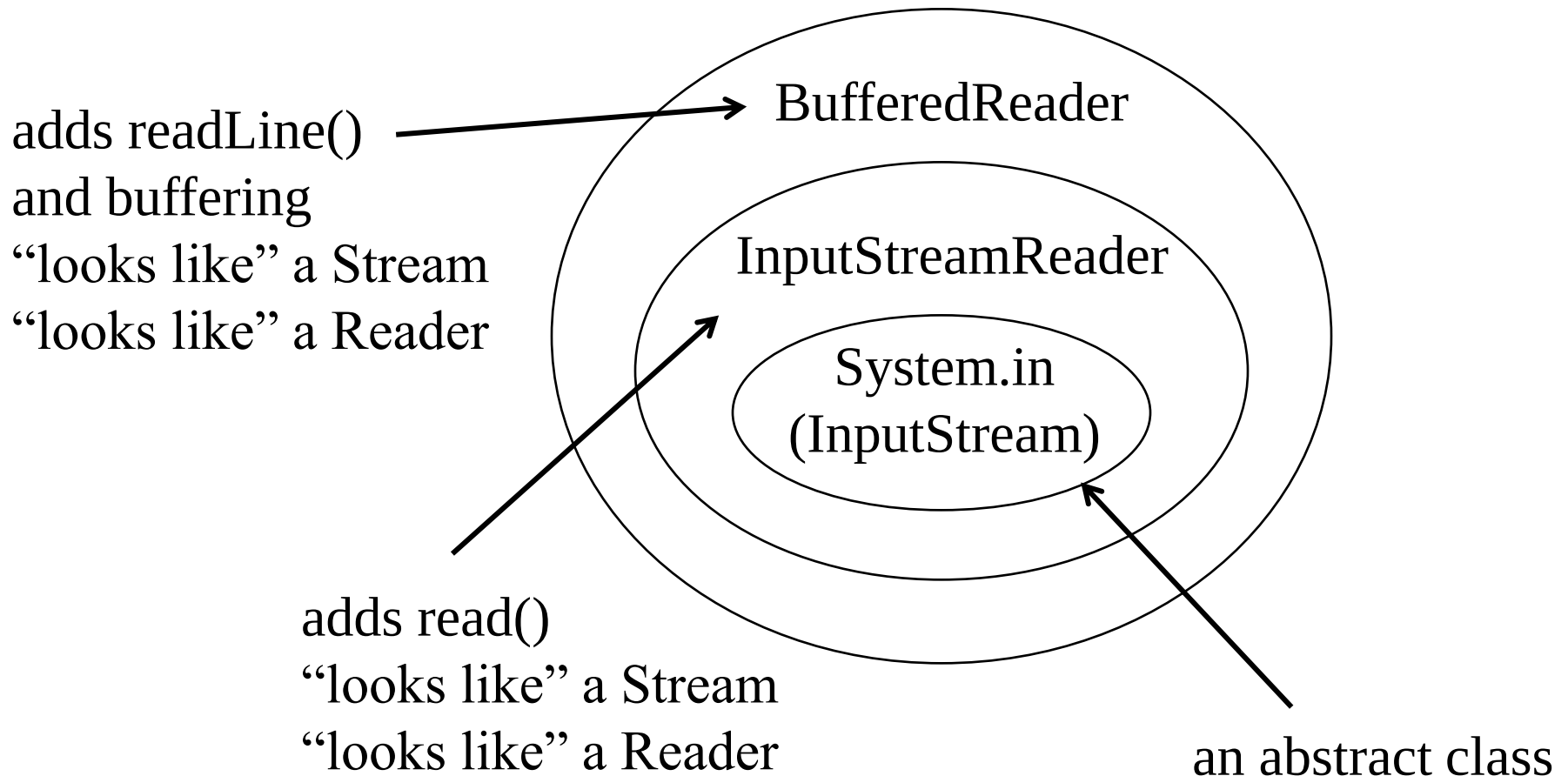
InputStreamReader

- This is a bridge between bytes and chars.
- The **read()** method returns an **int**, which must be cast to a **char**.
- **read()** returns -1 if the end of the stream has been reached.
- We need more methods to do a better job!

Use a **BufferedReader**

```
public static void main(String[] args) {  
    BufferedReader br =  
        new BufferedReader(new InputStreamReader(System.in));  
    String s;  
    try {  
        while ((s = br.readLine()).length() != 0)  
            System.out.println(s);  
    }  
    catch(IOException e) {  
    }  
}
```

“Transparent Enclosure”



java.io

- “Throws” *checked exceptions* try-catch statement should be used to handle code that throws checked exceptions.

Byte streams

- Two parent abstract classes: **InputStream** and **OutputStream**
- Reading bytes:
 - **InputStream** class defines an abstract method
public abstract int read() throws IOException
 - Designer of a concrete input stream class overrides this method to provide useful functionality.
 - E.g. in the **FileInputStream** class, the method reads one byte from a file
 - **OutputStream** class defines an abstract method
public abstract void write(int b) throws IOException

Example code1:

```
import java.io.*;
class CountBytes {

    public static void main(String[] args)
        throws IOException {
        FileInputStream in = new
            FileInputStream(args[0]);
        int total = 0;
        while (in.read() != -1)
            total++;
        in.close(); //Always close streams
        System.out.println(total + "bytes");
    }
}
```

Example code2:

```
import java.io.*;

class TranslateByte {
    public static void main(String[] args)
        throws IOException {

        byte from = (byte)args[0].charAt(0);
        byte to = (byte)args[1].charAt(0);
        byte x;

        while((x = System.in.read()) != -1)
            System.out.write(x == from ? to :
x) ;

    }
}
```

If you run "java TranslateByte b B" and enter text
bigboy via the keyboard the output will be: BigBoy

Character streams

- Two parent `abstract` classes for characters: **Reader** and **Writer**.
- The standard streams—`System.in`, `System.out` and `System.err`—existed before the invention of character streams. So they are byte streams though logically they should be character streams.

Stream Objects

All Java programs make use of standard stream objects

- `System.in`
 - To input bytes from keyboard
- `System.out`
 - To allow output to the screen
- `System.err`
 - To allow error messages to be sent to screen

Conversion between byte and character streams

```
-public InputStreamReader(InputStream in)  
-public OutputStreamWriter(OutputStream  
out)
```

- `read` method of `InputStreamReader`
 - read bytes from their associated `InputStream` and convert them to characters
- `write` method of `OutputStreamWriter`
 - take the supplied characters, convert them to bytes and write them to its associated `OutputStream`

Reading Characters

```
Import java.io.*;
class Reading{
    public static void main(String a[])throws IOException
    {
        char c;
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in))
        do{
            c=(char)br.read();
            System.out.println(c);
        } while(c!='q');
    }
}
```

Files

- A file is a collection of data in mass storage.
- A data file is not a part of a program's source code.
- The same file can be read or modified by different programs.
- The program must be aware of the format of the data in the file.

Files

- The file system is maintained by the operating system.
- The system provides commands and/or GUI utilities for viewing file directories and for copying, moving, renaming, and deleting files.
- The system also provides “core” functions, callable from programs, for reading and writing directories and files.

Some Classes for File Handling

- **FileInputStream** and **FileOutputStream** perform file input and output respectively
- **FileReader** and **FileWriter**
 - are used to read and write characters to a file
- **DataInputStream** and **DataOutputStream**
 - allow a program to read and write binary data using an **InputStream** and **OutputStream** respectively
- **ObjectInputStream** and **ObjectOutputStream**
 - deal with Objects implementing **ObjectInput** and **ObjectOutput** interfaces respectively

Reading From a File:

FileInputStream

- Its constructor takes a string containing the file pathname.

```
public static void main(String[] args) throws IOException {  
    InputStreamReader isr = new  
        InputStreamReader(new FileInputStream("FileInput.java"));  
    int c;  
    while ((c = isr.read()) != -1)  
        System.out.println((char) c);  
    isr.close();  
}
```

Reading From a File (cont.)

- -1: indicates end of the file.
- absolute path name is safer.
- The **read()** method can throw an **IOException**, and the **FileInputStream** constructor can throw a **FileNotFoundException**

The **File** Class

- Think of this as holding a file *name*, or a list of file *names* (as in a directory).
- You create one by giving the constructor a pathname, as in

```
File f = new File("d:/www/java/week10/DirList/.");
```
- This is a directory, so now the **File f** holds a list of (the names of) files in the directory.
- It's straightforward to print them out.

Listing Files

```
import java.io.*;
import java.util.*;
public class DirList {
    public static void main(String[] args) {
        File path = new File(".");
        String[] list;
        System.out.println(path.getAbsolutePath());
        list = path.list();
        for (int i = 0; i < list.length; i++)
            System.out.println(list[i]);
    }
}
```

Important Point

Data must be read in in the same form that it is written out to a file

Writing

```
output = new ObjectOutputStream  
        ( new FileOutputStream(filename) );  
output.writeObject( objectname );  
output.close( );
```

Reading

```
input = new ObjectInputStream  
        new FileInputStream( filename ) );  
record = ( ObjectType ) input.readObject( );  
input.close( );
```

The `File` class

- The `File` class is particularly useful for retrieving information about a file or a directory from a disk.
 - A `File` object actually represents a path, not necessarily an underlying file
 - A `File` object doesn't open files or provide any file-processing capabilities
- Three constructors
 - `public File( String name)`
 - `public File( String pathToName, String name)`
 - `public File( File directory, String name)`

Methods in the File class

- boolean canRead() / boolean canWrite()
- boolean exists()
- boolean isFile() / boolean isDirectory() / boolean isAbsolute()
- String getAbsolutePath() / String getPath()
- String getParent()
- String getName()
- long length()
- long lastModified()

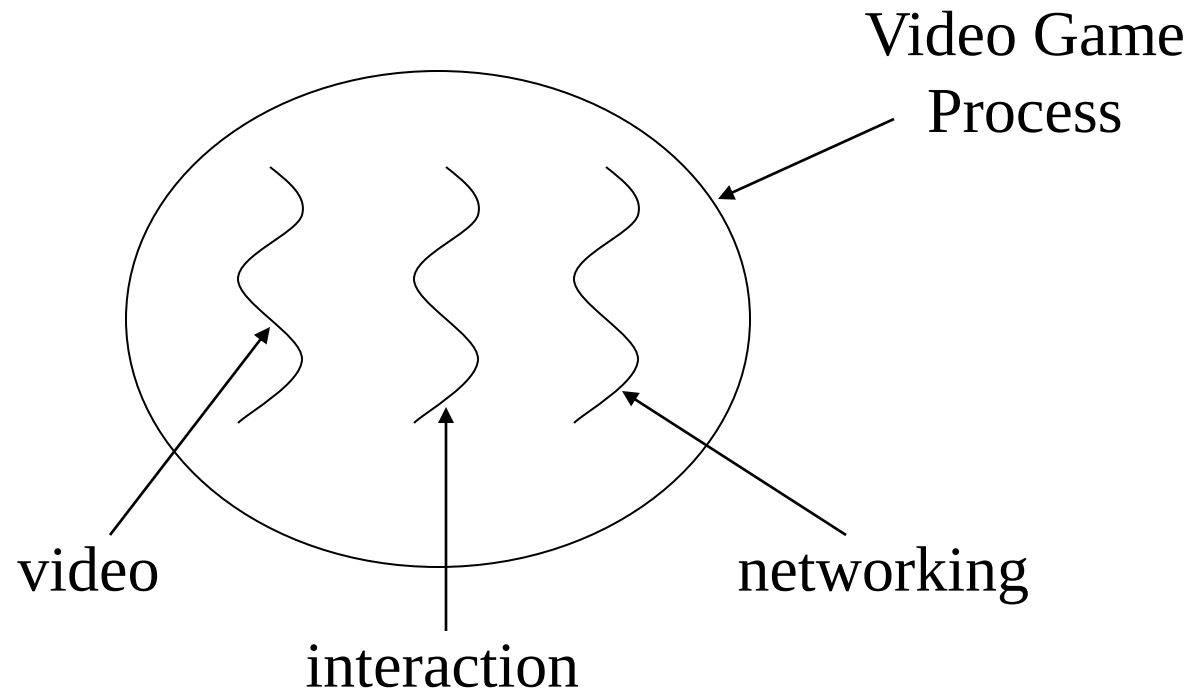
Java Threads

Introduction to multithreading in Java

What is a Thread?

- Individual and separate unit of execution that is part of a process
 - multiple threads can work together to accomplish a common goal
- Video Game example
 - one thread for graphics
 - one thread for user interaction
 - one thread for networking

What is a Thread?



Advantages

- easier to program
 - 1 thread per task
- can provide better performance
 - thread only runs when needed
 - no polling to decide what to do
- multiple threads can share resources
- utilize multiple processors if available

Disadvantage

- multiple threads can lead to deadlock
 - much more on this later
- overhead of switching between threads

Creating Threads (method 1)

- extending the **Thread** class
 - must implement the *run()* method
 - thread ends when *run()* method finishes
 - call *.start()* to get the thread ready to run

Creating Threads Example 1

```
class Output extends Thread {  
    private String toSay;  
    public Output(String st) {  
        toSay = st;  
    }  
    public void run() {  
        try {    for(;;) {  
                    System.out.println(toSay);  
                    sleep(1000);  
                }  
        } catch (InterruptedException e) {  
            System.out.println(e);  
        }  
    }  
}
```


Example 1 (continued)

```
class Program {  
    public static void main(String [] args) {  
        Output thr1 = new Output("Hello");  
        Output thr2 = new Output("There");  
        thr1.start();  
        thr2.start();  
    }  
}
```

- main thread is just another thread (happens to start first)
- main thread can end before the others do
- any thread can spawn more threads

Creating Threads (method 2)

- implementing **Runnable interface**
 - virtually identical to extending Thread class
 - must still define the *run()* method
 - setting up the threads is slightly different

Creating Threads Example 2

```
class Output implements Runnable {  
    private String toSay;  
    public Output(String st) {  
        toSay = st;  
    }  
    public void run() {  
        try {  
            for(;;) {  
                System.out.println(toSay);  
                Thread.sleep(1000);  
            }  
        } catch (InterruptedException e) {  
            System.out.println(e);  
        }  
    }  
}
```

Example 2 (continued)

```
class Program {  
    public static void main(String [] args) {  
        Output out1 = new Output("Hello");  
        Output out2 = new Output("There");  
        Thread thr1 = new Thread(out1);  
        Thread thr2 = new Thread(out2);  
        thr1.start();  
        thr2.start();  
    }  
}
```

- main is a bit more complex
- everything else identical for the most part

Advantage of Using Runnable

- remember - can only extend one class
- implementing runnable allows class to extend something else

Controlling Java Threads

- `_.start()`: begins a thread running
- `_.stop()`: kills a specific thread (deprecated)
- `_.join()`: wait for specific thread to finish

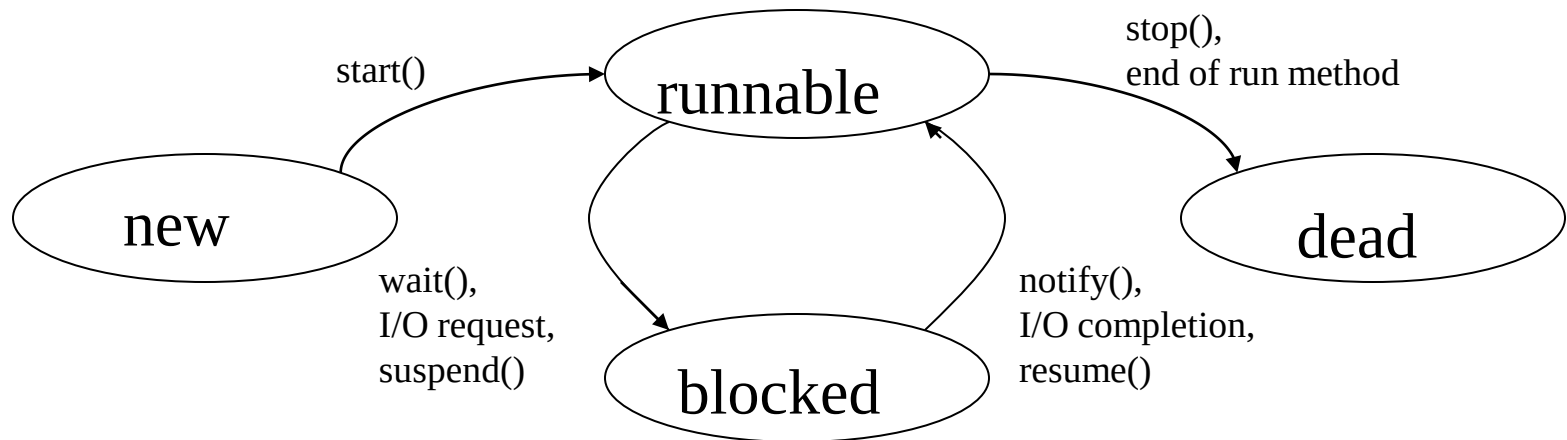
Java Thread Scheduling

- highest priority thread runs
 - if more than one, arbitrary
- *yield()*: current thread gives up processor so another of equal priority can run
 - if none of equal priority, it runs again
- *sleep(msec)*: stop executing for set time
 - lower priority thread can run

States of Java Threads

- 4 separate states
 - new: just created but not started
 - runnable: created, started, and able to run
 - blocked: created and started but unable to run because it is waiting for some event to occur
 - dead: thread has finished or been stopped

States of Java Threads



Java Thread Example 1

```
class Job implements Runnable {  
    private static Thread [] jobs = new Thread[4];  
    private int threadID;  
    public Job(int ID) {  
        threadID = ID;  
    }  
    public void run() { do something }  
    public static void main(String [] args) {  
        for(int i=0; i<jobs.length; i++) {  
            jobs[i] = new Thread(new Job(i));  
            jobs[i].start();  
        }  
        try {  
            for(int i=0; i<jobs.length; i++) {  
                jobs[i].join();  
            }  
        } catch (InterruptedException e) { System.out.println(e); }  
    }  
}
```